Chapter 2

Basics of Numerical Programming

Outline of Section

- How information is stored in a computer
- A review of the Taylor series
- The nature of computational errors
- Using natural units

2.1 Numerical Formats and Types

Here we discuss how computers store information at the hardware level. Computer memory is a succession of "bits" that can take one of two values: "off" and "on", or equivalently, "0" and "1". Since only a finite number of bits are available to represent the information, a way of representing this information efficiently has to be chosen.

And whilst "information" can take many forms, ultimately it is numbers that are handled inside a computer. Any other information, such as letters of an alphabet, are converted to/from numbers using what are effectively look-up tables such as the Unicode Standard.

We say "hardware-level" since it is these numbers that are directly manipulated by the CPUs that access the memory, and the entire system (CPU and memory) can be optimised for speed and efficiency for the basic mathematical operations that are needed, such as addition, subtraction, multiplication and division. Other numerical formats exist which are handled by software, at a higher level than the hardware. These include e.g. complex numbers, internally representing them using pairs of real numbers.

The two different basic types of numbers which are normally stored directly in computers are *integers* and *real* numbers. These are the ones most used in this course and each is described below.

2.1.1 Integer numbers

Integers are relatively simple—a binary representation of any integer, optionally with its sign, carries all the information possessed by the original number, so it is only the number of bits (called the *precision* or *width*) that are allocated to store the number that can vary. Modern systems often use 32-bit or 64-bit integers, which define the limits of the size of the integers that can be stored. How to access these different integer *types*, and the number of bits that are available, are hardware- and language-dependent.

It is up to the user to make sure enough bits are being used to their specific purposes. An unsigned integer is stored in terms of bits as a straightforward binary number. Specifically, N bits can store unsigned integers from 0 to $2^N - 1$. Signed integers use one bit to indicate the sign, so the non-negative values they can represent only go from 0 to $2^{N-1} - 1$. However, they can also go down to -2^{N-1}, which means for N bits, there are the same number of signed integers as unsigned, i.e. 2^N in total for both. The negative values are represented in bits using "two's complement", which means taking the bit pattern of the equivalent unsigned positive value, inverting all the bits

(so all 0's go to 1, and all 1's go to 0) and adding 1. For example, four binary bits gives $2^4 = 16$ possible values and allows unsigned decimal integers from 0 to $2^4-1 = 15$ or signed decimal integers from $-2^3 = -8$ to $2^3-1 = 7$ to be represented. The table shows all the possible 16 values in binary and in decimal for the two integer representations.

Binary	Unsigned decimal	Signed decimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	−8
1001	9	−7
1010	10	−6
1011	11	−5
1100	12	−4
1101	13	−3
1110	14	−2
1111	15	−1

Although the places where integers can be used in numerical computing are limited, it is a good idea to use them where possible. Their representation is exact, so there are no rounding errors (unlike the real numbers described below). All arithmetic calculations (except division) are also exact, as long as the result is within the integer range. However, if the calculation goes outside the range, all higher bits are discarded so the result is *very* wrong; hence take care. The one exception is `Python3`, where the amount of memory (i.e. bits) used is automatically increased as required, so there is effectively no range limit for integers from a user's perspective.

2.1.2 Real numbers

Storing a real number is a much more complicated affair, and the computing world has settled on the IEEE-754 standard, which describes how these can be represented in a computer.

Even limited to a finite range, it is clear there are an infinite number of real values so only some subset can be represented with a finite number of bits. Any other real value has to be rounded to the nearest representable value, so in general any uses of real numbers will incur a **rounding error**. The actual numbers which are representable, i.e. can be specified exactly and so have no rounding error, are a subset of integers or of rational numbers where the denominator is a power of two. Note, the real values which happen to be integers are stored differently from the integer representation discussed above.

There is another way to consider the real number representation, which is as *floating-point* numbers. The concept of floating-point representations of real numbers is similar to the exponential notation that is a compact way to write down numbers with a certain number of significant digits and an arbitrary magnitude: for example 1.541×10^{-8} or -9.34031×10^{22}. The 1.541 or −9.34031 are called the *mantissa*, while the −8 or 22 are called the *exponent*, which here is in powers of ten. Note, with a fixed number of digits in the mantissa, e.g. the first could be written as $1541 \times 10^{-11} = 1541/10^{11}$, showing it can be considered as a rational number.

The same works for binary but in this case computer exponents are in powers of two, e.g. 1.01×2^1, where the mantissa is written in binary. This could also be written as $101/2^1$. This last expression makes it easier to see that this number is equal to decimal $5/2 = 2.5$. In a similar way, any such floating point number with a fixed number of bits in the mantissa can always be written as a rational number.

The convention when writing binary mantissas as floating point is to have a leading 1 bit in the mantissa before the decimal point. Because this is usually present, it is not actually stored in the computer memory. It is called the *implied* (or *phantom* or *hidden*) bit. The one exception is for the lowest possible exponent value, where the mantissa is interpreted as having an implied 0 bit; this allows values down to zero to be represented. The numbers with an implied 1 bit are called *normal* while the ones with an implied 0 are called *denormal*.

The highest possible exponent value is also an exception; it is used to indicate invalid values. A value is said to *overflow* if it is too high to fit into the real number range. These values are replaced by the symbol `Inf` (or `inf`) in some languages. In addition, the IEEE standard uses `NaN` (or `nan`), i.e. Not-a-Number, to replace $0/0$, $\sqrt{-1}$, etc. Values less than the smallest non-zero value are called *underflows* and usually result in the value being set to zero.

The ways of considering the real values as rationals or floating point expressions are shown in the table below, which shows all possible values for real numbers using 4 bits. The first four values are denormal, the next eight are normal and the last four are invalid. Note that the actual power of two is not given by the binary value of the exponent bits; there is an offset between them. If signed real values are needed, a fifth bit could be used to indicate if the value is positive or negative.

Exponent	Implied bit	Stored mantissa	Integer mantissa	Rational	Real mantissa	Floating point	Real
00	0	00	000	$0/2^2$	0.00	0.00×2^0	0.00
00	0	01	001	$1/2^2$	0.01	0.25×2^0	0.25
00	0	10	010	$2/2^2$	0.10	0.50×2^0	0.50
00	0	11	011	$3/2^2$	0.11	0.75×2^0	0.75
01	1	00	100	$4/2^2$	1.00	1.00×2^0	1.00
01	1	01	101	$5/2^2$	1.01	1.25×2^0	1.25
01	1	10	110	$6/2^2$	1.10	1.50×2^0	1.50
01	1	11	111	$7/2^2$	1.11	1.75×2^0	1.75
10	1	00	100	$4/2^1$	1.00	1.00×2^1	2.00
10	1	01	101	$5/2^1$	1.01	1.25×2^1	2.50
10	1	10	110	$6/2^1$	1.10	1.50×2^1	3.00
10	1	11	111	$7/2^1$	1.11	1.75×2^1	3.50
11		00					“Inf”
11		01					“NaN”
11		10					“NaN”
11		11					“NaN”

In reality, as for integers, 32 and 64 bits are normally used to store real numbers in *single* and *double* precision, respectively, with the bits being allocated to the sign, mantissa and the exponent according to some agreed standard, as shown in the table below.

	Sign	Mantissa	Exponent
Single (32 bits)	1 bit	23 bits	8 bits
Double (64 bits)	1 bit	52 bits	11 bits

(2.1)

The number of bits allocated to the mantissa affects the number of significant digits that can be represented. The above results in a range of allowed numbers of roughly $2^{\pm 127} \sim 10^{\pm 38}$ and $2^{\pm 1023} \sim 10^{\pm 308}$ for single and double precision respectively. Absolute values smaller or greater than these will cause an underflow or overflow, respectively.

One of the best places to read more about that is in “What Every Computer Scientist Should Know About Floating-Point Arithmetic”, by David Goldberg, 1991. Online full text is available on the Library web site.

2.2 Review of Taylor series

Functions and their derivatives are often approximated as truncated series. A function $f(x)$ can be expanded around the point $x = a$ to n^{th} order as

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{1}{2!}f''(a)(x-a)^2 + ... + \frac{1}{n!}f^{n\prime}(a)(x-a)^n, \tag{2.2}$$

where $f^{n\prime}(a)$ is the n^{th} derivative of the function evaluated at the point $x = a$.

In practice, it will be more useful to expand functions as $f(x+h)$ around the point x. This is because we will be interested in the value of the function is a *small* step h away from the point x where the value of the function $f(x)$ is already known. In this notation, the Taylor series can be written as

$$f(x+h) \approx f(x) + f'(x)\,h + \frac{1}{2!}f''(x)\,h^2 + ... + \frac{1}{n!}f^{n\prime}(x)\,h^n \ \ldots \tag{2.3}$$

We cannot calculate all the infinite number of terms, so the function can be written as an n^{th} order expansion plus a remainder term which sums up all the remaining terms to infinite order

$$\boxed{f(x+h) = \sum_{i=0}^{i=n} \frac{1}{i!}f^{i\prime}(x)h^i + R_n(x,h)} \qquad \text{where} \qquad R_n(x,h) \equiv \sum_{i=n+1}^{i=\infty} \frac{1}{i!}f^{i\prime}(x)h^i\,. \tag{2.4}$$

Using the **Mean Value Theorem**, it can be shown that there is a (generally unknown) value ξ which lies somewhere in the interval between x and $x+h$ for which

$$\boxed{R_n(x,h) = \frac{1}{(n+1)!}f^{n+1\prime}(\xi)h^{n+1} \qquad \text{where} \qquad x \le \xi \le x+h} \tag{2.5}$$

Note the remainder looks like the $(n+1)^{\text{th}}$ term of the expansion, i.e. the first discarded term, but the derivative is evaluated at the point ξ, not x. This tells us the error, i.e. the remainder, is $\propto h^{n+1}$, although the constant of proportionality is not known. This is a useful way of expressing the error in the truncated series expansion for what follows. Remember that in order to possess an n^{th} order Taylor expansion, the function has to be n times differentiable (and $n+1$ times, if we want the remainder term).

For functions of two independent variables, say x and y, the Taylor series is

$$f(x+h_x, y+h_y) \approx \sum_{i=0}^{i=n} \frac{1}{i!}\left(h_x\frac{\partial}{\partial x} + h_y\frac{\partial}{\partial y}\right)^i f(x,y) \tag{2.6}$$

where the differential operator $(\ldots)^i$ can understood by expanding as a binomial. E.g. for $i = 2$

$$\left(h_x\frac{\partial}{\partial x} + h_y\frac{\partial}{\partial y}\right)^2 = h_x^2\frac{\partial^2}{\partial x^2} + 2h_xh_y\frac{\partial^2}{\partial x\partial y} + h_y^2\frac{\partial^2}{\partial y^2} \tag{2.7}$$

This operates on the function and the result is then evaluated at (x,y). The remainder term is similar to Eq. (2.5), in that it involves $(n+1)^{\text{th}}$ order partial derivatives of f evaluated at the point (ξ,η), where $x \le \xi \le x+h_x$ and $y \le \eta \le y+h_y$. Eq. (2.6) can be generalised to more independent variables by adding the relevant partial derivatives (e.g. $\partial/\partial z$ or $\partial/\partial t$) into the $(\ldots)^i$ term and using a multinomial expansion.

2.3 Numerical Accuracy and Errors

There are many sources which lead to a loss of accuracy and other errors that are inevitably introduced when performing mathematical calculations using numbers with finite representations. Here we describe the basic categories that these fall into.

2.3.1 Calculational failures

A calculation which results in an integer or real value which is outside the representable range will give a calculational failure. For integers, these values can be completely wrong although the program will often continue without giving any indication of a problem. For real numbers, a value which is too small gives an underflow and will be set to zero. Depending on the calculation, this may or may not be a good thing to do. A real number which is too big gives an overflow and results in an invalid value. Any attempt to use such a value usually results in crashing the program. Careful coding and choice of units (see section (2.4)) can avoid these errors.

2.3.2 Rounding errors

A rounding error is inherent to all digital computers. For integers, this arises only when doing integer division resulting in a non-zero remainder, which usually results in rounding towards zero to give an integer. E.g. if `x` is an integer, `x=5/3` results in `x=1`; avoid this in numerical calculations.

Rounding is unavoidable for real numbers. This is because the floating point format used by computers stores any real number with only a finite precision. Hence, even writing `x=2.1` will cause the stored value of `x` not to be exactly 2.1. Numbers are rounded off to the closest representable value. With a 64-bit representation and a values within the normal range, the maximum rounding error is $\pm 2^{-53} \sim 1 \times 10^{-16}$ of the value of the number. This is known as the *machine accuracy* (or *machine epsilon*). This is the precision of the `Python` type '**float**' and the `C++` type '**double**'. A real value in the normal range with a 32-bit representation has a maximum rounding error of $\pm 2^{-24} \sim 6 \times 10^{-8}$ of the value of the number. In `C++`, this type is (slightly confusingly) called '**float**'. A perhaps surprising example of the effect of rounding error is subtracting 1/9 away from 1 nine times, e.g. in `Python`;

```
n=9;  x=1.0; dx=x/n
for i in range(n):
    x=x-dx
print x
```

Rather than obtaining zero, the output (on a Linux machine) is $-$**1.665e$-$16** to 3 d.p. This round-off phenomenon occurs because the variable `dx` $= 1/9 = 0.11\dot{1}$ is only accurate to about 16 significant figures and hence is not exactly 1/9.

2.3.3 Truncation errors

The Taylor series discussed above cannot be used with an infinite number of terms so when we discard higher terms, i.e. truncate the series, it will mean some inaccuracy. This is called a truncation error. However, this can also happen even without us knowing. Even the most precise computer often evaluates functions internally using approximations. These approximations are typically in the form of a series and so can also lead to truncation errors. Most programming languages use some form of power expansion to approximate standard mathematical functions, e.g. the sine function. The Taylor expansion of $\sin(x)$ around $x = 0$ is

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + (-1)^n \frac{x^{2n+1}}{(2n+1)!} + \ldots . \tag{2.8}$$

The computer will typically keep a reasonable number of terms so as to keep the truncation error comparable with the representation accuracy, but it cannot be eliminated completely.

2.3.4 Initial condition errors

Even in the absence of any truncation error or round-off error, an error in defining the starting point of the calculation can cause an error in the result.

For example, when solving for $u(t)$ starting from an initial time t_0, an error on $u(t_0)$ can put the calculation onto a different solution of the equation being solved. This different solution (i.e. the

calculated $u(t)$ curve) may diverge from the intended solution, perhaps more quickly with time (or whatever the independent variable is). Chaotic systems are by their nature particularly sensitive with respect to initial conditions.

2.3.5 Propagation errors

Propagation errors are closely related to initial condition errors. This error is what would be seen in the next step of a calculation given an error present at the current step, *if the rest of the step calculation were able to be done exactly (i.e. without truncation or round-off errors).* The **inherited error** at the current step is the accumulation of errors up to that point, i.e. the propagated error from all previous steps.

If the propagated error increases with each step then the calculation is said to be **unstable**. If it remains constant or decreases then the calculation is **stable**. The stability of a calculation can depend both on the type of equations involved and the numerical methods used. Unstable solutions *can* be used in limited circumstances, but care is needed.

2.4 Natural units

It is very important to use sensible units when calculating values numerically due to the limited range of numbers a computer can deal with. It also helps in understanding the dynamics if we can scale the variables relative to appropriate characteristic units of measure that encapsulate important physical properties of the system. This in turn greatly aids debugging.

The scaling process removes SI units from the variables (leaving them in 'natural' or 'dimensionless' units) and for this reason is also known as *equation non-dimensionalisation.* Thus **natural units**, **scaling** and **non-dimensionalisation** all refer to the same concept.

Consider solving a differential equation. All the independent variables should be scaled. Scaling of the dependent variables is optional, but often insightful; initial values or asymptotic values can be good choices. For example, consider the dimensionful system for $u(t)$ describing exponential decay with a source:

$$\frac{\mathrm{d}u}{\mathrm{d}t} + \alpha\, u \;=\; g \tag{2.9}$$

where α is a decay rate and g a constant growth/source term. Writing the dependent variable SI dimensions as $[u]$, then $[\alpha] = \mathrm{s}^{-1}$ and $[g] = [u]/\mathrm{s}$. For the independent variable t, we can then define a scaled dimensionless variable $\hat{t} = \alpha\, t$ such that[1]

$$\frac{\mathrm{d}}{\mathrm{d}t} = \frac{\mathrm{d}\hat{t}}{\mathrm{d}t}\frac{\mathrm{d}}{\mathrm{d}\hat{t}} = \alpha\frac{\mathrm{d}}{\mathrm{d}\hat{t}} \tag{2.10}$$

and the equation becomes

$$\frac{\mathrm{d}u}{\mathrm{d}\hat{t}} + u \;=\; \frac{g}{\alpha} \equiv G, \tag{2.11}$$

where $[G] = [u]$. We could choose to go further and scale the dependent variable u also. Given that the steady-state solution (i.e. where the time derivative is zero) is $u_s = g/\alpha = G$, a good choice is $\hat{u} = u/u_s = u/G = \alpha u/g$, which makes $\hat{u}$ also dimensionless, and the equations becomes

$$\frac{\mathrm{d}\hat{u}}{\mathrm{d}\hat{t}} + \hat{u} \;=\; 1. \tag{2.12}$$

This is the original equation but with time scaled to the exponential decay time and the amplitude scaled to the final, steady-state value.

Changing to units where the independent variable is dimensionless means we do not have to worry about units in our integration of the above equation. Specific advantages are:

[1] There are many notations for scaled variables. Common alternatives include using a prime (e.g. t'), a tilde, (e.g. $\tilde{t}$), using Greek symbols (e.g. τ) and using capitalised symbols (e.g. T).

- Step-size: we typically solve such equations by taking many small time steps Δt. In these natural units, "small" means $\Delta\hat{t} \ll 1$. If we still had dimensions in the problem this would be a lot harder to define.
- Time range: the characteristic timescale in the dimensionless units is $\hat{t} \sim 1$ so we know that solving from $\hat{t} = 0$ to $\hat{t}_f$ (where $\hat{t}_f \sim$ a few) will cover the dynamic range of interest.
- Reuse: the equation can be solved once and simply scaled appropriately for any values of α and g, rather than having to solve explicitly for each set of parameter values.

One thing we *must* remember is to either put the SI units back in when we present our results (e.g. in plots of the integrated solution) or else explicitly state what natural units are (to give meaning to the numbers). For the latter, for the example of Eq. (2.12), in a plot of $\hat{u}(\hat{t})$, we would label the horizontal axis in units of $1/\alpha$ and the vertical axis in units of g/α.

Warning: some problems can have more than one natural unit which would be appropriate for scaling the independent variable(s). This makes it difficult to select how to scale and so some thought is needed in these cases.

Chapter 3

Fourier Transforms

Outline of Section

- Fourier Transforms—recap
- Discrete FTs (DFTs)
- Sampling & Aliasing
- Fast Fourier Transform (FFT) algorithm
- Pseudocode

In Chapter 4 we will look at how to take a set of data points and produce a function that returns a value for any point in the range of these data points, interpolating between them. Here we discuss another way in which such a set of data points can be processed, which is to perform a Fourier Transform on them, to extract the frequency-based properties of the data set.

We focus on how to numerically calculate the Fourier transform of *regular,* discrete, samples of a function. The **Discrete Fourier Transform** (DFT) is used for this, and is the discrete equivalent of the Fourier transform. The sampled function could be the function found by solving an ODE or PDE using finite difference methods. Or it might be data sampled from, e.g., an oscilloscope or a microphone. Considerations and limitations caused by the sampling procedure will be discussed.

The **Fast Fourier Transform** (FFT)—an efficient implementation on the DFT—is used extensively, e.g., for

- Noise suppression—Data analysis
- Image compression—JPEG formats
- Audio and video compression—MPEG formats
- Medical imaging
- Interferometric imaging
- Modelling of optical systems
- Solution of periodic boundary value problems

to name a few examples.

Finally, "Pseudocode" is a concept that is very useful and is widely-used in discussions of computer algorithms. While this has no direct connection with Fourier Transforms, we make the use of the opportunity to describe the Fast Fourier Transform algorithm in this chapter to introduce the concept of writing and using pseudocode.

3.1 Fourier Transforms—Recap

The continuous Fourier Transform (FT), both forward and backward, of a function $f(t)$ are defined as

$$\tilde{f}(\omega) = \int_{-\infty}^{+\infty} e^{i\omega t} f(t)\,\mathrm{d}t = \mathcal{F}(f(t)), \tag{3.1}$$

$$\text{and} \qquad f(t) = \frac{1}{2\pi}\int_{-\infty}^{+\infty} e^{-i\omega t}\tilde{f}(\omega)\,\mathrm{d}\omega = \mathcal{F}^{-1}(\tilde{f}(\omega)), \tag{3.2}$$

for the case of time t and angular frequency $\omega = 2\pi\nu$ (where ν is frequency). Time t and ω are reciprocal 'coordinates' or variables. $\tilde{f}(\omega)$ is the (angular) frequency spectrum of the function $f(t)$. The FT is an expansion on plane waves $\exp(-i\omega t)$ which constitute an orthonormal basis, and $\tilde{f}(\omega)$ can be thought of as the "coefficients" of each component wave of angular frequency ω. (Notation note: do not confuse this with the tilde '$\sim$' notation mentioned elsewhere in the course, such as to denote numerical approximation or for scaled variables!) In general $\tilde{f} \in \mathbb{C}$, even for a real function $f(t)$, with $|\tilde{f}(\omega)|$ being the amplitude and $\arg(\tilde{f}(\omega))$ the phase of the component wave $\exp(-i\omega t)$. For $f(t) \in \mathbb{R}$ the **reality condition** applies in reciprocal space (also referred to as 'Fourier space')

$$\tilde{f}(-\omega) = \tilde{f}^*(\omega), \tag{3.3}$$

so that the negative frequencies are degenerate; there is no extra information in them. However, for an arbitrary $f(t) \in \mathbb{C}$, $\tilde{f}(-\omega)$ is unrelated to $\tilde{f}^*(\omega)$.

The $f(t)$ and $\tilde{f}(\omega)$ are different representations of the same thing in the time and frequency domain, respectively, and the relationship between such FT pairs is often written as

$$f(t) \rightleftharpoons \tilde{f}(\omega). \tag{3.4}$$

$\mathcal{F}^{-1}(\tilde{f}(\omega))$ is often called the inverse Fourier transform. Note that there are different conventions for defining the FT operations, e.g., which operation gets the $1/2\pi$ factor or whether both share it (i.e. $1/\sqrt{2\pi}$ in front of each), and the sign in the transform kernel (i.e. $\exp(-i\omega t)$ or $\exp(i\omega t)$) , etc.

For spatial FTs in one dimension (e.g. x) the reciprocal variables become $t \to x$ and $\omega \to k$ where $k = 2\pi/\lambda$ is the wavenumber and λ is wavelength. In multiple spatial dimensions the wavenumber becomes a wave vector

$$\vec{x} \equiv (x, y, z) \qquad \leftrightarrow \qquad \vec{k} \equiv (k_x, k_y, k_z)\,, \tag{3.5}$$

and the FTs are modified accordingly:

$$\tilde{f}(\vec{k}) = \int_{-\infty}^{+\infty} e^{i\vec{k}\cdot\vec{x}} f(\vec{x}) d\vec{x} = \mathcal{F}(f(\vec{x})), \tag{3.6}$$

$$f(\vec{x}) = \frac{1}{(2\pi)^3} \int_{-\infty}^{+\infty} e^{-i\vec{k}\cdot\vec{x}} \tilde{f}(\vec{k}) d\vec{k} = \mathcal{F}^{-1}(\tilde{f}(\vec{k})). \tag{3.7}$$

Derivatives and FTs

FTs can be very useful in manipulating differential equations. This is because spatial or time derivatives become algebraic operations in Fourier space. As an example consider the following equation in real space

$$\frac{\mathrm{d}}{\mathrm{d}x} u(x) = v(x). \tag{3.8}$$

Taking the FT of this equation yields

$$-ik\tilde{u}(k) = \tilde{v}(k). \tag{3.9}$$

This can be shown as follows: replace $u(x)$ and $v(x)$ by their inverse FTs

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{1}{2\pi}\int_{-\infty}^{+\infty} e^{-ikx}\tilde{u}(k)\,\mathrm{d}k\right) = \frac{1}{2\pi}\int_{-\infty}^{+\infty} e^{-ikx}\tilde{v}(k)\,\mathrm{d}k\,. \tag{3.10}$$

The only bits containing x are the transform kernel (i.e. the plane wave part $\exp(-ikx)$). Therefore using Leibnitz's integral rule yields

$$\frac{1}{2\pi}\int_{-\infty}^{+\infty} -ike^{-ikx}\tilde{u}(k)\,\mathrm{d}k = \frac{1}{2\pi}\int_{-\infty}^{+\infty} e^{-ikx}\tilde{v}(k)\,\mathrm{d}k\,. \tag{3.11}$$

Equating the integrands on both sides we have (3.9).

This can be generalised to higher derivatives:

$$\frac{\mathrm{d}^n}{\mathrm{d}x^n} f(x) \rightleftharpoons (-ik)^n \tilde{f}(k). \tag{3.12}$$

It can be generalised to 3D too:

$$\begin{aligned} \nabla f(\vec{x}) &\rightleftharpoons -i\vec{k}\tilde{f}(\vec{k}) \quad , & \nabla^2 f(\vec{x}) &\rightleftharpoons -|\vec{k}|^2 \tilde{f}(\vec{k}) \quad , \\ \nabla \cdot \vec{E}(\vec{x}) &\rightleftharpoons -i\vec{k} \cdot \tilde{\vec{E}}(\vec{k}) \quad , & \nabla \times \vec{E}(\vec{x}) &\rightleftharpoons -i\vec{k} \times \tilde{\vec{E}}(\vec{k}), \end{aligned} \tag{3.13}$$

where $\nabla^2 f = \nabla \cdot (\nabla f)$ has been used.

3.2 Discrete FTs

DFTs are the equivalent of complex Fourier series, which are defined on a finite length domain, but for a **discretely sampled function** rather than a continuous function. We illustrate here with time and angular frequency.

Time domain—We assume the function is sampled by N equally spaced samples on a time domain of length $T = N\Delta t$ as illustrated in figure 3.1 ;

$$f_n \equiv f(t_n) \qquad \text{with} \qquad n = 0, 1, 2, ..., N-1 \qquad \text{and} \qquad t_n = n\Delta t. \tag{3.14}$$

The function $f(t)$ is ***assumed to be periodically extended*** beyond the domain $0 \leq t \leq T$ so that $f(t + mT) = f(t)$ for integer m. For the sampled function, periodicity means $f_N = f_0$. This does not mean that the value of the function at $t \to T$ has to be the same as that at $t = 0$; but it needs to be single-valued, and for discretising it one would choose the value for $t = 0$.

If one is only focused on the time domain $0 \leq t < T$, the periodicity of the function does not affect things directly.

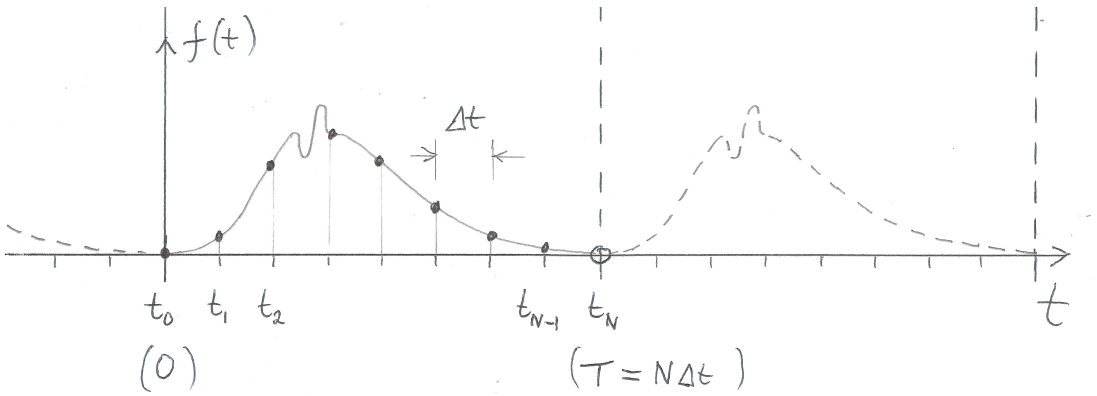

Figure 3.1: Illustration of a signal in the time domain. $N = 8$ here. The signal/function is assumed to be periodically extended beyond $0 \leq t \leq T$.

Frequency domain— Figure 3.2 illustrates the frequency domain and discrete spectrum of the signal sampled in figure 3.1. The longest wave that can fit exactly into the time domain has an angular frequency $\omega_{min} = 2\pi/T \equiv \Delta\omega$. The shortest wave that can be *recognised* by the grid and fits periodically into the time domain has duration $2\Delta t$ (i.e. one peak and one trough in just two time samples) so that $\omega_{max} = 2\pi/(2\Delta t) = \pi/\Delta t$. This maximum frequency is known as the **Nyquist frequency**

$$\boxed{\omega_{max} = \frac{\pi}{\Delta t} = \Delta\omega \frac{N}{2}.} \tag{3.15}$$

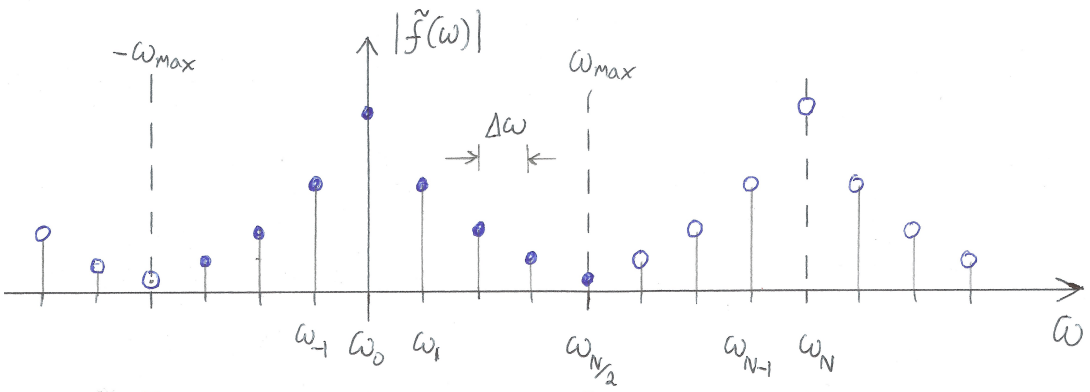

Figure 3.2: Frequency domain and discrete spectrum corresponding to figure 3.1. The discrete spectrum is also periodically extended (beyond $|\omega_{max}|$ in this case). The modulus of $\tilde{f}(\omega)$ is depicted.

Allowing for negative angular frequencies too, these frequencies define the discrete, finite, angular frequency grid on which $\tilde{f}$ is sampled;

$$\boxed{\tilde{f}_p \approx \frac{1}{\Delta t}\tilde{f}(\omega_p) \quad \text{with} \quad p = -\frac{N}{2}, \ldots, 0, \ldots, \frac{N}{2} \quad \text{and} \quad \omega_p = p\Delta\omega = p\frac{2\pi}{N\Delta t}.} \tag{3.16}$$

Each discrete frequency corresponds to a wave that fits exactly an integer number of times 'p' into the finite domain. Note that the integer index p seems to run over $N+1$ values but in fact the $-N/2$ and $N/2$ samples are degenerate, i.e.,

$$\tilde{f}_{-N/2} \equiv \tilde{f}_{N/2}.$$

so there are only N degrees of freedom. (See section 3.3.)

Why does $\tilde{f}_p \approx \tilde{f}(\omega_p)/\Delta t$ rather than $\tilde{f}_p = \tilde{f}(\omega_p)/\Delta t$ in equation (3.16) above? This is because $\tilde{f}_p$ represents the DFT which is not always exactly $\tilde{f}(\omega)$, the true spectrum of $f(t)$, simply picked out at discrete frequencies ω_p. We assume that $f(t)$ *is* exactly, discretely sampled. If there is no aliasing (see 3.3) then $\tilde{f}_p = \tilde{f}(\omega_p)/\Delta t$. If there is aliasing, then the DFT is a distorted, discrete, version of the true spectrum. (The $1/\Delta t$ factor is just the convention used in the definition of the DFT, see below.)

The DFT is the analogue of a continuous FT, but with the continuous integral $\int \ldots \mathrm{d}t$ replaced by a finite sum $\sum \ldots \Delta t$:

$$\boxed{\tilde{f}_p = \sum_{n=0}^{N-1} f_n e^{i\omega_p t_n} = \sum_{n=0}^{N-1} f_n e^{i2\pi pn/N},} \tag{3.17}$$

where $\omega_p t_n = \left(\frac{2\pi p}{N\Delta t}\right)(n\Delta t)$ has been used to obtain the second form. The backwards transform is similarly defined as

$$f_n = \frac{1}{N}\sum_{p=-N/2+1}^{N/2} \tilde{f}_p e^{-i2\pi pn/N}. \tag{3.18}$$

The different normalisation factor of $1/N$ accounts for the discrete sampling. This comes from $\mathrm{d}t \to \Delta t$ and $d\omega \to \Delta\omega = 2\pi/(N\Delta t)$. (Note that in going from the FT (3.1) to the DFT (3.17) a factor of Δt has been absorbed into $\tilde{f}_p$, i.e., $\tilde{f}_p \approx \tilde{f}(\omega_p)/\Delta t$.) The backward DFT (3.18) is usually defined as

$$\boxed{f_n = \frac{1}{N}\sum_{p=0}^{N-1} \tilde{f}_p e^{-i2\pi pn/N},} \tag{3.19}$$

which is more symmetrical with the forward DFT. Why this is possible will be seen in section 3.3.

Relation to complex Fourier series – Discrete FTs (DFTs) are intimately related to complex Fourier series (CFS).

$$c_k = \frac{1}{T}\int_0^T f(t)e^{i\omega_k t}\,\mathrm{d}t\,, \tag{3.20}$$

$$f(t) = \sum_{k=-\infty}^{+\infty} c_k e^{-i\omega_k t}. \tag{3.21}$$

Complex Fourier series represent the discrete spectrum of a ***continuous function*** $f(t)$, also on a domain of finite length. The allowed angular frequencies have the same spacing $\Delta\omega$, but are now infinitely many.

FFTs—Fast Fourier transforms

The DFTs above, equations (3.17) and (3.18), are easily implemented on a computer, however the naive method of implementing it requires $\mathcal{O}(N^2)$ operations. With typical samples volumes of $N \sim 10^6$ in modern applications this becomes prohibitive even on the fastest computers. The DFT can actually be done in $\boldsymbol{\mathcal{O}(N \log_2 N)}$ **operations**; the algorithm for this is known as the **fast Fourier transform** or just **FFT**. For $N = 10^6$, the speed-up factor $N/log_2 N$ is approximately 50,000. FFTs work best when $N = 2^m$ with integer m, and works by recursively splitting the DFT into separate sums over only the even or odd indices. ***The FFT implementation of the DFT is what is used in practice.*** If $N \neq 2^m$, then it is usual to ***pad out*** the samples with zeros until the size is $N = 2^m$.

DFT in 2D

For a function $f_{n,m} = f(x_n, y_m)$ defined on a 2D grid $x_n = m\Delta x$ for $0 \leq n \leq N-1$ and $y_m = m\Delta y$ for $0 \leq n \leq M-1$ its DFT is given by

$$\tilde{f}_{p,q} = \sum_{n=0}^{N-1}\sum_{m=0}^{M-1} f_{n,m} e^{i2\pi pn/N} e^{i2\pi qm/M}. \tag{3.22}$$

The backward DFT is

$$f_{n,m} = \frac{1}{NM}\sum_{p=0}^{N-1}\sum_{q=0}^{M-1} \tilde{f}_{p,q} e^{-i2\pi pn/N} e^{-i2\pi qm/M}. \tag{3.23}$$

Going to 3-dimensions just adds another nested sum (with a new, independent index) and another exponential wave factor (incorporating the new dimension) into the transform kernel.

3.3 Sampling & Aliasing

The discrete sampling of the function to be FT'd, given by equation (3.14), introduces some sampling effects and care has to be taken in allowing for them. There is a minimum sampling rate that needs to be used so that we do not lose information. If the function to be sampled fits into the finite length time (or spatial) domain of length T, and is **bandwidth-limited** to below the Nyquist frequency then all frequency content of the function is exactly captured by the discrete sampling process. If the function has $\tilde{f}(\omega) \neq 0$ for $|\omega| > \omega_{max}$ then the frequency content for $|\omega| > \omega_{max}$ will be **aliased** down into the range $|\omega| < \omega_{max}$ and distort the DFT compared to the exact FT of $f(t)$. If the original function is bandwidth-limited but is longer than T, then it will be ***clipped*** which will introduce a sharp cutoff at $t = 0$ and/or T. This will effectively increase the bandwidth of the clipped function and aliasing will occur again during sampling of it.

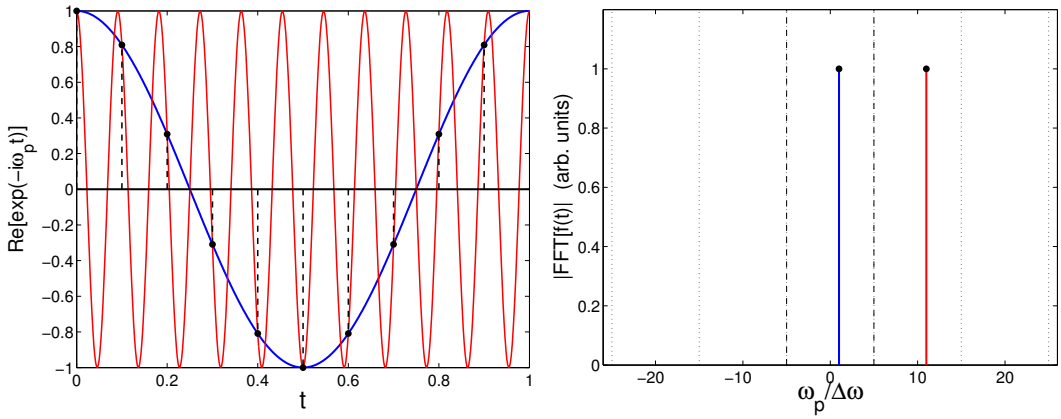

Figure 3.3: (Left) Indistinguishable harmonic waves below and above the Nyquist frequency, for $N = 10$. Blue line: $\omega = \omega_1 = \Delta\omega$. Red line: $\omega = \omega_{N+1} = \Delta\omega + 2\omega_{max}$. (Waves are periodically extended beyond range shown.) (Right) Corresponding (angular) frequency spectrum. The vertical dashed lines lie at $\pm$ the Nyquist frequency.

Aliasing & indistinguishable frequencies – For any wave with a frequency ω_p lying in the frequency domain captured by the DFT, there are higher frequency waves with $\omega_{p'} = \omega_p + m\Omega$ (where $m \in \mathbb{Z}$ and $\Omega = 2\omega_{max}$ is the width of the frequency domain) that look exactly the same to the discrete time-sampling grid. This is illustrated in figure 3.3. This can be understood by considering the kernel of the DFT, i.e., $\exp\left(i\omega_{p'}t_n\right)$.

$$\begin{aligned}\exp\left[i(\omega_p + m\Omega)t_n\right] &= \exp\left[i\left(\frac{2\pi pn}{N} + m\frac{2\pi}{\Delta t}n\Delta t\right)\right] = \exp\left(i\frac{2\pi pn}{N} + i2\pi mn\right) \\ &= \exp\left(i\frac{2\pi pn}{N}\right) = \exp\left(i\omega_p t_n\right).\end{aligned}$$

This has the following implications;

(a) It explains why $\tilde{f}_{-N/2} \equiv \tilde{f}_{N/2}$.

(b) It also explains why equation (3.19) for the backwards DFT 'works'. The "negative" frequency part of the spectrum $p = \left\{-\frac{N}{2}+1, \ldots, -1\right\}$ maps to the positive spectrum at $p' = p + N = \left\{\frac{N}{2}+1, \frac{N}{2}+2, \ldots, N-1\right\}$ lying beyond the Nyquist frequency. In other words, although the $\sum_{p=N/2+1}^{N-1}$ parts of (3.19) are above the Nyquist frequency, they happen to capture the desired negative frequencies within the Nyquist range.

(c) Aliasing:

$$\tilde{f}_p = \sum_m \mathcal{F}(f)|_{\omega_p + m\Omega}, \tag{3.24}$$

i.e., the DFT (the LHS) folds in the Fourier components from the *exact FT of the continuous function* from the indistinguishable set of waves $\omega'_p = p + m\Omega$.

Figure 3.4 shows the phenomenon of aliasing for a top-hat function

$$f(t) = \begin{cases} 1 & |t| < a/2 \\ 0 & |t| > a/2 \end{cases}. \tag{3.25}$$

3.4 Fast Fourier Transforms—FFTs

The FFT algorithm for doing a discrete Fourier transform in $\mathcal{O}(N \log N)$ operation was described by Daniel & Lanczos in 1942, and earlier versions existed in the 1800s (indeed, the first example of

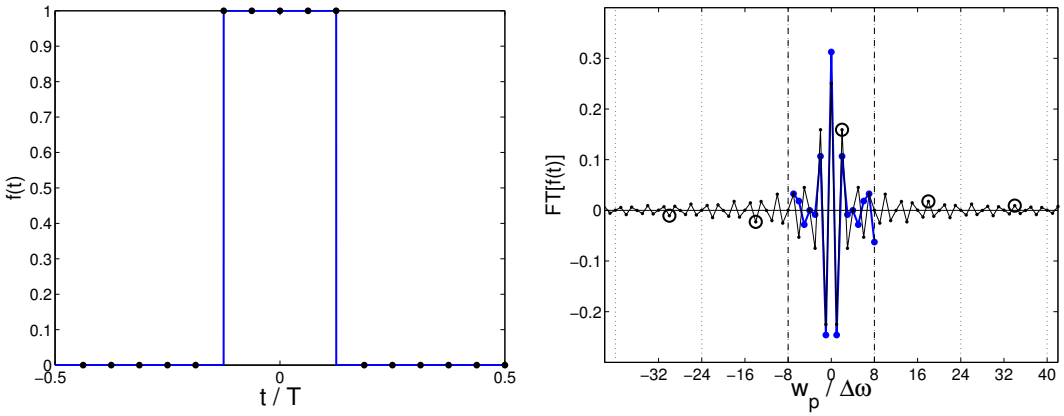

Figure 3.4: Aliasing: (Left) $f(t)$ in the time domain; centred "rect" function of width $T/4$. Sampled with $N = 16$. (Right) The DFT (blue, solid line + markers), and exact FT (black, thin solid line) in the frequency domain. The open circles denote Fourier components outside the range $-\omega_{max} \leq \omega \leq \omega_{max}$ that are aliased into that range during the DFT. The real part of the DFT & FT are shown. The DFT been divided by Δt.

the algorithm was given by Gauss in 1805), but was rediscovered and popularised in the computer age by Cooley & Tukey in 1965.

Directly coding the DFT (as we did last time) results in an $\mathcal{O}(N^2)$ algorithm. But this can be reduced to $\mathcal{O}(N \log_2 N)$ with the Fast Fourier Transform, for $N = 2^m$. The central idea is that an N-sample DFT can be split into two $N/2$-sample ones:

$$\begin{aligned}
\tilde{f}_p &= \sum_{n=0}^{N-1} f_n e^{i2\pi pn/N} \\
&= \sum_{n=0,2,\ldots}^{N-2} f_n e^{i2\pi pn/N} + \sum_{n=1,3,\ldots}^{N-1} f_n e^{i2\pi pn/N} \\
&= \tilde{f}_p^{\text{even}} + e^{i2\pi p/N} \times \tilde{f}_p^{\text{odd}}
\end{aligned}$$

Each half of these is an $\mathcal{O}(\frac{N}{2} \log \frac{N}{2})$ problem, and this can be done recursively.

Here, $\tilde{f}_p^{\text{even, odd}}$ are $N/2$-sample DFTs of the even and odd samples of the original f_n. These are each periodic such that $\boldsymbol{\tilde{f}_{p+N/2}^{\text{even, odd}} = \tilde{f}_p^{\text{even, odd}}}$ This results in, for $0 \leq p < N/2$:

$$\begin{aligned}
\tilde{f}_p &= \tilde{f}_p^{\text{even}} + e^{i2\pi p/N} \times \tilde{f}_p^{\text{odd}} \\
\tilde{f}_{p+N/2} &= \tilde{f}_p^{\text{even}} - e^{i2\pi p/N} \times \tilde{f}_p^{\text{odd}}.
\end{aligned} \tag{3.26}$$

This leads to a simple algorithm for implementation, which we can discuss using pseudocode.

3.4.1 Pseudocode

We often need to explain an algorithm to each other, in a way that clearly shows how you would code it up (in any language). In this case, it helps not to be burdened by the (language-specific) overhead that real code brings with it (as well as a need to be syntactically perfect). This to say that it helps to omit constructs such as:

"`import numpy as np`"
"`tarray = np.zeros(N)`"
"`#include <stdio.h>`"
"`COMMON/CRZT/IXSTOR,IXDIV,IFENCE(2),LEV,LEVIN,BLVECT(LURCOR)`"

and the positioning of colons and semicolons etc., and allow ourselves to focus on the logic of the algorithm that is being presented. We call this "Pseudocode"; it is intended to be for humans, but has a code-like structure because of the logic of algorithms. While attempts have been made in the past to standardise pseudocode, these have failed because this runs counter to the benefits of using pseudocode; therefore there is no formal syntax; it is up to the author to make things clear for their purpose and their audience. One is allowed to use language-specific elements if that is not confusing (e.g., logic statements from Python or C etc.).

Examples of pseudocode of all sorts of styles are readily available online.

3.4.2 Fast Fourier Transforms in Pseudocode

The following is an example of the FFT algorithm, in pseudocode. The recursive nature of the algorithm is made clear through "calls" to the FFT() function itself:

```
def FFT(f): # f is an array
    N = size of array f
    if N == 1:
        # for a sample size of 1, the FT is the value itself
        return f[0]
    if N is not a power of 2, exit

    # recursive calls
    farray_even = FFT(even entries of f) # size N/2
    farray_odd  = FFT(odd  entries of f) # size N/2

    # return an N-element array made from
    for p = 0..N/2-1
        farray[p] = farray_even[p]
                        + exp(i2pi p/N) * farray_odd[p]
        farray[p+N/2] = farray_even[p]
                        - exp(i2pi p/N) * farray_odd[p]

    return farray
```

Note the use of expressions such as a) `N = size of array f`, b) `if N is not a power of 2, exit` and c) `odd entries of f`. These are of course not syntactically correct in any programming language, but are easy to parse for a human being, without being confusing for people who do not speak a particular language. These examples could be written as a) `N = $#f + 1;`, b) `if(x&(x-1)) exit;` and c) `f[1::2]`, none of which can necessarily be said to be easy to understand if one is not aware of the programming language that is in use.

The conversion of the above pseudocode into real code is left for the problem sheets.

Recursive function calls, whilst being logically clear, can require substantial overheads when the code is executed. In the past, when coding in BASIC and FORTRAN recursive function calls were not even part of the standards for the languages.

In addition to the above, further shortcuts exist which can speed up the code by sorting the elements of the calculations in a clever way.

Contracting the "even" and "odd" superscripts in Equation 3.26 to "e" and "o" yields:

$$\begin{aligned} \tilde{f}_p &= \tilde{f}^{\rm e}_p + e^{i2\pi p/N} \times \tilde{f}^{\rm o}_p \\ \tilde{f}_{p+N/2} &= \tilde{f}^{\rm e}_p - e^{i2\pi p/N} \times \tilde{f}^{\rm o}_p. \end{aligned}$$

Iterating once again, the quantity $\tilde{f}^{\rm e}$ can be written, for $p = 0, \ldots, N/4 - 1$:

$$\begin{aligned} \tilde{f}^{\rm e}_p &= \tilde{f}^{\rm ee}_p + e^{i2\pi p/(N/2)} \times \tilde{f}^{\rm eo}_p \\ \tilde{f}^{\rm e}_{p+N/4} &= \tilde{f}^{\rm ee}_p - e^{i2\pi p/(N/2)} \times \tilde{f}^{\rm eo}_p. \end{aligned}$$

Assuming $N = 2^m$ (i.e. even), after m even/odd splitting we are left with N functions of period 1 which are each trivially Fourier Transformed; the transform of each is the same as itself

$$\tilde{f}_p^{eooeeoeoeo...e} = f_n, \tag{3.27}$$

for some n.

All we have left to do is identify which combination of $eoeeo...e$ corresponds to each n—this can be done by assigning the binary value $e \equiv 0$ and $o \equiv 1$ to each element in the sequence and then **reversing** the sequence. The series of 1s and 0s obtained is a binary representation of n.

This can be seen empirically in the following, where an input function represented by a series of 2^3 samples is broken up in to two even and odd series, continuing recursively into single-sample elements:

$$\begin{array}{cccccccc}
||f_0, & f_1, & f_2, & f_3, & f_4, & f_5, & f_6, & f_7|| \\
||f_0, & f_2, & f_4, & f_6|| & ||f_1, & f_3, & f_5, & f_7|| \\
||f_0 & f_4|| & ||f_2 & f_6|| & ||f_1 & f_5|| & ||f_3 & f_7|| \\
||f_0|| & ||f_4|| & ||f_2|| & ||f_6|| & ||f_1|| & ||f_5|| & ||f_3|| & ||f_7||
\end{array}$$

This results in the sequence of indices: 0, 4, 2, 6, 1, 5, 3, 7. This sequence can be compared with the original sequence, in binary form:

Original		Transformed	
Decimal	Binary	Decimal	Binary
0	000	0	000
1	001	4	100
2	010	2	010
3	011	6	110
4	100	1	001
5	101	5	101
6	110	3	011
7	111	7	111

Here, the binary representation is made of of bits that are reversed between the original and transformed order of indices. The "bit-reversing" procedure may not be particularly fast in software, but it is easy to implement directly in signal-processing chips etc.

The complete method requires

$$N \times p = \frac{N \log N}{\log 2} \sim \mathcal{O}(N \log N), \tag{3.28}$$

operations. As an example consider a problem with $N \sim 10^4$ samples, in this case the FFT is faster than the naive DFT by a factor of about 750.

The FFT algorithm leads naturally to the fast convolution of functions, filtering, and finding the correlation between functions etc.

Chapter 4

Interpolation

4.1 Introduction

Finding the value of a function at an arbitrary point in a range, given a set of known points that represent it, is another essential piece of a computational physicist's toolkit. This is not a fully-determined problem—the information is not there to completely specify the values between the data points—and therefore additional assumptions are needed. It is up to the physicist to decide what is adequate for the purpose.

One approach is to fit a function to the data points, as is often done with experimental data. This approximate fit will probably not go right through any of the data points, which is often what is desired, particularly if there are uncertainties associated to the data points themselves. This will be discussed in later chapters.

Another approach, which is considered here, is to make a function that connects the known points. This is interpolation. When might interpolation be needed? One example is in solving ODEs numerically, where time is typically discretised (t_n). The value of the solution at a time between these t_n might be required. As we will see later, when solving PDEs, space is discretised into a grid and the numerical method provides the (approximate) solution at these points. Again, the solution at other points in space is often needed, and can only be obtained by interpolation.

Interpolation is an extensive subject. Here we briefly look at a few of the simplest methods. These simple methods will get you surprisingly far, and can still be found in most numerical software suites today. More advanced and powerful methods certainly exist too. One of our favourites is splines under tension,[1] where one uses an additional parameter to interpolate the interpolation scheme between linear and cubic splines. Neural networks and other machine learning algorithms are, at the end of the day, also just fancy multi-dimensional interpolators. A machine-learning algorithm uses functions trained on some known data points to produce a good guess at a quantity somewhere between the data points on which it has been trained.

As with any other numerical method, it is up to the physicist to show that the chosen algorithm is appropriate for their purposes, as we shall see.

4.2 Linear interpolation

Linear interpolation is simply to connect adjacent points with a straight line, connect-the-dots style. If two, adjacent, known points are (x_i, f_i) and (x_{i+1}, f_{i+1}), to find f at a point x in between we simply move along the straight line between the two points:

$$\boxed{f(x) = \frac{(x_{i+1} - x)f_i + (x - x_i)f_{i+1}}{x_{i+1} - x_i}.} \tag{4.1}$$

This is the simplest form of interpolation, and is useful if the density of the provided data points is high, or if you somehow know that the function is linear in the regions between the points. Its

[1] See e.g. TSPACK, http://dl.acm.org/citation.cfm?id=151277

behaviour is certainly more predictable than other, more involved forms of interpolation. However, in the more general case, it is unlikely that a true function looks like anything that is made up of only a succession of straight segments, and the limitations of linear interpolation will be quite clear.

4.3 Bi-linear interpolation

Bi-linear interpolation works for a function of two variables $f(x, y)$. Actually it is just linear interpolation in two dimensions. You can probably work it out for yourself already. Imagine f is known on four points which are the corners of a rectangle in the x–y coordinate system; (x_i, y_k), (x_{i+1}, y_k), (x_i, y_{k+1}) and (x_{i+1}, y_{k+1}). We want f at an arbitrary point (x, y) within this square. Linear interpolation is first applied to f in the x direction, along both the bottom ($y = y_k$) and top ($y = y_{k+1}$) of the square. For instance, along the bottom, equation (4.1) is used with $(x_i, f(x_i, y_k))$ and $(x_{i+1}, f(x_{i+1}, y_k))$ to obtain the intermediate value $f(x, y_k)$. Similarly at the top linear interpolation yields $f(x, y_{k+1})$. Now these two intermediate values are linearly interpolated in the y-direction, using an equation analogous to (4.1). Doing interpolation in y to get the intermediate values and then interpolation in x yields exactly the same value.

Of course you can do this in as many dimensions as you like; multivariate interpolation is the generalisation to functions with more than one variable; $f(x, y, z, \ldots)$. In this case, you need to use a (hyper-)box around your desired position, with 2^D vertices, where D is the dimension of your problem.

4.4 Lagrange polynomials

Starting with $n + 1$ distinct points (x_i, f_i) where $0 \leq i \leq n$ and $x_i < x_j$, it is clear that there is a unique n^{th} degree polynomial (i.e. with $n + 1$ degrees of freedom) that goes exactly through these points. This is called the Lagrange polynomial for these points, and can be written:

$$\boxed{P_n(x) = \sum_{i=0}^{n} \left(\prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \right) f_i,} \tag{4.2}$$

where in the product, j run over integers from 0 to n but misses out the value i. Eq. 4.1 is the case of $n = 1$. For the $n = 2$ case

$$P_2(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f_0 + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f_1 + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f_2. \tag{4.3}$$

Note that the x_i do not need to be equally spaced.

Such a polynomial does meet the requirements of being a smooth, well-behaved function that passes through all the data points, but unless it is known that the data follow a polynomial form of the correct degree (which is unlikely), the Lagrange polynomial is likely to "contort" itself to make it go through the data points, resulting in unnaturally wavy behaviour, making it a poor interpolant.

For specific cases, when the data points and the physics problem at hand suit it, the Lagrange polynomial can be extremely useful.

4.5 Cubic splines

Linear interpolation is nice, because it is easy and fast, and does not display the odd behaviour that the Lagrange polynomial can if you are not careful. But it is not ideal—the resulting approximation to the underlying function is clearly not a very good representation of reality in most cases, as it has zero second derivative and a discontinuous first derivative at every data point. No self-respecting function has discontinuous derivatives, especially not one that purports to represent a physical

quantity. The data points given to us are usually not accompanied by the statement "these points are also where the first derivatives can be discontinuous".

What can be done about this? We can introduce non-zero higher derivatives to our interpolating function, and force them to be continuous across the data-points, but to do that we need to use polynomials of higher order than one. The lowest-order option is to make all our interpolating functions quadratics. This lets us have continuous first derivatives, and non-zero second derivatives—but the second derivatives will still be discontinuous across the data points. The lowest order polynomial interpolant that allows for continuous first and second derivatives is cubic.

As a general term, a function that matches polynomials, in piecewise fashion along the data points, is called a "spline".

However, once we start introducing additional terms in the interpolating functions, we will have more free parameters to constrain for each piece of the approximating function, so we are going to have to start using more data points at a time to fit those parameters. We therefore need to start making the interpolant somewhat non-local, such that it is not just fit to the two data points either side of it, but more points, further away in each direction. The more continuous derivatives you want, the higher-order your interpolant needs to be, and the more data points you must use to constrain it.

For the cubic spline, we need to use four points at a time; one at each end of the interval being interpolated, and one more each on either side of the interval. We can start by taking the expression for the linear interpolant:

$$f(x) = A(x)f_i + B(x)f_{i+1}, \tag{4.4}$$

with

$$A(x) \equiv \frac{x_{i+1} - x}{x_{i+1} - x_i}, \tag{4.5}$$

$$B(x) \equiv 1 - A(x) = \frac{x - x_i}{x_{i+1} - x_i} \tag{4.6}$$

and supplementing it with some additional corrections, proportional to the second derivatives at each of the adjacent points:

$$f(x) = A(x)f_i + B(x)f_{i+1} + C(x)f_i'' + D(x)f_{i+1}'', \tag{4.7}$$

with

$$C(x) \equiv \tfrac{1}{6}(A(x)^3 - A(x))(x_{i+1} - x_i)^2 \tag{4.8}$$

$$D(x) \equiv \tfrac{1}{6}(B(x)^3 - B(x))(x_{i+1} - x_i)^2 \tag{4.9}$$

At this point, Eq. 4.7 seems pretty useless—how do we know the values of the second derivative of the function at the data points? We only have data on the value of the function itself, not its derivatives. The key here is to solve for these second derivatives at the points $\{x_0, x_2, x_3, \ldots, x_n\}$.

But how? First we need to impose continuity of the *first* derivative across each data point. Using

$$\frac{\mathrm{d}A}{\mathrm{d}x} = -\frac{\mathrm{d}B}{\mathrm{d}x} = -\frac{1}{x_{i+1} - x_i}, \tag{4.10}$$

$$\frac{\mathrm{d}C}{\mathrm{d}x} = \frac{1 - 3A^2}{6}(x_{i+1} - x_i) \tag{4.11}$$

$$\frac{\mathrm{d}D}{\mathrm{d}x} = \frac{3B^2 - 1}{6}(x_{i+1} - x_i) \tag{4.12}$$

we get

$$\frac{\mathrm{d}f}{\mathrm{d}x} = \frac{f_{i+1} - f_i}{x_{i+1} - x_i} - \frac{3A^2 - 1}{6}(x_{i+1} - x_i)f_i'' + \frac{3B^2 - 1}{6}(x_{i+1} - x_i)f_{i+1}''. \tag{4.13}$$

Imposing the continuity of $\frac{\mathrm{d}f}{\mathrm{d}x}$ at x_i (i.e. where intervals $[x_{i-1}, x_i]$ and $[x_i, x_{i+1}]$ meet)

$$\left.\frac{\mathrm{d}f}{\mathrm{d}x}\right|_{x\to x_i^-} = \left.\frac{\mathrm{d}f}{\mathrm{d}x}\right|_{x\to x_i^+} \tag{4.14}$$

then gives the fundamental expression

$$\frac{x_i - x_{i-1}}{6} f''_{i-1} + \frac{x_{i+1} - x_{i-1}}{3} f''_i + \frac{x_{i+1} - x_i}{6} f''_{i+1} = \frac{f_{i+1} - f_i}{x_{i+1} - x_i} - \frac{f_i - f_{i-1}}{x_i - x_{i-1}}. \tag{4.15}$$

This equation is valid for $i = 1\,..\,n{-}1$. This means that it constitutes a system of $n-1$ equations in $n+1$ unknowns ($f''_{0..n}$). We know from linear algebra that this means it has a two-dimensional solution space, implying that we need two more constraints to solve the system. Those constraints come from the chosen **boundary conditions**, which are up to the user of the algorithm to choose as they please. These may take the form of a specified value of the first or second derivative at each of the two end-points of the region being interpolated. The choice $f''_0 = f''_n = 0$ has a special name: the 'natural' spline. This is because if there is a thin massless and frictionless beam that is fixed such that it passes through each data point, it will take on the form that is given by the natural spline. One can also choose to use the two degrees of freedom to fix the first derivatives at the end points to pre-determined values.

The simultaneous equations for $f''_{0..n}$ that are given by this can be set up as a matrix equation, and the matrix be inverted to solve for all the second derivative values. Once you have these, you can use Eq. 4.7 to evaluate your cubic spline at any and as many values of x as you want, at your leisure—without ever having to re-evaluate the second derivatives.

With any interpolation scheme, it is important that validity of the scheme for your set of data is checked, on a case-by-case basis. With that caveat, the cubic spline is a excellent algorithm for general use, with continuity and smoothness of the interpolant being guaranteed with a small number of free parameters, which is often what one needs in a physics context.

Extensions of the cubic spline into multiple dimensions, as well as more sophisticated algorithms do exist, and are implemented in many external library packages. However, from a physicist's point of view, the decision whether or not to interpolate, or to rather fit a function to the data, is often the most important—and we will look at function fitting in a later lecture.

Chapter 5

Numerical Calculus

Outline of Section

- Numerical Differentiation
- Numerical Integration
- The Trapezoidal Rule
- Simpson's Rule
- Romberg integration
- Improper Integrals

5.1 Numerical Differentiation

We are used to defining a derivative, e.g., $\frac{\mathrm{d}y}{\mathrm{d}x}$ or $y'(x)$, as

$$\frac{\mathrm{d}y(x)}{\mathrm{d}x} \equiv \lim_{h\to 0} \frac{y(x+h)-y(x)}{h} \equiv \lim_{\Delta x\to 0} \frac{\Delta y}{\Delta x}\,. \tag{5.1}$$

We approach this limit on the computer by defining a **forward difference scheme (FDS)**

$$\boxed{\tilde{y}'_f(x) \equiv \frac{y(x+h)-y(x)}{h}\,.} \tag{5.2}$$

The FDS is an example of a **finite difference approximation**.

Notation: in the this part of the course a tilde ($\sim$) over a quantity signifies that it is an approximated version, which is subject to truncation error.

By considering the Taylor expansion of $y(x+h)$ around the point x we can understand how the error in the above scheme is first order;

$$y(x+h) = y(x) + y'(x)\,h + \frac{y''(x)}{2}\,h^2 + \frac{y'''(x)}{3!}\,h^3 + ... \tag{5.3}$$

We can define the truncation error as

$$\epsilon = \frac{y''(\xi)}{2}\,h^2, \tag{5.4}$$

where ξ is an unknown value which lies in the interval $x < \xi < x+h$. The correct choice of ξ results in the following exact relation

$$y(x+h) = y(x) + y'(x)\,h + \frac{y''(\xi)}{2}\,h^2\,, \tag{5.5}$$

Thus we can write the first derivative as

$$y'(x) = \frac{y(x+h)-y(x)}{h} - \frac{y''(\xi)}{2}\,h = \tilde{y}'_f(x) - \mathcal{O}(h)\,. \tag{5.6}$$

So the simple forward difference scheme has error $\mathcal{O}(h)$. Notice that even if we reduce this truncation error by making h really small, in practice, the accuracy of the derivative will have a limit imposed by the round-off error of the computer.

We can also use a **backward difference scheme (BDS)**

$$\boxed{\tilde{y}'_b(x) \equiv \frac{y(x) - y(x-h)}{h}\,,} \tag{5.7}$$

but as you can easily show (expand $y(x-h)$ around x) this will have a similar error to the forward difference estimate. However since they are estimates of the same quantity there must be a way to combine the two to get a more accurate estimate. This is achieved by the **central difference scheme (CDS)** which is the average of the two estimates and is 2nd order accurate,

$$\boxed{\tilde{y}'_c(x) \equiv \frac{\tilde{y}'_f(x) + \tilde{y}'_b(x)}{2} = \frac{y(x+h) - y(x-h)}{2h}\,.} \tag{5.8}$$

Consider the Taylor series (to 3rd order) for $y(x+h)$ and $y(x-h)$ giving

$$y(x+h) = y(x) + y'(x)\,h + \frac{1}{2}y''(x)\,h^2 + \frac{1}{3!}y'''(\xi)\,h^3 \tag{5.9}$$

and

$$y(x-h) = y(x) - y'(x)\,h + \frac{1}{2}y''(x)\,h^2 - \frac{1}{3!}y'''(\zeta)\,h^3. \tag{5.10}$$

Crucially the y'' terms cancel when subtracting these

$$y(x+h) - y(x-h) = 2\,y'(x)\,h + \mathcal{O}(h^3), \tag{5.11}$$

which gives

$$y'(x) = \frac{y(x+h) - y(x-h)}{2h} - \mathcal{O}(h^2) \equiv \tilde{y}'_c(x) - \mathcal{O}(h^2), \tag{5.12}$$

so the central difference scheme gives the derivative up to an $\mathcal{O}(h^2)$ error. Remember that h is a small step i.e. in suitable units (see later) it will satisfy the condition $h \ll 1$ so the error *decreases* with *increasing* order of magnitude in h.

You can understand why the CDS is more accurate than either FDS or BDS by looking at Figure 5.1; the central difference gives a better estimate of the gradient (tangent line) at the point x than the forward or backward difference.

Higher order derivatives

It is possible to find numerical approximations to 2nd and higher order derivatives too. A 2nd order accurate version of d^2y/dx^2 is

$$\boxed{\tilde{y}''(x) \equiv \frac{y(x+h) - 2y(x) + y(x-h)}{h^2} = y''(x) + \mathcal{O}(h^2).} \tag{5.13}$$

This can be obtained by adding together the 4th order Taylor expansions for $y(x+h)$ and $y(x-h)$, i.e., equations (5.9) and (5.10) with one more term. Another way to get (5.13) is to take the central difference versions of 1st order derivative at $x+h/2$ and $x-h/2$, and then find the central difference of these;

$$\tilde{y}''(x) = \frac{\tilde{y}'_c(x + h/2) - \tilde{y}'_c(x - h/2)}{h}\,.$$

Three points, $y(x-h)$, $y(x)$ and $y(x+h)$, are needed to get $\tilde{y}''$, the finite difference approximation of y''. In general, to get the finite difference approximation $\tilde{y}^n$ requires at least $n+1$ points and going further away from x, e.g., $y(x+2h)$, $y(x-2h)$, $y(x+3h)$, $y(x-3h)$, etc.

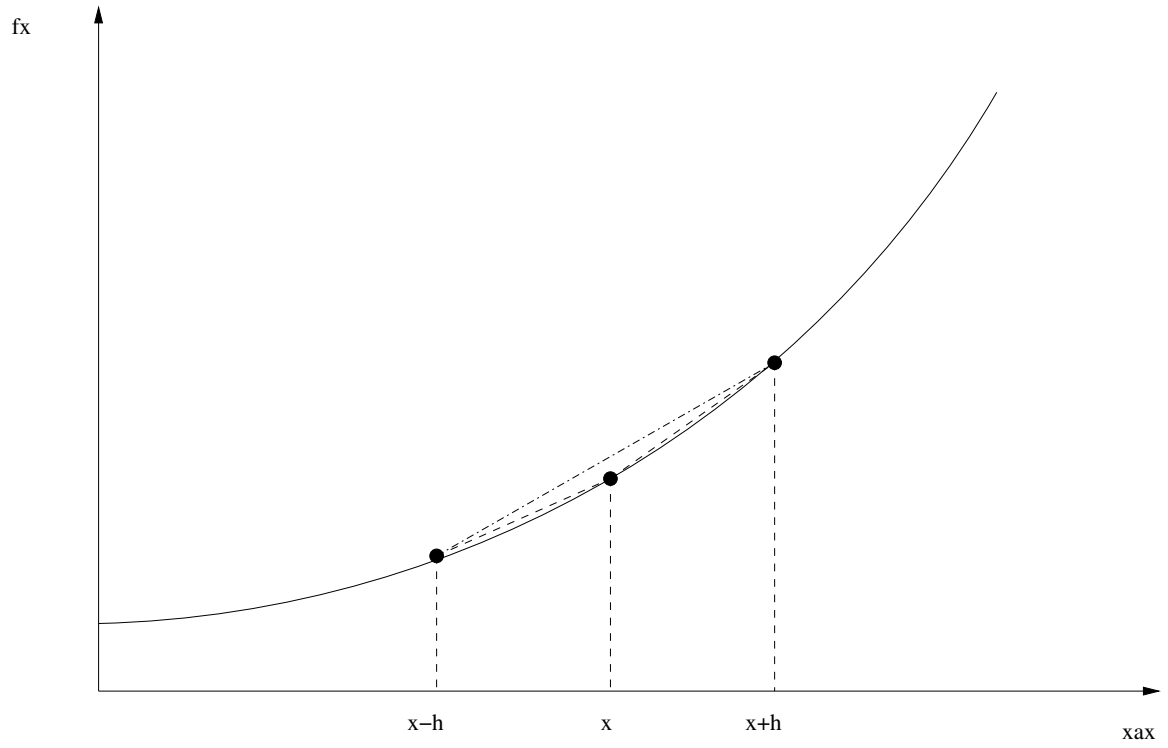

Figure 5.1: Forward, backward, and central difference schemes for approximating the derivative of the function $y(x)$ at the point x

5.2 Numerical Integration

One of the most commonly-encountered problems in everyday physics research is to efficiently evaluate a definite integral

$$I = \int_a^b f(x)\mathrm{d}x. \tag{5.14}$$

Numerical integration is an entire sub-field of numerical analysis, with numerous sophisticated methods and codes available. These basically all fall into three main categories: quadrature, ODE and Monte Carlo methods. We will cover ODE methods in Chapter 10 and Monte Carlo integration in Chapter 7. In this Chapter, we will deal predominantly with Newton-Cotes quadrature methods. We will also see how to transform badly-behaved integrals into a form that is more conducive to numerical evaluation.

5.2.1 Quadrature Methods

The basic idea of quadrature methods is to divide the integral into a number of sub-regions, and integrate over them independently. In a single dimension, this boils down to approximating the integral as a number of rectangle-like sub-regions, as shown in Fig. 5.2 – basically a fancy Riemann sum. The reason we say 'fancy' here is that, unlike in a Riemann sum, the tops of the rectangles are generally not taken to be flat, but rather slanted or with an even more complicated shape, in order to better approximate the behaviour of the integrand between samples.

There are thus three main decisions to be made in designing a quadrature algorithm:

(a) The number of samples to take of the integrand $f(x)$
(b) The distribution of the samples over the integration domain, i.e. the stepsize between samples h, and its variation with x
(c) The interpolating function to assume at the tops of the rectangle-like shapes.

The simplest methods in this class are the *Newton-Cotes rules*, which use a constant stepsize h; quadrature methods with a variable h across the integration domain are referred to as *Gaussian*

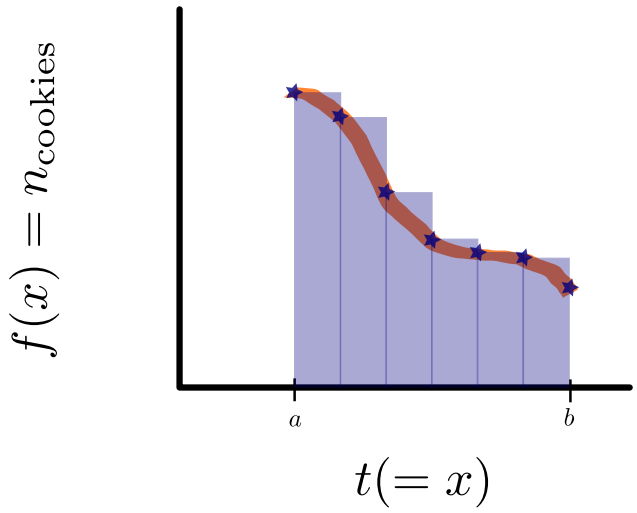

Figure 5.2: Basic anatomy of numerical integration by quadrature: the integral is divided up into rectangle-like shapes, the integrand is sampled at their edges, and the final integral is obtained by assuming some interpolating function passing between the sampled points. Cookies for dramatic effect only.

quadrature. The example shown in Fig. 5.2 is therefore in fact a Newton-Cotes method. Newton-Cotes rules take a number of equal-spaced samples of the integrand, and then estimate the overall integral using a weighted sum of the samples. The different weightings correspond to different interpolating functions across the tops of the rectangles. The next three subsections deal with three different Newton-Cotes methods; the difference between them is essentially in the choice of interpolating function.

The Trapezoidal Rule

Apart from a basic Riemann sum, the simplest way to approximate the integrand between samples is to interpolate between them linearly. This is the so-called *Trapezoidal Rule*. The Trapezoidal Rule is a *two-point* rule, in that it estimates the integral in a region between $x = x_i$ and $x = x_{i+1}$ using the value of the integrand at only two points, namely the two integration limits:

$$\int_{x_i}^{x_{i+1}} f(x)\,\mathrm{d}x = h\left[\frac{1}{2}f(x_i) + \frac{1}{2}f(x_{i+1})\right] + O\left(h^3\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\right). \tag{5.15}$$

To compute an entire integral however, we need to patch together many instances of the Trapezoidal Rule into the *Extended Trapezoidal Rule*:

$$\int_{x_0}^{x_{n-1}} f(x)\,\mathrm{d}x = h\left[\frac{1}{2}f(x_0) + f(x_1) + f(x_2) + \ldots + f(x_{n-2}) + \frac{1}{2}f(x_{n-1})\right] + O\left(nh^3\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\right), \tag{5.16}$$

where the stepsize $h = \frac{(x_{n-1}-x_0)}{n}$ for n samples.

This rule provides an estimate of the integral using n samples of the integrand, all connected via linear interpolation according to the Trapezoidal Rule. Turning this into a useful integration algorithm is then just a matter of wrapping it in a loop that steadily increases the number of samples until some convergence criterion is reached.

So, the basic algorithm for computing a definite integral using the Trapezoidal Rule to some desired relative accuracy ϵ is

(a) Evaluate $f(a)$ and $f(b)$
(b) Use these as a first estimate $I_1 = h_1\frac{1}{2}\left[f(a) + f(b)\right]$
(c) Evaluate the midpoint $f(\frac{a+b}{2})$
(d) Use this to update your estimate $I_2 = h_2\left[\frac{1}{2}f(a) + f(\frac{a+b}{2}) + \frac{1}{2}f(b)\right]$
(e) If $\left|\frac{I_2-I_1}{I_1}\right| < \epsilon$, terminate (convergence has been reached).

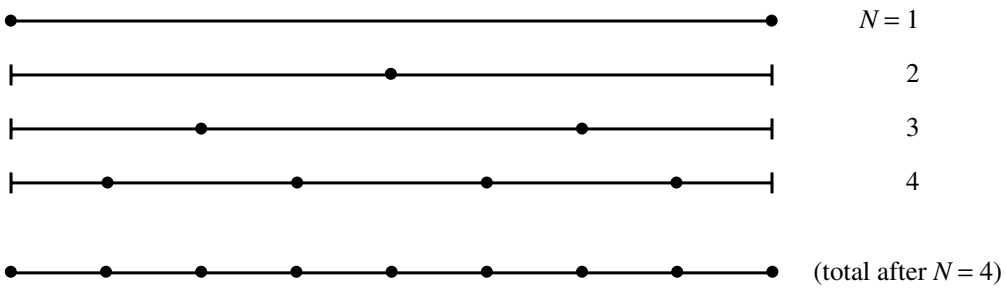

Figure 5.3: Sampling strategy for successive iterations of the Extended Trapezoidal Rule. Each new step fills in the midpoints between the previous points, never evaluating the integrand at the same value of x twice.

(f) If not, keep filling in intermediate points and making more trapezoids until the convergence criterion is satisfied.

Note in particular the use of the word 'intermediate' – there is no reason to ever evaluate the integrand twice at the same value of x. The idea is to decrease h by a factor of 2 at each iteration (Fig. 5.3), so that each successive iteration of the Extended Trapezoidal Rule can be built up from the previous one by simply reweighting the old estimate to account for the change in h, and adding in the new samples at $x = x_1, x_3, x_5, \ldots, x_{n-2}$ (remembering that our samples are indexed from 0 to $n-1$, not 1 to n):

$$T_{j+1} = \frac{1}{2}T_j + \frac{1}{2}h_j \sum_{i=1}^{2^{j-1}} f\left(x_0 + \frac{2i-1}{2}h_j\right), \tag{5.17}$$

$$h_{j+1} = \frac{h_j}{2}. \tag{5.18}$$

Simpson's Rule

The next level of sophistication is to add in an additional point to the basic Newton-Cotes rule, and build an extended rule from this instead. The rule goes by the name of *Simpson's Rule*, and looks like

$$\int_{x_i}^{x_{i+2}} f(x)\,\mathrm{d}x = h\left[\frac{1}{3}f(x_i) + \frac{4}{3}f(x_{i+1}) + \frac{1}{3}f(x_{i+2})\right] + O\left(h^5\frac{\mathrm{d}^4 f}{\mathrm{d}x^4}\right). \tag{5.19}$$

We can see immediately that this three-point rule is no longer doing linear interpolation. In fact, with the additional point, we can now uniquely define a polynomial of one degree higher, i.e. a quadratic. Simpson's Rule therefore improves on the Trapezoidal Rule by approximating the tops of the equal-width sub-integrals with quadratic curves rather than straight lines.

The corresponding *Extended Simpson's Rule* can then be build up in the same manner as for the Extended Trapezoidal Rule, by patching together successive copies of the basic 3-point Simpson's Rule:

$$\begin{aligned}\int_{x_0}^{x_{n-1}} f(x)\,\mathrm{d}x =& h\left[\frac{1}{3}f(x_0) + \frac{4}{3}f(x_1) + \frac{2}{3}f(x_2) + \frac{4}{3}f(x_3)\ldots\right. \\ &\left.\ldots + \frac{4}{3}f(x_{n-4}) + \frac{2}{3}f(x_{n-3}) + \frac{4}{3}f(x_{n-2}) + \frac{1}{3}f(x_{n-1})\right] \qquad (5.20)\\ &+ O\left(nh^5\frac{\mathrm{d}^4 f}{\mathrm{d}x^4}\right), \qquad (5.21)\end{aligned}$$

where again the stepsize $h = \frac{(x_{n-1}-x_0)}{n-1}$ for n samples. Note that the 3-point sub-rules here do not overlap at all, and are simply patched together end-to-end.

The implementation of the Extended Simpson's Rule in a full integration algorithm follows that for the Trapezoidal Rule fairly closely, except in two important ways. The first is pretty obvious

– one needs to start with 3 points in the initial step, at the two limits of integration and the midpoint. The second is more subtle, and interesting. Simpson's Rule is specifically designed so that the higher-order interpolant results in a higher-order method, i.e. so that the error terms of order h^3 and h^4 from the trapezoidal rule cancel. This means that it can be achieved by taking two successive iterations of the Trapezoidal Rule and combining them with the appropriate weights to cancel the lower-order errors induced by the two lower-order steps:

$$S_j = \frac{4}{3}T_{j+1} - \frac{1}{3}T_j. \tag{5.22}$$

This is quite cute, as it means that an implementation of the Extended Simpson's Rule can be easily piggybacked onto an existing implementation of the Extended Trapezoidal Rule. This gives a more accurate result, more or less 'for free' in terms of computational time.

Romberg integration

Unsurprisingly, successive iterations of Simpson's Rule can also be combined in such a way as to cause the $\mathcal{O}(h^5)$ and even higher-order errors to cancel. The trade-off of this sort of exercise though is that the interpolating function across the tops of the sub-integrals becomes steadily more non-local the higher order one goes to. Much as higher-order spline interpolation starts to become a bit dicey due to non-local effects causing substantial higher derivatives to produce large excursions and 'ringing', higher-order Newton-Cotes schemes can also start to suffer stability problems with rapidly-varying integrands. This isn't catastrophic, but it does offset any real speed gains that one can achieve from them, as it can mean that they need smaller h than one might naively expect, to avoid non-local effects.

Romberg integration is an algorithm that extrapolates the Newton-Cotes strategy to arbitrarily high-order accuracy, at the same time as extrapolating the stepsize h to zero. This sounds almost too good to be true, and it basically is: except for extremely smooth functions, Romberg integration doesn't usually provide much speedup compared to a simple Simpson's Rule integrator – and it adds quite a lot more complexity. For moderately-varying integrands, variable-h methods such as ODE integration usually provide more significant speedup than Romberg integration.

5.2.2 Improper Integrals

Everything we have seen so far in this Chapter corresponds to a *closed* Newton-Cotes rule. This means that the integrand is evaluated directly at the edge of every sub-interval. However, it is also possible to construct *open* rules, which evaluate the integral in an interval without directly evaluating it at the limits. One such rule is the *midpoint rule*,

$$\int_{x_0}^{x_1} f(x)\,\mathrm{d}x = h\left[f\left(\frac{x_0+x_1}{2}\right)\right], \tag{5.23}$$

which simply does a central Reimann sum, approximating the integrand in the interval by its value at the midpoint of the interval.

Open rules can be particularly useful when it is difficult or impossible to evaluate the integrand at one of the limits of integration. This may happen if the integrand is undefined at the limit, e.g. $\frac{0}{0}$, or if the integral diverges at the limit.

Notice that the midpoint rule only covers a small sub-domain of integration, just like the 2-point Trapezoidal and 3-point Simpson's Rules introduced earlier. To use it for doing a real integration, we need to patch it together with many other basic rules to make an extended rule, and put it inside an iterative algorithm that increases the number of sub-domains until we can get a convergent result. However, there is nothing that forces us to patch together only rules of exactly the same kind when creating our extended rules. For dealing with an integrand that is well-behaved right up to the limit of integration, but undefined exactly on the limit, a good approach is therefore simply to patch a single copy of the midpoint rule (at the limit where the integrand is undefined) on to an extended Simpson's rule (for use in the interior and at the other limit).

In the case where an integrand diverges as one approaches one of the limits, or where one end of the integral extends to infinity, a little more care is needed.

Take the case of integrating to infinity first. Here, we have an integral

$$I = \int_a^b f(x)\mathrm{d}x. \tag{5.24}$$

where $a = -\infty$ and $b < 0$ or $b = \infty$ and $a > 0$. The best thing to do is to transform the asymptotic part of the integral via the transformation $x \to \frac{1}{t}$, which gives

$$\int_a^b f(x)\,\mathrm{d}x = \int_{1/b}^{1/a} \frac{1}{t^2} f\left(\frac{1}{t}\right)\,\mathrm{d}t, \tag{5.25}$$

i.e. making the previously badly-behaved limit correspond to $t = 0$. The integrand still can't be evaluated at this limit, but the asymptotic behaviour has been compressed into a finite range, allowing us to employ an open rule on the transformed integral, and simply avoid evaluating the integrand at exactly $t = 0$. Note that this trick only works for one limit at a time, and only when a and b have the same sign; otherwise, you will need to split the integral at some opportune location and do the two parts separately. This trick is also inefficient if the entire integrand is not asymptotically decreasing at least as quickly as x^{-2}; in this case it also pays to split the integral, so as to isolate the part where it *is* dropping faster than x^{-2}, and use the transformation only on that part (and a regular Simpson's Rule on the rest).

The case of a divergent integral requires even more thought. In this case, we have an integrable singularity at some special value of x; call this x_s. If we know the value of x_s, we can proceed fairly safely: split the integral at the singularity. Now you have two integrals each with singularities at the edges of their domains. We can then transform the nasty parts of each integral as $x \to \alpha$, with

$$\alpha = t^{\frac{1}{1-\gamma}} + x_s \quad \text{for lower limit } x_s \tag{5.26}$$

$$\alpha = x_s - t^{\frac{1}{1-\gamma}} \quad \text{for upper limit } x_s. \tag{5.27}$$

This gives (with either $a' = x_s$ or $b' = x_s$)

$$\int_{a'}^{b'} f(x)\,\mathrm{d}x = \frac{1}{1-\gamma} \int_0^{(b'-a')^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}} f(\alpha)\,\mathrm{d}t, \tag{5.28}$$

which we can happily integrate with any extended rule that is open at $t = 0$. Here γ is a power-law index that we are basically free to choose to be anything between 0 and 1. Note that the most efficient integration will result if we choose γ to match the slope of our divergence as closely as possible. That is, if our function behaves like $f(x) \to (x - x_s)^{-\beta}$ as $x \to x_s$, then the most efficient choice is $\gamma = \beta$.

If you run into a situation where you know that there is a singularity somewhere in $a < x_s < b$, but don't actually know the value of x_s, then you're in significantly more trouble. In this case, you need to try to somehow 'sneak past' the singularity using variable-stepsize methods such as ODE integration, Gaussian quadrature or Monte Carlo methods. The challenge of course is that the area around $x = x_s$ generally contributes significantly to the total integral, so you *also* need to sample this area fairly densely to get a converged result – but at the same time, you need to avoid evaluating the integrand so close to the pole that the result overflows the floating-point type you're using. It's not a lot of fun. In most cases, it's worth putting some time into instead trying to transform your problem so that the singularity either goes away, or sits at some known value of some suitably transformed integration variable.

Chapter 6

Solving Algebraic Equations

Outline of Section

- Introduction
- Linear Algebraic Equations
- Direct Methods
- Iterative Methods: general concepts
- Iterative Methods: specific methods
- Eigensystems
- Non-linear algebraic equations of one variable
- Non-linear algebraic equations of multiple variables

6.1 Introduction

In this section we will look at solving simultaneous algebraic equations. By "algebraic", we mean equations which do not involve derivatives; those are handled later in the course.

Consider the case of one unknown x where we generally need one equation to define the solution. By gathering all terms on the left hand side, any equation can be written in the form

$$f(x) = 0 \tag{6.1}$$

For N unknowns x_i, where $i = 0, N-1$, we will often use the notation where the x_i are written as a vector $\vec{x}$. For N unknowns we generally need N equations which again can be written as

$$f_i(\vec{x}) = 0 \qquad \text{where} \qquad i = 0, N-1 \tag{6.2}$$

or most compactly as

$$\vec{f}(\vec{x}) = 0 \tag{6.3}$$

Hence, all algebraic problems can be expressed as function root-finding problems.

This chapter first considers linear equations and later non-linear equations. Linear equations have terms with powers of at most x_i, i.e. not x_i^2 or higher. The general linear one variable case can be written as $Ax = b$ so

$$f(x) = Ax - b = 0 \qquad \text{giving} \qquad x = \frac{b}{A} = A^{-1}b \tag{6.4}$$

There is one and only one solution, except when $A = 0$. For N variables and M equations, the general case is

$$\begin{aligned} A_{00}\,x_0 + A_{01}\,x_1 + A_{02}\,x_2 + ... + A_{0\,N-1}\,x_{N-1} &= b_1, \\ A_{10}\,x_0 + A_{11}\,x_1 + A_{12}\,x_2 + ... + A_{1\,N-1}\,x_{N-1} &= b_2\,, \\ ... &= ..., \\ A_{M-1\,0}\,x_0 + A_{M-1\,1}\,x_1 + A_{M-1\,2}\,x_2 + ... + A_{M-1\,N-1}\,x_{N-1} &= b_{M-1}, \end{aligned} \tag{6.5}$$

which can be written as the matrix operation

$$\sum_j A_{ij}x_j = b_i \qquad \text{or} \qquad \mathbf{A}\cdot\vec{x} = \vec{b}, \tag{6.6}$$

where $\mathbf{A}$ is an $M\times N$ matrix, $\vec{x}$ is a vector with N components and $\vec{b}$ is a vector with M components. Any coefficient A_{ij} in the above simultaneous equations becomes the element of the matrix located at row i (the first subscript) and column j (the second subscript). We want to solve for the unknown variables $\vec{x}$ for a given matrix $\mathbf{A}$ and right-hand side $\vec{b}$. M is the number of equations in the system and N is the number of unknowns we have to solve for.

In the following, we will focus on the case where $M = N$, i.e. $\mathbf{A}$ is a square matrix. The matrix equation $\mathbf{A}\cdot\vec{x} = \vec{b}$ is called *homogeneous* if $\vec{b} = 0$ and *inhomogeneous* otherwise. The homogeneous case only has non-trivial solutions (i.e. not $\vec{x} = 0$) if $\det\mathbf{A} = 0$. The solutions are not unique and need specific methods. For the inhomogeneous case, if all the equations are **linearly independent** then there should exist a unique solution for $\vec{x}$. However, if some equations are degenerate (i.e. can be written as linear combination of a smaller number of other equations) then effectively the system has fewer equations than unknowns (i.e., $M < N$) and there may not be a solution (or not a unique solution). This will be evident in the matrix $\mathbf{A}$ being singular (some eigenvalues are zero) and so having a vanishing determinant. We will only consider cases of non-singular $\mathbf{A}$ in the next few sections.

All methods may involve manipulating the matrix to shift around elements so that the system can be more easily solved. These manipulations involve swapping and scaling rows; these 'row-swapping' and 'row-scaling' operations are known as 'pivoting', and the best approach can in general be tricky to spot.

6.2 General considerations for linear equations

The need to solve a matrix equation such as Eq. (6.6) arises frequently in many numerical methods. In section (9.7) we will see them in implicit finite difference methods for solving sets of coupled linear ODEs. In section (12) we will encounter them in solving PDEs via finite difference methods; the $N\times N$ matrices there can be very large, with N equal to the number of spatial grid points. Matrix equations will also occur in solving boundary value problems in section (11) and minimisation of functions in section (8). Of course matrix equations arise directly in many physical problems too, such as quantum mechanics and classical mechanics.

Mathematically, in principle the solution is trivial, as

$$\mathbf{A}\cdot\vec{x} = \vec{b} \qquad \text{means} \qquad \vec{x} = \mathbf{A}^{-1}\cdot\vec{b} \tag{6.7}$$

The methods we will study generally solve for $\vec{x}$ without necessarily finding $\mathbf{A}^{-1}$ explicitly, although some do give $\mathbf{A}^{-1}$ as part of the method.

However, any method which can solve for $\vec{x}$ can be used to find $\mathbf{A}^{-1}$ as well, because the system of equations can be extended to find more than one unknown vector. Consider two equations

$$\mathbf{A}\cdot\vec{x} = \vec{b} \qquad \text{and} \qquad \mathbf{A}\cdot\vec{y} = \vec{c} \tag{6.8}$$

We can form a two-column matrix from $\vec{x}$ and $\vec{y}$, usually written as $(\vec{x}|\vec{y})$, and do the same for $\vec{b}$ and $\vec{c}$, written as $\left(\vec{b}|\vec{c}\right)$. Both equations can then be written as one equation

$$\mathbf{A}\cdot(\vec{x}|\vec{y}) = (\vec{b}|\vec{c}) \tag{6.9}$$

Extending this to N vectors gives an $N\times N$ matrix $\mathbf{X} = (\vec{x}|\vec{y}|\vec{z}|\dots)$ to be solved for, and a known matrix $\mathbf{B} = \left(\vec{b}|\vec{c}|\vec{d}|\dots\right)$. The equation can then be written as

$$\mathbf{A}\cdot\mathbf{X} = \mathbf{B}, \tag{6.10}$$

Each column of $\mathbf{X}$ is one of the vectors like $\vec{x}$ of the N equations (where $0 \leq i \leq N-1$). Similarly, each column of $\mathbf{B}$ is a right hand-side vector, like $\vec{b}$, of one of the equations. If we can solve the original equation for $\vec{x}$, we can repeat this solution N times to find all the columns in $\mathbf{X}$. The relevance to finding $\mathbf{A}^{-1}$ is that we can set $\mathbf{B}$ to the identity matrix, so that

$$\mathbf{A} \cdot \mathbf{X} = \mathbf{I} = \mathbf{A} \cdot \mathbf{A}^{-1} \tag{6.11}$$

Hence the solution will be $\mathbf{X} = \mathbf{A}^{-1}$. This technique will work for any method, although it is N times slower than solving for $\vec{x}$ directly.

We will also briefly look at eigenvalue equations of a matrix, which are of the form

$$\mathbf{A} \cdot \vec{v} = \lambda \vec{v}, \tag{6.12}$$

where λ is the eigenvalue to be found and $\vec{v}$ is the eigenvector. Eigenvalues are important for convergence in some equation-solving methods. This is related to the homogeneous equation case since the equation above can be written as

$$(\mathbf{A} - \lambda \mathbf{I}) \cdot \vec{v} = 0 \tag{6.13}$$

The need to find eigenvalues of a matrix as given by Eq. (6.12) is also a common task both in numerical methods and in physics problems. For example, we will encounter one such case in section (6.4.3) and another in the stability analysis of finite difference methods for coupled ODEs in section (10.4).

6.3 Direct methods

Direct methods estimate the solution and/or inverse matrix without using series approximations (which have truncation errors), so if there were no calculational errors due to floating point rounding, the result would be exact. We will first consider some special cases of matrices which have specific methods for finding solutions. If the matrix is not one of these special cases, we have to use more general methods.

6.3.1 Special cases

The easiest special case is when the matrix is a **diagonal matrix**, i.e. all non-diagonal elements are zero, or can be made diagonal by pivoting. As long as all the diagonal elements are non-zero, the inverse is obvious.

$$\mathbf{A} = \begin{pmatrix} D_{00} & 0 & 0 \\ 0 & D_{11} & 0 \\ 0 & 0 & D_{22} \end{pmatrix} \qquad \text{gives} \qquad \mathbf{A}^{-1} = \begin{pmatrix} 1/D_{00} & 0 & 0 \\ 0 & 1/D_{11} & 0 \\ 0 & 0 & 1/D_{22} \end{pmatrix} \tag{6.14}$$

and so the solution for all i is

$$x_i = \frac{b_i}{D_{ii}} \tag{6.15}$$

Another special case is when the matrix $\mathbf{A}$ is a **lower diagonal matrix**, e.g.

$$\mathbf{A} \cdot \vec{x} = \begin{pmatrix} L_{00} & 0 & 0 \\ L_{10} & L_{11} & 0 \\ L_{20} & L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \end{pmatrix} . \tag{6.16}$$

Writing the first equation out explicitly gives

$$L_{00} x_0 = b_0 \qquad \text{so} \qquad x_0 = \frac{b_0}{L_{00}} \tag{6.17}$$

Given we now know x_0, the second equation gives

$$L_{10} x_0 + L_{11} x_1 = b_1 \qquad \text{so} \qquad x_1 = \frac{b_1 - L_{10} x_0}{L_{11}} \tag{6.18}$$

Similarly, as we now know x_0 and x_1

$$L_{20}x_0 + L_{21}x_1 + L_{22}x_2 = b_2 \qquad \text{so} \qquad x_2 = \frac{b_2 - L_{20}x_0 - L_{21}x_1}{L_{22}} \tag{6.19}$$

The trick here is that it is trivial to solve a triangular system, i.e. one where each equation is a function of one more unknown than the previous one. For a general $N \times N$ lower diagonal matrix, the solution is

$$x_0 = \frac{b_0}{L_{00}} \qquad \text{and} \qquad x_{i>0} = \frac{b_i - \sum_{j=0}^{i-1} L_{ij}x_j}{L_{ii}} \tag{6.20}$$

where, because we need to ensure the x_j used to calculate later values of x_i have been correctly evaluated, the x_i must be evaluated in the order of increasing i. This is called **forward substitution** as we effectively bring known earlier x_j values forward to solve for later x_i values.

As might be expected, a similar method can be applied if $\mathbf{A}$ is an **upper diagonal matrix**.

$$\mathbf{A} \cdot \vec{x} = \begin{pmatrix} U_{00} & U_{01} & U_{02} \\ 0 & U_{11} & U_{12} \\ 0 & 0 & U_{22} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \end{pmatrix} . \tag{6.21}$$

The critical difference is that we have to start from the last equation, not the first, and evaluate x_i in descending order of i. The equations are then

$$x_{N-1} = \frac{b_{N-1}}{U_{N-1\,N-1}} \qquad \text{and} \qquad x_{i<N-1} = \frac{b_i - \sum_{j=i+1}^{N-1} U_{ij}x_j}{U_{ii}} \tag{6.22}$$

This is called **backward substitution**. Note: in both cases, $\mathbf{A}^{-1}$ is not found explicitly.

6.3.2 Cofactor inversion method

This is probably a general method you have seen before. It finds the inverse of $\mathbf{A}$ by a direct method, after which you can solve for $\vec{x}$.

For every element A_{ij}, construct the *minor* matrix, which is the $(N-1) \times (N-1)$ matrix with the row i and column j containing the element A_{ij} removed. The cofactor C_{ij} is the determinant of the minor multiplied by a factor $(-1)^{i+j}$. Form a matrix of all cofactors and transpose it and then divide by the determinant of $\mathbf{A}$ to get the inverse.

The major issue with this method is that it scales as $N!$ which is much worse than other methods and becomes unfeasible very quickly as the matrices get larger. Hence, other general methods are always used in practise.

6.3.3 Gaussian and Gauss-Jordan elimination

These are two related general direct methods for solving the linear equations; the **Gaussian elimination** method solves for $\vec{x}$, while **Gauss-Jordan elimination** solves for $\mathbf{A}^{-1}$. These are basically what you would do to solve using pen and paper. They work by adding (or subtracting) multiples of different rows to each other (in addition to any pivoting needed). Each of these manipulations can be shown to be equivalent to multiplying from the left by a matrix.

For Gaussian elimination, these manipulations are applied to both $\mathbf{A}$ and $\vec{b}$ until the matrix $\mathbf{A}$ is reduced to a unit matrix. Let the product of all the manipulation matrices be $\mathbf{M}$, so this means the end result is $\mathbf{M} \cdot \mathbf{A} = \mathbf{I}$ and hence

$$\mathbf{A} \cdot \vec{x} = \vec{b} \qquad \text{becomes} \qquad \mathbf{M} \cdot \mathbf{A} \cdot \vec{x} = \mathbf{M} \cdot \vec{b} \qquad \text{so} \qquad \vec{x} = \mathbf{M} \cdot \vec{b} \tag{6.23}$$

Hence, the result of all the manipulations on $\vec{b}$ gives the solution.

For Gauss-Jordan elimination, the manipulation matrices are applied to both $\mathbf{A}$ and the unit matrix $\mathbf{I}$, again until the matrix $\mathbf{A}$ is reduced to a unit matrix. This works because

$$\mathbf{A} \cdot \mathbf{A}^{-1} = \mathbf{I} \qquad \text{becomes} \qquad \mathbf{M} \cdot \mathbf{A} \cdot \mathbf{A}^{-1} = \mathbf{M} \cdot \mathbf{I} \qquad \text{so} \qquad \mathbf{A}^{-1} = \mathbf{M} \cdot \mathbf{I} \tag{6.24}$$

so the result of all the manipulations on $\mathbf{I}$ give the inverse of $\mathbf{A}$.

In both methods, the entries in $\mathbf{A}$ inform your (or your code's) choices about what manipulations to perform, and the same manipulations on $\vec{b}$ (in Gaussian elimination) or $\mathbf{I}$ (in Gauss-Jordan elimination) 'shadow' those moves, and thereby become transformed to the answer you want.

6.3.4 LU Decomposition

LU decomposition is another general direct method which uses lower ($\mathbf{L}$) and upper ($\mathbf{U}$) diagonal matrices. We considered the special cases of $\mathbf{A}$ being in either of these forms previously. Of course, it is rare that we would be lucky enough to have a matrix $\mathbf{A}$ which is either a lower or an upper diagonal matrix. However, if we assume that the matrix can generally be factorised ("decomposed") into upper and lower triangular matrices

$$\mathbf{A} \equiv \mathbf{L} \cdot \mathbf{U} \tag{6.25}$$

then we can write the equation as

$$\boxed{\mathbf{A} \cdot \vec{x} = \mathbf{L} \cdot \mathbf{U} \cdot \vec{x} = \mathbf{L} \cdot \vec{y} = \vec{b},} \tag{6.26}$$

where we have defined $\vec{y} \equiv \mathbf{U} \cdot \vec{x}$. We can now carry out a forward substitution to solve $\mathbf{L} \cdot \vec{y} = \vec{b}$ for $\vec{y}$ and then use a backward substitution to solve $\mathbf{U} \cdot \vec{x} = \vec{y}$ for $\vec{x}$. Hence, if we can decompose $\mathbf{A}$, we can solve for $\vec{x}$.

Here we briefly outline how such a decomposition can be performed; this method is called "Crout's algorithm". The matrix equation Eq. (6.25) will be something like

$$\begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{00} & 0 & 0 \\ L_{10} & L_{11} & 0 \\ L_{20} & L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{00} & U_{01} & U_{02} \\ 0 & U_{11} & U_{12} \\ 0 & 0 & U_{22} \end{pmatrix} \tag{6.27}$$

which represents $N \times N$ individual equations, with $N^2 + N$ unknown variables (all the L_{ij} and U_{ij} terms). This means that we have too many unknowns, so N of these variables must be specified arbitrarily. A common choice (called "Doolittle") chooses to fix the diagonal L_{ii} values to 1. An alternative choice ("Crout") fixes the U_{ii} to 1. (Do not confused the latter with Crout's algorithm, which can be adapted to either choice.)

Specifically here choosing Doolittle, so $L_{ii} = 1$, multiplying each column of $\mathbf{U}$ by the first row of $\mathbf{L}$ gives in turn

$$L_{00}U_{00} = U_{00} = A_{00}, \qquad L_{00}U_{01} = U_{01} = A_{01}, \qquad L_{00}U_{02} = U_{02} = A_{02} \tag{6.28}$$

Given U_{00} is known, multiplying the first column of $\mathbf{U}$ by the second row of $\mathbf{L}$ gives

$$L_{10}U_{00} = A_{10} \qquad \text{so} \qquad L_{10} = \frac{A_{10}}{U_{00}} \tag{6.29}$$

and similarly, multiplying the second column of $\mathbf{U}$ by the second row of $\mathbf{L}$ gives

$$L_{10}U_{01} + L_{11}U_{11} = L_{10}U_{01} + U_{11} = A_{11} \qquad \text{so} \qquad U_{11} = A_{11} - L_{10}U_{01} \tag{6.30}$$

The basic idea of solving the equations in the correct order, so there is only one unknown at a time, is very similar to the concept of forward and backward substitution. The general solution requires for each column $j = 0, 1, 2, \ldots, N-1$ in turn starting from 0, we first to solve for the U_{ij} in that column; these have $i \leq j$ so we solve for $i = 0, 1, 2, \ldots, j$ using

$$U_{ij} = A_{ij} - \sum_{k=0}^{i-1} L_{ik}U_{kj} \tag{6.31}$$

We must then solve for the L_{ij} in the same column; these have $i \geq j$ but L_{ii} is already known, so for $i = j+1, j+2, \ldots, N-1$ using

$$L_{ij} = \frac{1}{U_{jj}} \left(A_{ij} - \sum_{k=0}^{j-1} L_{ik} U_{kj} \right). \tag{6.32}$$

When all the U_{ij} and L_{ij} in the column are evaluated, the next column must be processed in the same way. The above procedure must be done in the stated order as it determines all the **L** and **U** values by the time that they are needed in the algorithm.

A significant benefit of LU decomposition is that it only depends on **A**; if $\mathbf{A} \cdot \vec{x} = \vec{b}$ is to be solved many times for different $\vec{b}$, it is advantageous to spend the time decomposing **A** once first. As an additional benefit, once the matrix is LU-decomposed using the Doolittle choice, the determinant is easy to calculate because (quoted here without proof)

$$\det \mathbf{A} = \prod_j U_{jj} \tag{6.33}$$

As an alternative to Crout's algorithm, LU decomposition can be done by a process identical to Gaussian elimination but instead of manipulating **A** to get a unit matrix, the manipulations are chosen to set all the elements below the diagonal to zero, i.e. to obtain an upper diagonal matrix (although it will not necessarily be the *same* matrix as from Crout's algorithm due to the arbitrary choice of N variables). Hence we have $\mathbf{M} \cdot \mathbf{A} = \mathbf{U}$ and applying the same operations to $\vec{b}$ means

$$\mathbf{A} \cdot \vec{x} = \vec{b} \qquad \text{becomes} \qquad \mathbf{M} \cdot \mathbf{A} \cdot \vec{x} = \mathbf{M} \cdot \vec{b} \qquad \text{so} \qquad \mathbf{U} \cdot \vec{x} = \mathbf{M} \cdot \vec{b} \tag{6.34}$$

Hence defining $\mathbf{M} \cdot \vec{b} = \vec{y}$, we have $\mathbf{U} \cdot \vec{x} = \vec{y}$. Using the resulting **U** and $\vec{y}$ allows us to solve for $\vec{x}$ using backward substitution as before. Note, since $\mathbf{A} = \mathbf{L} \cdot \mathbf{U}$, this also means $\mathbf{M} = \mathbf{L}^{-1}$.

6.4 Iterative Methods: general concepts

We now look at *indirect* methods for solving $\mathbf{A} \cdot \vec{x} = \vec{b}$. Given that direct inversion of large matrices explicitly can be prohibitive even on the fastest computers, indirect methods use approximations to speed up the solution. Specifically, they use *iteration* to converge onto a solution from an initial guess and hence are more often called **iterative methods**. The methods will include truncation errors as we can never iterate to get all terms of an infinite series. However, they are useful when the truncation errors can be made smaller than the accumulated rounding errors which arise in direct methods. For large matrices, the indirect methods turn out to be more suitable (faster or with smaller memory footprints) than direct methods.

The basic idea is the same for all methods. An approximation to **A** is made $\mathbf{A} \approx \mathbf{B}$, and this is used to find an approximate solution, effectively using the inverse of **B**. The remaining part of **A** which was not included in the approximation, $\mathbf{S} = \mathbf{A} - \mathbf{B}$, is then used to make an iterative correction. This works best when the **B** matrix contains "big" elements and the **S** matrix contains "small" elements, as **B** is then a good approximation to **A** and the iteration converges quickly.

The general concept can be understood by analogy with the Taylor expansion of the function $f(x) = 1/x$. Expanding this function around $x = 1$ by a change in x given by g gives

$$f(1+g) = (1+g)^{-1} \approx f(1) + g f'(1) + \frac{g^2}{2!} f''(1) + \frac{g^3}{3!} f'''(1) + \ldots \tag{6.35}$$

$$\approx 1 - g + g^2 - g^3 + \ldots \tag{6.36}$$

By taking more terms in this series, the result should become more accurate, as long as the series converges. (Note, it should be clear this series will not converge for $|g| \geq 1$.) The trick is to notice the series has a very useful property that adding each new term can be done by the same iterative

calculation on the previous result. Specifically, the approximations of $f(1+g)$ are successively

$$\begin{aligned} f^{(0)} &= 1 \\ f^{(1)} &= 1-g & &= 1-gf^{(0)} \\ f^{(2)} &= 1-g+g^2 & &= 1-gf^{(1)} \\ f^{(3)} &= 1-g+g^2-g^3 & &= 1-gf^{(2)} \\ f^{(i+1)} & & &= 1-gf^{(i)} \end{aligned} \tag{6.37}$$

The equivalent expression using matrices is effectively identical; defining a matrix $\mathbf{F} = (\mathbf{I}+\mathbf{G})^{-1}$ gives

$$\mathbf{F} \approx \mathbf{I} - \mathbf{G} + \mathbf{G}^2 - \mathbf{G}^3 + \ldots \tag{6.38}$$

and in the same way, the matrix $\mathbf{F}$ can be found by initially approximating it as $F^{(0)} = \mathbf{I}$ and then iterating using

$$\mathbf{F}^{(i+1)} = \mathbf{I} - \mathbf{G}\cdot\mathbf{F}^{(i)} \tag{6.39}$$

The matrix $\mathbf{G}$ is called the *update matrix* as it defines how the solutions are iterated.

To apply this to solving $\mathbf{A}\cdot\vec{x} = \vec{b}$, we set $\mathbf{A} = \mathbf{B}+\mathbf{S}$ and multiply our equation by $\mathbf{B}^{-1}$ to give

$$\mathbf{B}^{-1}\cdot\mathbf{A}\cdot\vec{x} = \mathbf{B}^{-1}\cdot(\mathbf{B}+\mathbf{S})\cdot\vec{x} = (\mathbf{I}+\mathbf{B}^{-1}\cdot\mathbf{S})\cdot\vec{x} = \mathbf{B}^{-1}\cdot\vec{b} \tag{6.40}$$

Defining $\mathbf{G} = \mathbf{B}^{-1}\cdot\mathbf{S}$, this leads to

$$(\mathbf{I}+\mathbf{G})\cdot\vec{x} = \mathbf{B}^{-1}\cdot\vec{b} \qquad \text{so} \qquad \vec{x} = (\mathbf{I}+\mathbf{G})^{-1}\cdot\mathbf{B}^{-1}\cdot\vec{b} = \mathbf{F}\cdot\mathbf{B}^{-1}\cdot\vec{b} \tag{6.41}$$

For each iteration of $\mathbf{F}$, we can find an approximate solution for $\vec{x}$. Since $\mathbf{F}^{(0)} = \mathbf{I}$, we start from $\vec{x}^{(0)} = \mathbf{B}^{-1}\cdot\vec{b}$, which is consistent with the zeroth-order approximation that $\mathbf{A} \approx \mathbf{B}$. The $i+1$ iteration will give

$$\vec{x}^{(i+1)} = \mathbf{F}^{(i+1)}\cdot\mathbf{B}^{-1}\cdot\vec{b} = (\mathbf{I}-\mathbf{G}\cdot\mathbf{F}^{(i)})\cdot\mathbf{B}^{-1}\cdot\vec{b} = \mathbf{B}^{-1}\cdot\vec{b} - \mathbf{G}\cdot\mathbf{F}^{(i)}\cdot\mathbf{B}^{-1}\cdot\vec{b} \tag{6.42}$$

The last term contains the expression for $\vec{x}^{(i)}$ so we can iterate the $\vec{x}$ solutions directly

$$\boxed{\vec{x}^{(i+1)} = \mathbf{B}^{-1}\cdot\vec{b} - \mathbf{G}\cdot\vec{x}^{(i)}} \tag{6.43}$$

without finding $\mathbf{F}$ explicitly, which makes this very convenient. Since $\mathbf{A}^{-1} = \mathbf{F}\cdot\mathbf{B}^{-1}$, a similar calculation gives an iterative equation for the inverse

$$\boxed{(\mathbf{A}^{-1})^{(i+1)} = \mathbf{B}^{-1} - \mathbf{G}\cdot(\mathbf{A}^{-1})^{(i)}} \tag{6.44}$$

We must also see how the errors on the solution change at each iteration. We define the error ϵ by

$$\vec{\epsilon}^{(i)} = \vec{x}^{(i)} - \vec{x} \qquad \text{and} \qquad \vec{\epsilon}^{(i+1)} = \vec{x}^{(i+1)} - \vec{x}, \tag{6.45}$$

where $\vec{x}$ is the exact solution. Hence

$$\vec{\epsilon}^{(i+1)} = \vec{x}^{(i+1)} - \vec{x} = \mathbf{B}^{-1}\cdot\vec{b} - \mathbf{G}\cdot\vec{x}^{(i)} - \vec{x} = \mathbf{B}^{-1}\cdot\vec{b} - \mathbf{G}\cdot\vec{\epsilon}^{(i)} - (\mathbf{I}+\mathbf{G})\cdot\vec{x} \tag{6.46}$$

But since for the exact solution $(\mathbf{I}+\mathbf{G})\cdot\vec{x} = \mathbf{B}^{-1}\vec{b}$, this means

$$\boxed{\vec{\epsilon}^{(i+1)} = -\mathbf{G}\cdot\vec{\epsilon}^{(i)}} \tag{6.47}$$

Hence, $\mathbf{G}$ also determines how the errors change on each iteration step; the so-called *error amplification*. For the iteration to converge, $\mathbf{G}$ must reduce the magnitude of all the errors at each step, as discussed in more detail later in section (6.4.3).

6.4.1 Benefits of iterative methods

Iterative methods for solving for $\vec{x}$ are usually faster than the direct inversion methods (such as LU decomposition). This is because we only need to carry out one matrix–vector multiplication (i.e. $\mathbf{G} \cdot \vec{x}^{(i)}$) at each step. The $\mathbf{B}^{-1} \cdot \vec{b} = \vec{x}^{(0)}$ calculation need only be done once and the vector subtraction at each step carries little extra computational cost in comparison to the multiplication.

In general, direct methods require $\mathcal{O}(N^3)$ operations which can be prohibitive. For iterative methods, the number of operations needed to perform the matrix–vector multiplication will be $\mathcal{O}(N^2)$. Hence to obtain the solution will be $\mathcal{O}(N^2 \times N_{\text{iter}})$, where N_{iter} is the number of iterations required to reach a chosen level of convergence. As long as N_{iter} is substantially less than N, an iterative solution scales much better than direct inversion methods. The key thing with iterative methods is to get quick convergence so that N_{iter} is as small as possible. This means choosing $\mathbf{B}$ to be as good an approximation as possible. However, note that when solving for the inverse, the matrix–matrix multiplication $\mathbf{G} \cdot (\mathbf{A}^{-1})^{(i)}$ needs $\mathcal{O}(N^3)$ operations so this method will be $\mathcal{O}(N^3 \times N_{\text{iter}})$ and so generally slower than some direct methods.

A second advantage of iterative methods is that in practice, they often yield more accurate solutions than direct methods because they are much less susceptible to round-off error. This is especially true the larger the size of the matrix.

6.4.2 Stopping condition

We must stop iterating at some point according to some **stopping condition**. This can be because the solution has converged to a sufficiently accurate value, or has failed to converge.

Typically, checks for convergence use one of the generalised "norms" of a vector to decide when to stop. There are many measures of the norm of a vector; the general ℓ-norm is

$$\|\vec{x}\|_\ell \equiv \left(\sum_i |x_i|^\ell \right)^{\frac{1}{\ell}} . \tag{6.48}$$

Commonly the $\ell = 1$ (sum of $|x_i|$), the $\ell = 2$ (the familiar Euclidean sum of squares) or the $\ell = \infty$ (the largest component of the vector) are used.

Using one of these norms, a common stopping condition is that the fractional change of the norms of the solution vectors

$$\varepsilon^{(i)} = \left| \frac{\|\vec{x}^{(i+1)}\| - \|\vec{x}^{(i)}\|}{\|\vec{x}^{(i)}\|} \right| \qquad \text{or} \qquad \frac{\|\vec{x}^{(i+1)} - \vec{x}^{(i)}\|}{\|\vec{x}^{(i)}\|} \tag{6.49}$$

is below a specified tolerance; typically something a few orders of magnitude above the fractional round-off error due to finite precision arithmetic (e.g. $\varepsilon_{\text{tol}} \sim 100 \times 10^{-16} \sim 10^{-14}$ for 64-bit precision). Note, do not confuse ε with the error ϵ discussed previously.

Another way to check for convergence is to monitor the **residual** which is $\vec{r} = \mathbf{A} \cdot \vec{x}^{(i+1)} - \vec{b}$. This would be zero for a perfect solution and otherwise gives us $\mathbf{A} \cdot \vec{\epsilon}^{(i+1)}$, i.e. the matrix acting on the error vector. One would then monitor $\varepsilon = \|\vec{r}\|/\|\vec{b}\|$. This is more costly to do every iteration.

6.4.3 Convergence criteria

We need to know if the iteration will converge at all, or whether the error amplification causes at least one of the errors to grow with the iteration steps and hence never converge.

As stated previously for a single variable, the iteration converges only if $|g| < 1$. For the matrix case, in general the iteration will converge if all the eigenvalues of the update matrix $\mathbf{G}$ satisfy $|\lambda_i| < 1$. [1] We now need to introduce some new terminology: the **spectral radius** of $\mathbf{G}$. The spectral radius ρ of any matrix $\mathbf{M}$ is defined as the largest modulus of any of its eigenvalues, i.e. $\rho(\mathbf{M}) = \max\{|\lambda_i|\}$. Hence for convergence, the spectral radius of $\mathbf{G}$ must be less than unity.

[1] Note that a real-valued matrix can have complex eigenvalues.

The condition $|\lambda_i| < 1$ is simple to understand if $\mathbf{G}$ is a diagonalisable matrix. It can then be factorised using the eigen-decomposition, resulting in the diagonal containing the eigenvalues, i.e.

$$\mathbf{G} = \mathbf{R} \cdot \widetilde{\mathbf{G}} \cdot \mathbf{R}^{-1} \quad \text{with} \quad \widetilde{\mathbf{G}} \equiv \text{diag}(\lambda_0, \lambda_1, ..., \lambda_{N-1}), \tag{6.50}$$

We can then map to a 'rotated' error vector $\vec{\zeta}^{(i)} = \mathbf{R}^{-1} \cdot \vec{\epsilon}^{(i)}$, such that

$$\vec{\zeta}^{(i+1)} = -\widetilde{\mathbf{G}} \cdot \vec{\zeta}^{(i)}, \tag{6.51}$$

and therefore the components of ζ are uncoupled. This allows us to impose a criterion for convergence on each eigenvalue individually since

$$\left| \frac{\zeta_i^{(i+1)}}{\zeta_i^{(i)}} \right| \equiv |\lambda_i| < 1. \tag{6.52}$$

(Note that eigen-decomposition is just used here to prove the criteria for the iteration to converge; we don't actually have to explicitly eigen-decompose $\mathbf{G}$ to implement the methods.) As long as all the eigenvalues satisfy $|\lambda_i| < 1$, each iteration 'erodes' away the error vector present in our initial approximate solution $\vec{x}^{(0)}$. The slowest possible rate of erosion, and therefore the slowest rate of convergence to the solution of $\mathbf{A} \cdot \vec{x} = \vec{b}$, is governed by the spectral radius of the update matrix.

6.5 Iterative Methods: specific methods

The critical thing to note is that when breaking $\mathbf{A}$ into $\mathbf{B}$ and $\mathbf{S}$, we must pick a matrix $\mathbf{B}$ we can invert, as $\mathbf{B}^{-1}$ is used in the iterative solution. Clearly $\mathbf{B}$ is the same size as $\mathbf{A}$ so we have to assume it is too large to invert accurately by direct methods, or else we would being applying those methods to $\mathbf{A}$ and not iterating. Hence, we have to pick $\mathbf{B}$ to be one of the special cases described previously, i.e. a diagonal, lower diagonal or upper diagonal matrix. It is convenient to consider $\mathbf{A}$ as the sum of three matrices; one containing its diagonal elements $\mathbf{D}$, one containing the elements in the triangle below the diagonal $\mathbf{T}_L$ and the third containing the elements in the triangle above the diagonal $\mathbf{T}_U$. Hence

$$\boxed{\mathbf{A} = \mathbf{D} + \mathbf{T}_L + \mathbf{T}_U.} \tag{6.53}$$

This will be something like

$$\begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} A_{00} & 0 & 0 \\ 0 & A_{11} & 0 \\ 0 & 0 & A_{22} \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 \\ A_{10} & 0 & 0 \\ A_{20} & A_{21} & 0 \end{pmatrix} + \begin{pmatrix} 0 & A_{01} & A_{02} \\ 0 & 0 & A_{12} \\ 0 & 0 & 0 \end{pmatrix} \tag{6.54}$$

We then have

$$\mathbf{A} \cdot \vec{x} = (\mathbf{D} + \mathbf{T}_L + \mathbf{T}_U) \cdot \vec{x} = \vec{b}, \tag{6.55}$$

Note that the $\mathbf{T}_L$ and $\mathbf{T}_U$ matrices here are nothing to do with those from LU decomposition. (For a start, we are adding rather than multiplying to compose $\mathbf{A}$. Also the LU decomposition matrices have non-zero diagonal elements as well.)

The various specific iterative methods described below only differ in terms of the choice of how the approximation matrix $\mathbf{B}$ is formed from combinations of $\mathbf{D}$, $\mathbf{T}_L$ and $\mathbf{T}_U$.

6.5.1 Jacobi Method

The Jacobi method corresponds to taking $\mathbf{B} = \mathbf{D}$ as the approximation to $\mathbf{A}$. Finding $\mathbf{B}^{-1}$ is then trivial, as stated previously, as long as none of the diagonal elements are zero. This choice means $\mathbf{S} = \mathbf{T}_L + \mathbf{T}_U$ and so $\mathbf{G} = \mathbf{B}^{-1} \cdot \mathbf{S} = \mathbf{D}^{-1} \cdot (\mathbf{T}_L + \mathbf{T}_U)$. Hence Eq. (6.43), the general iterative case for $\vec{x}$, becomes

$$\boxed{\vec{x}^{(i+1)} = \mathbf{D}^{-1} \cdot \vec{b} - \mathbf{D}^{-1} \cdot (\mathbf{T}_L + \mathbf{T}_U) \cdot \vec{x}^{(i)}} \tag{6.56}$$

which we can use to find $\vec{x}$ starting from a first approximation $\vec{x}^{(0)} = \mathbf{D}^{-1} \cdot \vec{b}$. Eq. (6.56) is the Jacobi method. The condition $\rho(\mathbf{G}) < 1$ is *guaranteed* for this method for any matrix $\mathbf{A}$ that is strictly **diagonally dominant** [2], which means

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}| \tag{6.57}$$

for all rows i of the matrix. The Jacobi method can sometimes converge even for a matrix which is *not* diagonally dominant, if the eigenvalues of its associated update matrix are suitable. [3] In particular, **sparse systems**, where only a few variables appear in each equation, can often be rearranged using row manipulations (i.e. pivoting) into a diagonally dominant form such as a band diagonal matrix which looks like

$$\begin{pmatrix} \cdot & \cdot & & & & & \\ \cdot & \cdot & \cdot & & & & \\ & \cdot & \cdot & \cdot & & 0 & \\ & & \cdot & \cdot & \cdot & & \\ & 0 & & \cdot & \cdot & \cdot & \\ & & & & \cdot & \cdot & \cdot \\ & & & & & \cdot & \cdot \end{pmatrix} . \tag{6.58}$$

6.5.2 Gauss-Seidel Method

Gauss-Seidel (GS) is an alternative method which often converges faster than the Jacobi method. The approximation to $\mathbf{A}$ is $\mathbf{B} = \mathbf{D} + \mathbf{T}_L$, which leaves only $\mathbf{S} = \mathbf{T}_U$ as the correction and the update matrix is now $\mathbf{G} = (\mathbf{D} + \mathbf{T}_L)^{-1} \cdot \mathbf{T}_U$. This gives

$$\boxed{\vec{x}^{(i+1)} = (\mathbf{D} + \mathbf{T}_L)^{-1} \cdot \vec{b} - (\mathbf{D} + \mathbf{T}_L)^{-1} \cdot \mathbf{T}_U \cdot \vec{x}^{(i)}} \tag{6.59}$$

The sum $\mathbf{D} + \mathbf{T}_L$ is a lower diagonal matrix. (Again, note this will not be the *same* lower diagonal matrix as would be found using LU decomposition.) Hence, we can calculate its inverse using the methods described in section (6.3) and so apply the above iteration equation directly. However, finding the inverse even for this special case can take a long time for a large matrix. There is an alternative way to solve for each iteration which can be more computationally efficient. We can rewrite Eq. (6.59) as

$$(\mathbf{D} + \mathbf{T}_L) \cdot \vec{x}^{(i+1)} = \vec{b} - \mathbf{T}_U \cdot \vec{x}^{(i)} \tag{6.60}$$

Thanks to the shape of the matrix $\mathbf{D} + \mathbf{T}_L$, the GS iteration written this way has the same form as $\mathbf{L} \cdot \vec{x} = \vec{b}$ so we can use **forward substitution** to solve for $\vec{x}^{(i+1)}$ in each iteration instead. Both ways of doing the iteration should agree within rounding errors.

The improved convergence speed relative to the Jacobi method stems from the fact that the spectral radius of the GS update matrix is less than that of the Jacobi update matrix for a given $\mathbf{A}$, i.e., $\rho(\mathbf{G}_{GS}) < \rho(\mathbf{G}_J)$. Intuitively, GS performs better than the Jacobi method because it includes "more of $\mathbf{A}$" in the $\mathbf{B}$ matrix and so should be a better approximation. More quantitatively, consider the GS iteration equation in a different form. Writing

$$\mathbf{D} \cdot \vec{x}^{(i+1)} = \vec{b} - \mathbf{T}_L \cdot \vec{x}^{(i+1)} - \mathbf{T}_U \cdot \vec{x}^{(i)} \tag{6.61}$$

then we have

$$\vec{x}^{(i+1)} = \mathbf{D}^{-1} \cdot \vec{b} - \mathbf{D}^{-1} \cdot (\mathbf{T}_L \cdot \vec{x}^{(i+1)} + \mathbf{T}_U \cdot \vec{x}^{(i)}) \tag{6.62}$$

[2] If $\mathbf{A}$ is not diagonally dominant then the spectral radius of its Jacobi update matrix will need to be checked, to determine whether or not the Jacobi method will converge with $\mathbf{A}$.

[3] In particular, matrices with $|A_{ii}| \geq \sum_{j \neq i} |A_{ij}|$, i.e., some (but not all) rows not *strictly* diagonally dominant, will often work with the Jacobi method.

Comparing this to the Jacobi iteration Eq. (6.56), it can be seen that we are using some of the updated (improved) values from $\vec{x}^{(i+1)}$ in the RHS, rather than only $\vec{x}^{(i)}$, and so would expect a more accurate result.

Finally, note that the choice of $\mathbf{B} = \mathbf{D} + \mathbf{T}_U$, which means $\mathbf{S} = \mathbf{T}_L$, would be an equivalently good method and would be solved by backward substitution. The choice of $\mathbf{S} = \mathbf{T}_U$ or $\mathbf{T}_L$ should be governed by which would converge quicker, e.g. if one has smaller values in the elements than the other.

6.5.3 Successive Over-Relaxation Method

Abbreviated to SOR, this can be thought of as a modified version of Gauss-Seidel with superior speed of convergence. Introducing a parameter α, we write

$$\mathbf{A} = \alpha\mathbf{D} + \mathbf{T}_L + \mathbf{T}_U + (1-\alpha)\mathbf{D} \tag{6.63}$$

and then choose $\mathbf{B} = \alpha\mathbf{D} + \mathbf{T}_L$ so $\mathbf{S} = \mathbf{T}_U + (1-\alpha)\mathbf{D}$. Compared to GS, which corresponds to $\alpha = 1$, other values of α move part of $\mathbf{D}$ into the correction matrix. However, $\mathbf{B}$ is still a lower diagonal matrix and so can be inverted directly, or the equations can be solved using forward substitution, just as for GS. The direct equation is

$$\boxed{\vec{x}^{(i+1)} = (\alpha\mathbf{D} + \mathbf{T}_L)^{-1} \cdot \vec{b} - (\alpha\mathbf{D} + \mathbf{T}_L)^{-1} \cdot [\mathbf{T}_U + (1-\alpha)\mathbf{D}] \cdot \vec{x}^{(i)}} \tag{6.64}$$

and the forward substitution equation is

$$(\alpha\mathbf{D} + \mathbf{T}_L) \cdot \vec{x}^{(i+1)} = \vec{b} - [\mathbf{T}_U + (1-\alpha)\mathbf{D}] \cdot \vec{x}^{(i)} \tag{6.65}$$

Standardly, the parameter is actually defined as $\omega = 1/\alpha$ and is called the **relaxation parameter**. A high speed of convergence can be achieved by tuning it, which affects the spectral radius of SOR's update matrix. With $1 < \omega \leq 2$, SOR gives faster convergence than GS. However, the optimum value of ω is problem-specific. In simple cases it can be determined analytically (not covered in this course), otherwise it must be found by trial and error. For $\omega > 2$, SOR fails, while $\omega < 1$ corresponds to under-relaxation, which is not beneficial to speed of convergence.

6.6 Eigenvalues of a Matrix

We have seen how finding the largest eigenvalue of a matrix would be useful in checking if a method will converge. There are many other applications in physics where we need to find eigenvalues.

6.6.1 Largest eigenvalue

The **method of powers** is a simple method to approximate the eigenvalue with the largest modulus of any non-singular $N \times N$ matrix with N linearly independent eigenvectors. It also yields the associated eigenvector. Consider a system of the type

$$\mathbf{A} \cdot \vec{x} = \lambda\, \vec{x}, \tag{6.66}$$

with $\mathbf{A}$ an $N \times N$ matrix. In general if $\mathbf{A}$ is non-singular (i.e. $\det \mathbf{A} \neq 0$) it will have N distinct eigenvalues λ_i with associated linearly independent eigenvectors $(\vec{e}_i)$

$$\mathbf{A} \cdot \vec{e}_i = \lambda_i\, \vec{e}_i\,. \tag{6.67}$$

The linearly independent eigenvectors form a basis in which any vector (say $\vec{v}$) can be expanded

$$\vec{v} = \sum_{i=1}^{N} c_i\, \vec{e}_i\,. \tag{6.68}$$

We start with a random choice of vector $\vec{v}$ and act on it with $\mathbf{A}$ a number of times

$$\begin{aligned} \mathbf{A}\cdot\vec{v} &= \sum_{i=1}^{N} c_i\,\mathbf{A}\cdot\vec{e}_i = \sum_{i=1}^{N} c_i\,\lambda_i\,\vec{e}_i, \\ \mathbf{A}\cdot\mathbf{A}\cdot\vec{v} &= \sum_{i=1}^{N} c_i\,\lambda_i^2\,\vec{e}_i, \\ \mathbf{A}^n\cdot\vec{v} &= \sum_{i=1}^{N} c_i\,\lambda_i^n\,\vec{e}_i, \\ \therefore \quad \lim_{n\to\infty} \mathbf{A}^n\cdot\vec{v} &\to c_j\,\lambda_j^n\,\vec{e}_j, \end{aligned} \tag{6.69}$$

where λ_j is the eigenvalue with the largest modulus. Since any multiple of an eigenvector is still an eigenvector itself, we have found the eigenvector with the largest eigenvalue. We can normalise the vector we have just found as

$$\boxed{\vec{\omega}_j = \frac{\mathbf{A}^n\cdot\vec{v}}{|\mathbf{A}^n\cdot\vec{v}|} \equiv \frac{\vec{e}_j}{|\vec{e}_j|},} \tag{6.70}$$

such that $\vec{\omega}_j^{\,T}\cdot\vec{\omega}_j = 1$. (Here $\vec{\omega}_j^{\,T}$ is the transpose of vector $\vec{\omega}_j$.) Then it is simple to calculate the eigenvalue itself since

$$\boxed{\vec{\omega}_j^{\,T}\cdot\mathbf{A}\cdot\vec{\omega}_j = \vec{\omega}_j^{\,T}\cdot(\lambda_j\vec{\omega}_j) = \lambda_j.} \tag{6.71}$$

6.6.2 Smallest eigenvalue

We can also use the method of powers to find the eigenvalue of a matrix with the smallest modulus by using $\mathbf{A}^{-1}$ instead of $\mathbf{A}$ in equations Eq. (6.70) and Eq. (6.71). This works because the inverse of a matrix has the same eigenvectors but with the inverse eigenvalues. This can be shown since

$$\vec{e}_i = \mathbf{A}^{-1}\cdot\mathbf{A}\cdot\vec{e}_i = \mathbf{A}^{-1}\cdot(\lambda_i\,\vec{e}_i) = \lambda_i\,\mathbf{A}^{-1}\cdot\vec{e}_i \tag{6.72}$$

thus

$$\mathbf{A}^{-1}\cdot\vec{e}_i = \left(\frac{1}{\lambda_i}\right)\vec{e}_i. \tag{6.73}$$

So using the power method on $\mathbf{A}^{-1}$ finds its largest eigenvalue which will be the smallest eigenvalue of the original $\mathbf{A}$. We find $\mathbf{A}^{-1}$ using one of the methods described earlier in this section.

6.6.3 Other Eigenvalues

There are many methods for finding the remaining eigenvalues of a matrix but most use the **shift method** by finding the eigenvalue closest to a given value α. To see this define the shifted matrix

$$\mathbf{A}' \equiv \mathbf{A} - \alpha\,\mathbf{I}, \tag{6.74}$$

such that

$$\mathbf{A}'\cdot\vec{e}_i = \mathbf{A}\cdot\vec{e}_i - \alpha\,\mathbf{I}\cdot\vec{e}_i = \lambda_i\,\vec{e}_i - \alpha\,\vec{e}_i = (\lambda_i-\alpha)\,\vec{e}_i\,. \tag{6.75}$$

So the shifted matrix has the same eigenvectors but shifted eigenvalues, $\lambda_i' = \lambda_i - \alpha$. Thus finding the largest eigenvalue of $(\mathbf{A}')^{-1}$ (e.g. by the power method) will give the the smallest of $\mathbf{A}'$ and thus the eigenvalue of $\mathbf{A}$ closest to the value α. Again, we obtain $(\mathbf{A}')^{-1}$ using the methods already described.

> We need to choose a suitable value for α. It turns out that the values of the diagonal elements are good choices, particularly if $\mathbf{A}$ is diagonally dominant. This follows from **Gerschgorin's**

Theorem which states that for any eigenvalue λ_i the following inequality is satisfied

$$|\lambda_i - A_{ii}| \leq \sum_{j \neq i} |A_{ij}|\,, \tag{6.76}$$

i.e. the eigenvalue lies within a circle/disk in the complex plane of radius $\sum_{j\neq i}|A_{ij}|$ centred on the value of the diagonal element A_{ii}. Equivalently, each 'Gerschgorin disc' will have an eigenvalue in it (and possibly more than one if discs overlap and if eigenvalues happen to fall on such overlaps).

This is particularly useful if $\mathbf{A}$ is diagonally dominant since the radius will be small and therefore the values of the diagonal elements will be close to the eigenvalues, allowing the algorithm to work quickly and efficiently. For example take the following matrix

$$\mathbf{A} \equiv \begin{pmatrix} 1 & 0.1 & 0 \\ 0.1 & 5 & 0.2 \\ 0.1 & 0.3 & 10 \end{pmatrix}. \tag{6.77}$$

We then have that $0.9 \leq \lambda_0 \leq 1.1$, $4.7 \leq \lambda_1 \leq 5.3$, or $9.6 \leq \lambda_2 \leq 10.4$. We can then find the exact values by setting α to 1, 5, or 10 in the shift method.

Note, we only have to consider the rows, not the columns as well. This is basically because the rows multiply the eigenvectors when finding the eigenvalues. The eigenvalues and eigenvectors of the transposed matrix (when the columns would be relevant) are not the same.

Finally, it is worth noting that Gerschgorin's Theorem provides a convenient way of assessing upper bounds for the spectral radius of the update matrices of the iterative solvers seen earlier in section (6.5) and thus determining the suitability of a matrix $\mathbf{A}$ for solution by, e.g. Jacobi iteration.

6.7 Non-Linear Algebraic Equations of one variable

An important part of the computational physics 'toolkit' is to find roots of non-linear algebraic equations, i.e. the solution of

$$f(x) = 0, \tag{6.78}$$

where f involves transcendental equations, e.g. $\sin(x^2) - 1/(x+a) = 0$, or is even simply a polynomial of high order and therefore a solution in closed form is not possible, This is just the sort of situation where numerical methods are essential. Non-linear root finders are typically **iterative methods**, where an initial guess is refined iteratively. Functions with discontinuities and/or infinities pose problems unless the initial guess is carefully made. Some key methods are given below.

6.7.1 Bisection method

This is the simplest method and is very robust, but slow. One starts with two points x_l (the left point) and x_r (right point) which "bracket the root", meaning $f(x_l)$ and $f(x_r)$ must have opposite signs. Then $f(x)$ must pass through zero (perhaps several times) between these points. Now calculate the mid point and the distance between the points

$$x^{(0)} = \frac{x_l + x_r}{2} \qquad \text{and} \qquad \epsilon^{(0)} = x_r - x_l \tag{6.79}$$

where the superscript denotes the iteration number. This gives us an estimate of the solution and an approximation to its error. To iterate on this, check the sign of $x^{(0)}$ to determine whether the pair x_l and $x^{(0)}$ or the pair $x^{(0)}$ and x_r now bracket the root. Update x_l or x_r using the value of $x^{(0)}$ accordingly and calculate the next iteration

$$x^{(1)} = \frac{x_l + x_r}{2} \tag{6.80}$$

Note, $\epsilon^{(1)} = \epsilon^{(0)}/2$ by construction. These steps can be repeated until $\epsilon^{(i)}$ is sufficiently small or $|f(x^{(i)})|$ is sufficiently close to zero. (These are typical stopping conditions for this method.) This method converges linearly, with the error $\epsilon^{(i)}$ halving with each iteration

$$\epsilon^{(i+1)} = \frac{\epsilon^{(i)}}{2}. \tag{6.81}$$

Beware: the bisection method will also converge to points where discontinuous functions jump through zero, e.g. $\tan(x)$ at $x = \pi/2$.

6.7.2 Newton-Raphson method

This is also known as Newton's method. This trivially follows from applying the first order Taylor expansion at x to estimate the change Δx in x required to get to the zero of the function

$$f(x + \Delta x) = 0 \approx f(x) + f'(x)\Delta x \qquad \text{so} \qquad f'(x)\Delta x \approx -f(x) \tag{6.82}$$

Rearranging this gives an expression in terms of the function and its first derivative for the current iterative estimate $x^{(i)}$ to find the next iterative estimate $x^{(i+1)} = x^{(i)} + \Delta x^{(i)}$

$$\boxed{x^{(i+1)} = x^{(i)} - \frac{f(x^{(i)})}{f'(x^{(i)})}.} \tag{6.83}$$

The Newton-Raphson method can easily fail if it gets near maxima or minima, in which case dividing by the derivative $f' \approx 0$ can cause $x^{(i+1)}$ to shoot off 'to infinity', potentially giving an overflow. It is also possible to be locked in a cycle where the method ping-pongs between the same two x points. The advantage of Newton's method is its speed of convergence; it converges quadratically

$$\epsilon^{(i+1)} \propto [\epsilon^{(i)}]^2. \tag{6.84}$$

6.7.3 Secant method

This iterative method is related to the Newton-Raphson method, but is used when an analytical expression for the derivative function $f'(x)$ is unknown. The tangent to the function $f'(x^{(i)})$ is approximated using the last two estimates (which define the "secant line", hence the method name)

$$f'(x^{(i)}) \approx \frac{f(x^{(i)}) - f(x^{(i-1)})}{x^{(i)} - x^{(i-1)}}. \tag{6.85}$$

and so needs two points to start. Inserting into the Newton-Raphson formula gives

$$\boxed{x^{(i+1)} = x^{(i)} - f(x^{(i)})\ \frac{x^{(i)} - x^{(i-1)}}{f(x^{(i)}) - f(x^{(i-1)})}} \tag{6.86}$$

The speed of convergence of the secant method is somewhere between linear and quadratic.

6.7.4 Brent's method

This is the Rolls Royce of 1D root-finding. It combines bisection with inverse quadratic interpolation *and* the secant method, plus careful bracketing and error tracking—to give what is essentially the last algorithm you will ever need for solving equations in one variable. It has the downside of being quite fiddly to code, as it flips back and forth between the bisection, inverse quadratic and secant methods according to a series of different criteria, depending on how it is tracking at the time. You can read a good account of it, and see some example code, in Numerical Recipes.

6.8 Non-Linear Algebraic Equations of multiple variables

As stated previously, to solve for N unknowns $\vec{x}$, we need to consider N independent equations. This requires N functions $f_i(\vec{x}) = 0$, sometimes written as $\vec{f}(\vec{x}) = 0$.

The basic method to use is the generalisation of the Newton-Raphson method to N variables. The Taylor series to first order becomes

$$f_i(\vec{x} + \Delta\vec{x}) = 0 \approx f_i(\vec{x}) + \sum_{j=0}^{N-1} \Delta x_j \frac{\partial f_i}{\partial x_j} \,. \tag{6.87}$$

The matrix of the $N \times N$ partial derivatives is the Jacobian matrix; writing this as

$$\mathbf{J} = \begin{pmatrix} \partial f_0/\partial x_0 & \partial f_0/\partial x_1 & \partial f_0/\partial x_2 & \dots \\ \partial f_1/\partial x_0 & \partial f_1/\partial x_1 & \partial f_1/\partial x_2 & \dots \\ \partial f_2/\partial x_0 & \partial f_2/\partial x_1 & \partial f_2/\partial x_2 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{6.88}$$

gives

$$\sum_{j=0}^{N-1} J_{ij} \Delta x_j = -f_i \qquad \text{i.e.} \qquad \mathbf{J} \cdot \Delta\vec{x} = -\vec{f} \tag{6.89}$$

and is seen to be a linear matrix equation. This is obviously the generalisation of the right hand expression in Eq. (6.82). The iteration equation is formally

$$\boxed{\vec{x}^{(i+1)} = \vec{x}^{(i)} - \mathbf{J}^{-1} \cdot \vec{f}} \tag{6.90}$$

This means each step of the generalised Newton-Raphson method requires solving a matrix equation, using any of the methods discussed earlier in this chapter which are appropriate for the size of the matrices being handled in a particular problem. Note that for a linear function f, the Jacobian is the constant matrix which we called $\mathbf{A}$ previously.

The secant method cannot be easily generalised. To estimate the $N \times N$ derivatives needed for the Jacobian matrix requires more than just the current and previous iterations. Hence, if the derivatives cannot be calculated analytically, then each derivative must be estimated specifically for each iteration using a small step h applied to each component of $\vec{x}$ in turn. For example, one partial derivative can be estimated using

$$\frac{\partial f_0}{\partial x_1} \approx \frac{f_0(x_0, x_1 + h, x_2, \dots) - f_0(x_0, x_1, x_2, \dots)}{h} \,. \tag{6.91}$$

(This is known as a *forward difference* approximation and such derivative estimates are discussed in section (5.1).) The above estimate must then be repeated for all remaining combinations of the N variables and functions, meaning $N^2 + N$ function evaluations are needed. This makes the method slower than the Newton-Raphson generalisation although the overall speed of the method is often dominated by the matrix equation solver, which is common to both methods.

Chapter 8

Minimisation and Maximisation of Functions

Outline of Section

- One–dimensional minimisation
- Multi-dimensional minimisation
- Monte-Carlo minimisation
- Using minimisation to solve matrix equations

We first focus on **iterative methods** for finding local or global minima (maxima) of a function of one variable $f(x)$ or of many independent variables $f(x_1, x_2, \ldots, x_N)$, written more succinctly as $f(\vec{x})$ where $\vec{x}$ is an N-dimensional vector. We also discuss Monte Carlo methods for function minimisation, borrowing the principles introduced in the earlier chapter.

The function to be minimised is often called the "cost function" (and not just in economics), in which case the problem can be considered as minimising the "cost" of the system. It can also be called the "loss function". Both the position of a minimum $\vec{x}_*$ and the value of the function there $f(\vec{x}_*)$ may be of interest. The function can be non-linear. The methods covered will find minima; maxima can be found by applying these methods to $F(\vec{x}) = -f(\vec{x})$. Root-finding of non-linear functions $f(x)$ was introduced in section 6.7. In principle roots of a non-linear function involving many variables $f(\vec{x})$ can be found by finding minima $\vec{x}_*$ of $|f(\vec{x})|^2$ where $f(\vec{x}_*) = 0$. However, root-finding is a significant area of study in its own right and this is not a generally-appropriate method.

Interestingly, it turns out that solving a matrix equation can be formulated as a minimisation problem, as we shall see later. Some of the most efficient iterative matrix solvers are based on this approach and converge much faster than Jacobi, Gauss-Seidel and SOR.

We will deal with **unconstrained** minimisation here. ***Constrained*** minimisation—where further constraints are placed on the independent variables or the values of the cost function—are not addressed here. Many of the concepts introduced here are, however, applicable in extended form when performing constrained minimisation.

We will also limit ourselves to real valued, scalar functions $f(\vec{x}) \in \mathbb{R}$ and real valued variables; $x \in \mathbb{R}$ and $\vec{x} \in \mathbb{R}^N$.

One setting where finding the minimum of a complicated non-linear function occurs is the optimisation of parameters in a model when fitting to data. Consider the observed data d_i^{obs} with errors σ_i in figure 8.1. The model we would like to fit is a linear one

$$d_i^{\text{model}} = a + \alpha\, X_i\,. \tag{8.1}$$

The normal procedure is to find the minimum χ^2 with respect to the parameters a and α

$$\chi^2(a,\alpha) = \sum_{i=1}^{N^{\text{data}}} \frac{\left(d_i^{\text{obs}} - d_i^{\text{model}}\right)^2}{\sigma_i^2}\,. \tag{8.2}$$

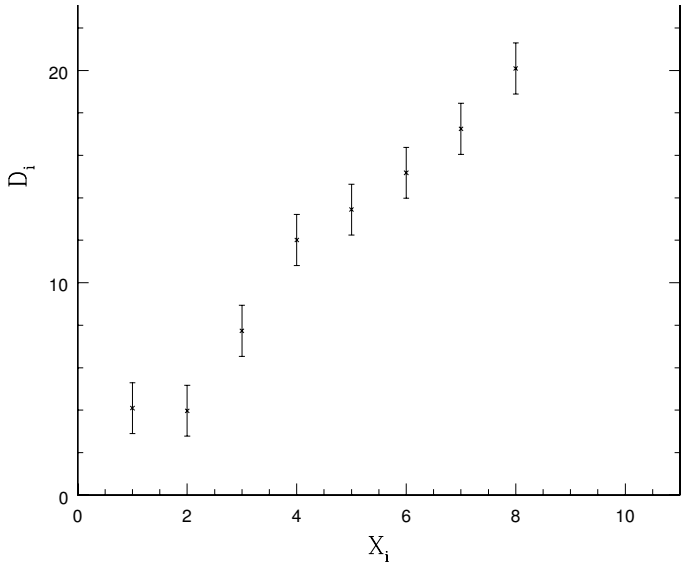

Figure 8.1: Some data with errors.

The assumption here is that the data are distributed as independent Gaussian random numbers and we are **maximising the likelihood**

$$L(a, \alpha) \propto e^{-\frac{1}{2}\chi^2(a,\alpha)}, \tag{8.3}$$

which is equivalent to finding the minimum in the χ^2. The result of the minimisation may look something like figure 8.2. In practice, the data that need to be fitted to and the models that are used are significantly more compliated than this example; cases with hundreds of free parameters (equivalently, degrees of freedom or dimensions) and numerous types of experimental data are common.

8.1 One–dimensional minimisation

Naively we might think that finding the minima of a function is trivial. All we need to do is differentiate the function and find the roots of the resulting equation. However there are many cases where the analytical method is not applicable, e.g., where the derivative in the function is discontinuous or the function has a regular (possibly infinite) set of minima, when all we are interested in is the single global minimum—which we cannot find easily using this method. For a function of many, many variables, it can be too computationally costly to evaluate $\partial f/\partial x_i$ for all variables or simply too laborious to implement them all in code!

In practice this is often the case when evaluating a function of some scattered data where the analytical dependence of the likelihood with respect to the parameters is not known *a priori* and must be evaluated numerically. In this case we have to choose between a method that searches for minima/maxima using just the function values or (more optimal) ones that also use approximations for the derivatives of the function.

Parabolic Method

Functions almost always approximate a parabola near a minimum (except for some pathological cases). A very simple method to find a local minimum of a function exploits this. The method works when the curvature of $f(x)$ is positive everywhere in the interval we are looking at. The

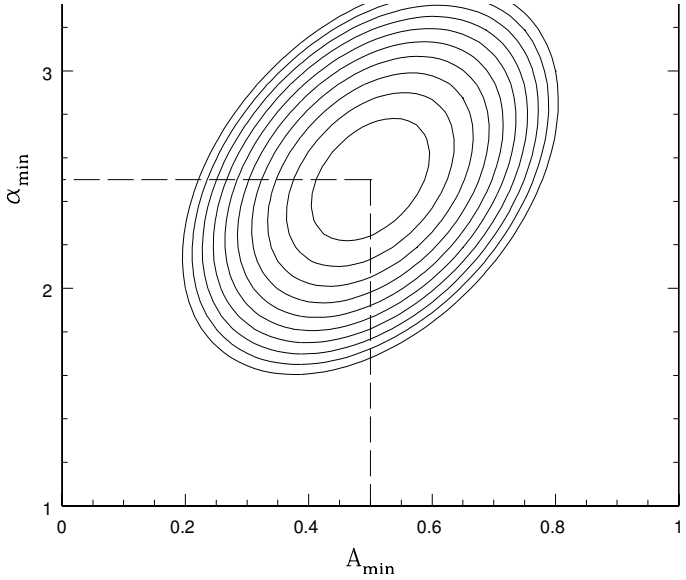

Figure 8.2: Contour plot of the χ^2 around the minimum. $a_{\rm min}$ and $\alpha_{\rm min}$ are the maximum likelihood values for the parameters in the model. The curvature around the minimum is related to the error in the parameters.

parabolic method consists of selecting three points along the function $f(x)$, e.g., x_0, x_1, and x_2 where

$$f(x_0) = y_0, \quad f(x_1) = y_1, \quad \text{and} \quad f(x_2) = y_2 \tag{8.4}$$

and then fitting $P_2(x)$, the 2nd-order Lagrange polynomial, through the points (see section 4.1). The location of the minimum of the interpolating parabola $P_2(x)$ is found using equation (8.6) below; this value is x_3. This procedure is illustrated in fig. 8.3. We then keep the three lowest points out of $f(x_0)$, $f(x_1)$, $f(x_2)$, and $f(x_3)$ and repeat the procedure. At each successive iteration the point x_3 will converge towards x_* where the minimum of the function $f(x)$ is located. The iterations can be stopped once the change in x_3 is less than a desired value.

The quadratic interpolating the 3 points is given by

$$P_2(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}y_0 + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}y_1 + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}y_2, \tag{8.5}$$

[c.f. equation (4.3)]. We want to find the minimum of $P_2(x)$, which will be a better estimate of the minimum's position (than the middle point of our trio). Differentiating $P_2(x)$ gives

$$\frac{{\rm d}P_2}{{\rm d}x} = \frac{[(x-x_1)+(x-x_2)]\,(x_2-x_1)}{d}y_0 + \frac{[(x-x_0)+(x-x_2)]\,(x_0-x_2)}{d}y_1 + \frac{[(x-x_0)+(x-x_1)]\,(x_1-x_0)}{d}y_2,$$

where $d = (x_1-x_0)(x_2-x_0)(x_2-x_1)$. Then setting $dP_2/dx = 0$ and solving for x gives (after some algebra) a new estimate of the minimum x_3

$$\boxed{x_3 = \frac{1}{2}\frac{(x_2^2-x_1^2)y_0 + (x_0^2-x_2^2)y_1 + (x_1^2-x_0^2)y_2}{(x_2-x_1)y_0 + (x_0-x_2)y_1 + (x_1-x_0)y_2}.} \tag{8.6}$$

This method works because any minimum can be approximated by a parabola and the approximation becomes more and more accurate as you get closer to the minimum.

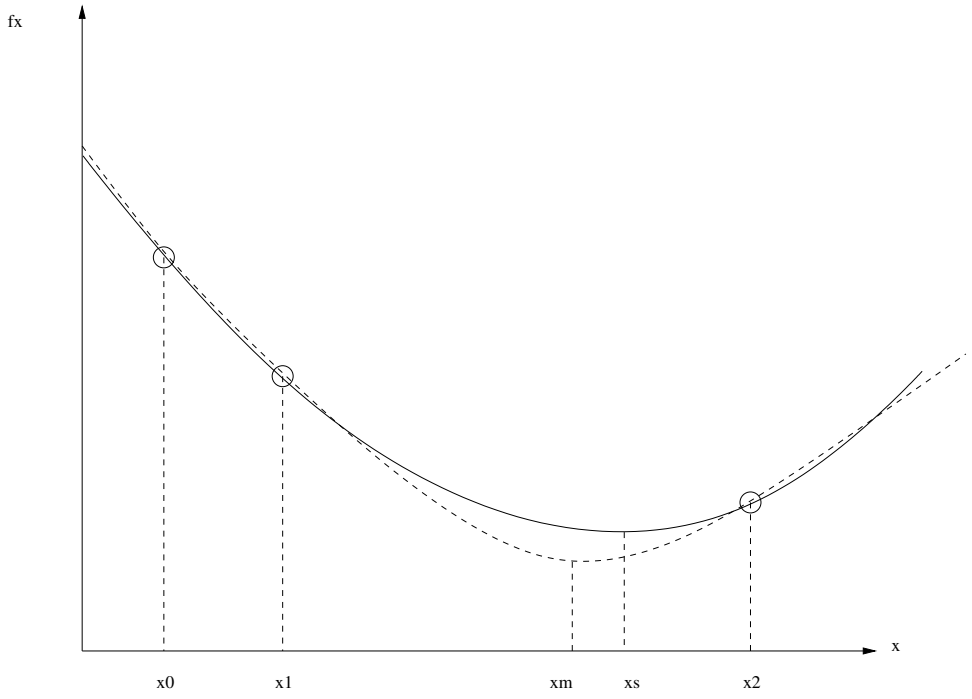

Figure 8.3: **The parabolic minimum search.** The function being minimised $f(x)$ is the dashed curve. A parabola (solid curve) is fit to the three points at x_0, x_1, x_2 and the minimum $x = x_3$ is found. Then the highest of the four points x_0, x_1, x_2, and x_3 is discarded and the method repeated. The minima of successive parabola approximations converge to the minimum of the function $(x_*, f(x_*))$.

For the case where the curvature is not positive throughout the initial interval we can first evaluate the function at two points inside the interval. We then keep the interval which includes the lowest point and then use the parabolic method to find the minimum within the new interval.

8.2 Multi-Dimensional methods

Univariate method

For the case where the function is multi-dimensional, i.e., $f(\vec{x})$, we can extend the one-dimensional parabolic method. This can be done by searching for the minimum in each direction successively and iterating. However this is a very inefficient method which is not useful in most cases.

Univariate search is illustrated by the black arrows in figure 8.4. Contours are shown for a 2D function $f(x, y)$. The univariate search spirals into the minimum, converging slowly.

Gradient method

We can follow the steepest descent towards the minimum. This is achieved by calculating the local gradient and following its (negative) direction. The gradient of $f(\vec{x})$ at point $\vec{x}_n$ is given by the vector

$$\vec{d}(\vec{x}_n) = \vec{\nabla} f(\vec{x}_n) \qquad \text{where} \qquad \vec{\nabla} f = \begin{pmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \vdots \\ \partial f/\partial x_N \end{pmatrix} \tag{8.7}$$

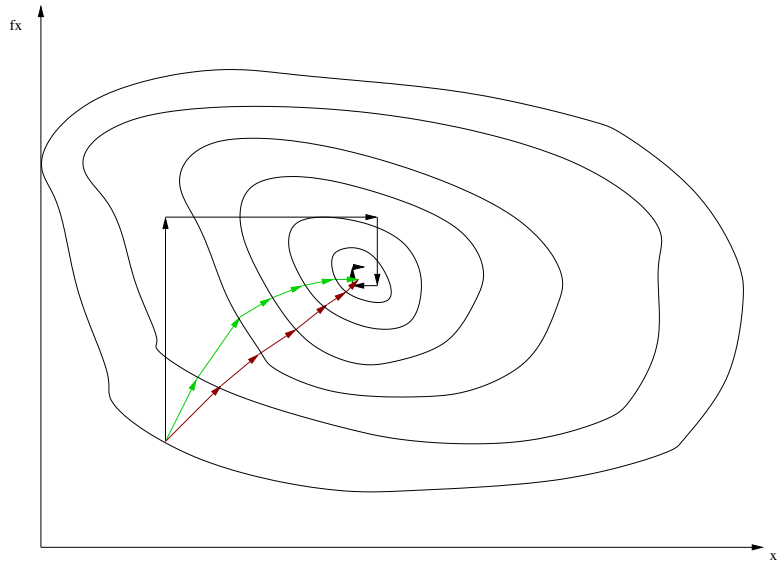

Figure 8.4: **Multi-dimensional minimisation** - for a function $f(x, y)$. Three methods are depicted: univariate method (black arrows), gradient method (green arrows) and Newton's method (red arrows). [Nb: this is not quite accurate; the green arrows are all supposed to be perpendicular to the contours!]

and is perpendicular to the local contour line. Note that the notation $\vec{\nabla} f(\vec{x}_0)$ is equivalent to $\vec{\nabla} f|_{\vec{x}_0}$, i.e., $\vec{\nabla} f$ evaluated at $\vec{x} = \vec{x}_0$. We can take a small step in the direction **opposite** to the gradient, i.e.,

$$\boxed{\vec{x}_{n+1} = \vec{x}_n - \alpha\, \vec{d}(\vec{x}_n),} \tag{8.8}$$

where $\alpha \ll 1$ and positive. If α is small enough then

$$f(\vec{x}_{n+1}) < f(\vec{x}_n). \tag{8.9}$$

We can iterate this to find the minimum. (See green arrows in fig. 8.4.)

The gradient $\vec{\nabla} f$ can be found analytically or using a finite difference approximation; e.g. via a forward-difference scheme applied in each variable x_i for $i = 1, \ldots, N$,

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x_1, x_2, \ldots, x_i + \Delta, \ldots, x_N) - f(x_1, x_2, \ldots, x_i, \ldots, x_N)}{\Delta}. \tag{8.10}$$

Newton's method

A more efficient method to find the minimum is achieved by including the local curvature at each step. Consider starting at a location $\vec{x}_0$. The minimum is displaced $\vec{\delta}$ away, at position $\vec{x}_0 + \vec{\delta}$. How can we estimate $\vec{\delta}$?

We use the Taylor series for the function about $\vec{x}$. (Later, $\vec{x}$ will be set to $\vec{x}_0$.)

$$f(\vec{x} + \vec{\delta}) = f(\vec{x}) + [\vec{\nabla} f(\vec{x})]^T \cdot \vec{\delta} + \frac{1}{2} \vec{\delta}^T \cdot \mathbf{H}(\vec{x}) \cdot \vec{\delta} + \mathcal{O}(|\vec{\delta}|^3), \tag{8.11}$$

where **H** is the **Hessian** or **Curvature** matrix (an $N \times N$ matrix)

$$H_{ij}(\vec{x}) = \frac{\partial^2 f(\vec{x})}{\partial x_i \partial x_j}. \tag{8.12}$$

(c.f. equation 2.6 and the paragraph which precedes it in section 2.2.)

To express this in an alternative format; any function that is the sum of the terms x^2, y^2, $x \times y$, x, y with coefficients and a constant can be written:

$$f(x,y) = \frac{1}{2}\begin{pmatrix} x & y \end{pmatrix} \mathbf{H} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} a & b \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + C, \tag{8.13}$$

which is to say:

$$f(x,y) = \frac{1}{2}\left(\frac{\partial^2 f}{\partial x^2}x^2 + \frac{\partial^2 f}{\partial x \partial y}2xy + \frac{\partial^2 f}{\partial y^2}y^2\right) + ax + by + C. \tag{8.14}$$

Since by definition $\vec{x} + \vec{\delta}$ is the location of the minimum, we must have $\vec{\nabla} f(\vec{x} + \vec{\delta}) = \vec{0}$. To actually be a minimum (rather than a maximum or a saddle point) the Hessian needs to be **positive definite**, i.e.,

$$\vec{\delta}^{\,T} \cdot \mathbf{H} \cdot \vec{\delta} > 0 \qquad \text{(for } \textit{all} \quad \vec{\delta} \neq \vec{0}\text{)}. \tag{8.15}$$

To determine $\vec{\delta}$ we can use the Taylor expansion (8.11) to first order in $\vec{\delta}$ and take its gradient yielding

$$\begin{aligned} \vec{\nabla} f(\vec{x} + \vec{\delta}) &= \vec{\nabla} f(\vec{x}) + \vec{\nabla}\left([\vec{\nabla} f(\vec{x})]^T \cdot \vec{\delta}\right) = \vec{0}, \\ &= \vec{\nabla} f(\vec{x}) + \mathbf{H}(\vec{x}) \cdot \vec{\delta} = \vec{0}. \end{aligned}$$

Since we start at $\vec{x} = \vec{x}_0$ we can solve the following matrix-equation for $\vec{\delta}$,

$$\boxed{\mathbf{H}(\vec{x}_0) \cdot \vec{\delta} = -\vec{\nabla} f(\vec{x}_0).} \tag{8.16}$$

The notation $\mathbf{H}(\vec{x}_0)$ means (8.12) evaluated at $\vec{x} = \vec{x}_0$. If $f(\vec{x})$ is exactly 'parabolic', i.e.,

$$f(\vec{x}) = \vec{x}^{\,T} \cdot \mathbf{A} \cdot \vec{x} + \vec{b}^{\,T} \cdot \vec{x} + c, \tag{8.17}$$

then there are no terms $\mathcal{O}(|\vec{\delta}|^3)$ in the Taylor expansion, and $\vec{x}_1 = \vec{x}_0 + \vec{\delta}$ is the exact position of the minimum.

If the function is not well approximated by a quadratic then the estimate of $\vec{\delta}$ obtained from solving (8.16) will not take us directly to the minimum so we iterate this until we get arbitrarily close to it, i.e., $\vec{x}_{n+1} = \vec{x}_n + \vec{\delta}$ which can be written as

$$\boxed{\vec{x}_{n+1} = \vec{x}_n - [\mathbf{H}(\vec{x}_n)]^{-1} \cdot \vec{\nabla} f(\vec{x}_n).} \tag{8.18}$$

This is illustrated by the red arrows in fig. 8.4. This method can be very efficient with **quadratic convergence**, $|\vec{\delta}|_{n+1} \sim |\vec{\delta}|_n^2$, however in problems with large dimensions the calculation of the inverse Hessian can become very slow. In addition calculating the Hessian using finite differences often yields a matrix which is not strictly positive definite.

Example (adapted from Gerald and Wheatley p. 424) – Consider the 2D parabolic function

$$f(x,y) = x^2 + 2y^2 + xy + 3x$$

with the minimum at $\vec{x_*} = (x_*, y_*) = (-12/7, 3/7)$ from equating the derivatives to zero. If we start at the origin, the gradient and Hessian are given by

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} 2x + y + 3 \\ x + 4y \end{pmatrix} \quad , \qquad \mathbf{H} = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix} \quad .$$

It is easy to find the inverse of the Hessian matrix:

$$\mathbf{H}^{-1} = \begin{pmatrix} \frac{4}{7} & \frac{-1}{7} \\ \frac{-1}{7} & \frac{2}{7} \end{pmatrix} \tag{8.19}$$

.

Hence starting at $\vec{x} = \vec{x}_0 = (x_0, y_0)$, equation (8.16) yields

$$\begin{aligned}
\vec{\delta} &= -\mathbf{H}^{-1}\nabla f \\
&= -\begin{pmatrix} \frac{4}{7} & \frac{-1}{7} \\ \frac{-1}{7} & \frac{2}{7} \end{pmatrix} \cdot \begin{pmatrix} 2x_0 + y_0 + 3 \\ x_0 + 4y_0 \end{pmatrix} \\
&= -\frac{1}{7}\left(\begin{pmatrix} 8x_0 + 4y_0 + 12 - x_0 - 4y_0 \\ -2x_0 - y_0 - 3 + 2x_0 + 8y_0 \end{pmatrix}\right) \\
&= \begin{pmatrix} -x_0 - \frac{12}{7} \\ -y_0 + \frac{3}{7} \end{pmatrix} \\
&= \vec{x_*} - \vec{x}_0
\end{aligned}$$

so that $\vec{\delta}$ takes us directly to the minimum (i.e. $\vec{x}_0 + \vec{\delta} = \vec{x_*}$) in this case of a parabolic function.

Aside – What we have done in obtaining equation (8.16) is completely analogous to what would be done for finding the extremum of a simple quadratic $f(x) = ax^2 + bx + c$ if we know its gradient and curvature at a point x_0. The turning point $f' = 2ax + b = 0$ occurs at $x = -b/2a \equiv x_*$. Also $f'' = 2a$. The exact Taylor expansion is, $f(x_0 + h) = f(x_0) + hf'|_{x_0} + \frac{1}{2}h^2 f''|_{x_0} = (ax_0^2 + bx_0 + c) + h(2ax_0 + b) + \frac{1}{2}h^2(2a)$. The gradient of the Taylor expansion is

$$f'|_{x_0+h} = f'|_{x_0} + h\, f''|_{x_0} = (2ax_0 + b) + h(2a) + h^2(0).$$

From this we get the value of h to take us from x_0 to the turning point ($f'|_{x_0+h} = 0$);

$$h = -f'|_{x_0}/f''|_{x_0} = -x_0 - b/2a.$$

This can be re-expressed as $h = x_* - x_0$, demonstrating that getting h from the gradient of the Taylor expansion does indeed work for a parabola. For the multi-dimensional case equation (8.16) corresponds to the expression for h above.

Quasi-Newton Method

A disadvantage of Newton's method is that we need to calculate both first and second derivatives of the multi-dimensional function, and the inverse of the curvature matrix which takes $\mathcal{O}(N^3)$.

The Quasi-Newton method gets around this by approximating the inverse Hessian using the local gradient. The basic iteration step is

$$\boxed{\vec{x}_{n+1} = \vec{x}_n - \alpha\, \mathbf{G}_n \cdot \vec{\nabla} f(\vec{x}_n),} \tag{8.20}$$

where $\mathbf{G}_n$ is an approximation of $\mathbf{H}^{-1}(\vec{x}_n)$ and $\alpha \ll 1$.

For the first iteration $\mathbf{G}_0$ is set to the identity matrix $\mathbf{I}$, which makes this iteration the same as a gradient search. To update $\mathbf{G}_n \to \mathbf{G}_{n+1}$ we use the updates in $\vec{x}$ and $\vec{\nabla} f$;

$$\vec{\delta}_n = \vec{x}_{n+1} - \vec{x}_n \qquad , \qquad \vec{\gamma}_n = \vec{\nabla} f(\vec{x}_{n+1}) - \vec{\nabla} f(\vec{x}_n). \tag{8.21}$$

Then comparing the above expression for the gradient update $\vec{\gamma}_n$ with the gradient of the Taylor expansion of $f(\vec{x})$, to linear order,

$$\vec{\nabla} f(\vec{x}_{n+1}) \approx \vec{\nabla} f(\vec{x}_n) + \vec{\nabla}\left(\vec{\nabla} f(\vec{x}_n) \cdot \vec{\delta}_n\right)$$

we see that to linear order

$$\mathbf{H}_n^{-1} \cdot \vec{\gamma}_n = \vec{\delta}_n.$$

(This is equivalent to equation (8.16) and the preceding paragraph when $\nabla f(\vec{x}_0 + \vec{\delta}) \neq 0$.) The trick is then to update $\mathbf{G}$ to satisfy

$$\mathbf{G}_n \cdot \vec{\gamma}_n = \vec{\delta}_n \,. \tag{8.22}$$

so that $\mathbf{G}_n$ mimics the inverse Hessian. A number of methods exist for this update, each with different efficiency and convergence characteristics. One of the most common is the DFP (Davidon–Fletcher–Powell) algorithm where the update is

$$\mathbf{G}_{n+1} = \mathbf{G}_n + \frac{(\vec{\delta}_n \otimes \vec{\delta}_n)}{\vec{\gamma}_n \cdot \vec{\delta}_n} - \frac{\mathbf{G}_n \cdot (\vec{\gamma}_n \otimes \vec{\gamma}_n) \cdot \mathbf{G}_n}{\vec{\gamma}_n \cdot \mathbf{G}_n \cdot \vec{\gamma}_n}, \tag{8.23}$$

where $(\vec{u} \otimes \vec{v})_{ij} \equiv u_i v_j$ is the outer product of $\vec{u}$ and $\vec{v}$.

The advantage of this scheme is that it only involves (at worst) matrix multiplications, i.e., $\mathcal{O}(N^k)$ operations at each iteration and the resulting (approximated) Hessian is positive definite by construction. For full matrices (such as $\mathbf{G}_n$ and $(\vec{\delta}_n \otimes \vec{\delta}_n)$ here), naively one would think that the index k is 3. However it can be shown that matrix multiplication can be done with $k < 3$, in particular the Coppersmith & Winograd (1990) method scales with $k = 2.376$ and the commonly used `BLAS` routine scales with $k = 2.807$. Although these may seem close to $k = 3$ they make a big difference in computation time!

Global minimum search

These methods do not allow for the fact that the function may have multiple minima (local minima) in the interval we are interested in. There is no fool-proof method to find the global minimum (lowest minimum) of a function and we often need to restart algorithms from different starting points to check we have not found a local minimum.

8.3 Monte Carlo Minimisation – Simulated Annealing

Simulated annealing and related methods for optimisation introduce some randomness in the search for the minimum. They are particularly useful in cases where;

- The degrees of freedom in the system are so many that a direct search for an optimal configuration is too time consuming.
- Many local minima exist in which direct search methods can get stuck instead of finding the global minimum.

The simulated annealing approach is motivated by thermodynamics where random fluctuations can temporarily move a system to a less optimal (i.e. higher energy) configuration. The analogy is quantified in the use of the **Boltzmann Probability Distribution**

$$P(E)\, dE \sim \exp\left(-\frac{E}{k_{_B} T}\right) dE, \tag{8.24}$$

where "energy" E encodes a **cost function** and "temperature" T is a variable which determines the probability of changes in the energy of the system. In particular, even for low temperatures, it allows for the system to move to higher energies with very low probability. This is what can allow the system to get 'unstuck' from local minima and continue the search for a global minimum.

In practice the method involves the use of a **Metropolis Algorithm** where at each step in the iteration the configuration of the system is changed randomly. The "energy" of the system before (E_1) and after (E_2) the change are calculated to obtain the change in the system's energy: $\Delta E = E_2 - E_1$. The step is **accepted** with a probability p_{acc} given by the simple algorithm

$$\boxed{p_{\text{acc}} = \begin{cases} 1 & \text{if} \quad \Delta E \leq 0 \\ \exp(-\Delta E / k_{_B} T) & \text{if} \quad \Delta E > 0 \end{cases}} \tag{8.25}$$

In functional minimisation the "energy" is simply the value of the function $E = f(\vec{x})$ and the "temperature" T is slowly lowered to 0, hence the analogy with annealing of metals. At high T the steps explore a large region of the the parameter space $\vec{x}$. As T is reduced the accepted steps move closer to the minimum of the function, but always have the possibility of stepping to a different region to escape local minima. After running the annealing for sufficient time one can examine the accepted steps in $\vec{x}$ to identify $\vec{x}_*$

8.4 Using minimisation to solve matrix equations

The value of $\vec{x}$ that minimises the **quadratic equation**

$$\boxed{f(\vec{x}) = \frac{1}{2}\vec{x}^T \cdot \mathbf{A} \cdot \vec{x} - \vec{b}^T \cdot \vec{x},} \tag{8.26}$$

happens to be the solution to

$$\mathbf{A} \cdot \vec{x} = \vec{b}$$

for a **symmetric, positive-definite** $N \times N$ matrix $\mathbf{A}$. This is because the gradient of this quadratic equation is

$$\vec{\nabla} f(\vec{x}) = \mathbf{A} \cdot \vec{x} - \vec{b}. \tag{8.27}$$

At the minimum $\vec{x} = \vec{x}_*$,

$$\vec{\nabla} f(\vec{x_*}) = 0 \qquad \text{hence} \qquad \mathbf{A} \cdot \vec{x}_* = \vec{b}.$$

(Note the similarity of this quadratic with equation (8.17) seen earlier.) So to solve the matrix equation, one of the multi-dimensional iterative methods seen in section 8.2 can be used with the quadratic above.

The best iterative methods will (in the absence of round-off error) arrive at the exact solution in $N_{\text{iter}} = N$ iterations or less. This is better than Jacobi where convergence is as slow as $N_{iter} \sim \mathcal{O}(N^2)$ for some problems, and better than G-S & SOR. Convergence can be further accelerated by using a technique called ***pre-conditioning***. This effectively pushes the contours of $f(\vec{x})$ towards being spherical (in N dimensions), rather than very elongated ellipsoids (i.e. imagine fig. 8.2, but more elongated), so that a descent along the local gradient comes close to the global minimum.

To show that $\vec{\nabla} f = \mathbf{A} \cdot \vec{x} - \vec{b}$, consider the k-th component of the gradient of (8.26);

$$\begin{aligned}
(\vec{\nabla} f)_k &= \frac{\partial}{\partial x_k}\left[\frac{1}{2}\sum_i x_i \left(\sum_j A_{ij}x_j\right) - \sum_i b_i x_i\right] \\
&= \frac{1}{2}\sum_i \left[\delta_{ik}\left(\sum_j A_{ij}x_j\right) + x_i\left(\sum_j A_{ij}\delta_{jk}\right)\right] - b_k \\
&= \frac{1}{2}\left[\sum_j A_{kj}x_j + \sum_i x_i A_{ik}\right] - b_k \\
&= \frac{1}{2}\left[\mathbf{A}\cdot\vec{x}\right]_k + \frac{1}{2}\left[\vec{x}^T \cdot \mathbf{A}\right]_k - b_k \\
&= \frac{1}{2}\left[\mathbf{A}\cdot\vec{x}\right]_k + \frac{1}{2}\left[\mathbf{A}^T \cdot \vec{x}\right]_k - b_k \\
&= \left[\mathbf{A}\cdot\vec{x}\right]_k - b_k
\end{aligned}$$

where the last line makes use of the fact that $\mathbf{A}$ is symmetric. We have also used $\partial x_i/\partial x_k = \delta_{ik}$ where δ_{ik} is the Kronecker delta function.

Chapter 7

Random Numbers and Monte Carlo Methods

Outline of Section

- Pseudo-Random Numbers
- Transformation Method
- Rejection Method
- Monte Carlo Minimisation
- Monte Carlo Integration

The generation of (pseudo) random numbers according to pre-defined distributions is a tool that is used in many places in Computational Physics. Their use can, perhaps paradoxically given their "unpredictable" nature, add stability to otherwise deterministic but unstable algorithms, allow for calculations such as integrations to be made in high-dimensions that would not be feasible using more direct methods, and also mimic physical process that are by their nature fundamentally random—such as radioactive decay and other quantum mechanical processes where it is the probability distribution of outcomes that is known.

We first look at how to generate a uniform distribution of pseudo-random numbers in the range 0 to 1. We then see how to use this to generate arbitrary non-uniform distributions (e.g. triangular, Gaussian, etc.). Such distributions can directly be used in physical systems where randomness occurs, e.g., to determine the random direction of a particle emitted by a decay process. Another example is a ***stochastic force*** $\vec{F}_{\text{coll}}$ such as that producing Brownian motion of a small grain in a fluid, due to collisions with the fluid molecules.

Monte Carlo Methods are a broad class of methods where random numbers are used to statistically solve a problem. The 'problem' might be:

(a) A deterministic system where an enormous number of things are coupled together, and exact solution is unfeasible. Classical Brownian motion system is an example. Here it is impractical to actually follow the classical motion of the all the individual molecules. However, the statistics of all the collisions in a small time yield a distribution of net force on the grain. This can be randomly sampled.

(b) A statistical physics problem, such as calculating macroscopic thermodynamic quantities given the energy levels of the system.

(c) Minimising a function of many variables.

(d) Performing an integral over many variables/dimensions.

Generally, Monte Carlo Methods are useful for systems of many variables or dimensions.

7.1 Random Numbers

There are countless uses for random numbers in computational physics. Generally we need random numbers when we are simulating stochastic systems, e.g., Brownian motion, thermodynamical systems, state realisations and when we use Monte Carlo methods to calculate the variability of stochastic systems (more on this later).

A **uniform deviate** (or variate) is a random number lying within a range where any number has the same probability as any other. The range is usually $0 \leq x < 1$. Here x is a *random variable* and the deviates are the values that x can randomly take.

Most high-level languages come with a uniform random number generator that returns **pseudo-random numbers**. This means that the same sequence of (seemingly) random numbers can be regenerated by using the same **seed** number(s) to initiate the sequence. The seed is the number (or set of numbers) that the algorithm needs to start the sequence—once it has started, it will carry on indefinitely. This ability to regenerate the same sequence is an essential feature for debugging codes and reproducing results—compared to only being able to generate truly random sequences that are different every time.

A simple implementation of a uniform deviate generator is the **linear congruential generator**

$$\boxed{I_{n+1} = (a\, I_n + c)\%m,} \tag{7.1}$$

where I_n are the random numbers, a is a multiplier, c is the increment, and m is the modulus. ($a\%b$ denotes modulo division; i.e. the remainder of dividing a by b.) The parameters a, c and m are all integers. This will iteratively generate a sequence of pseudo-random **integers** in the (closed) interval between 0 and $m-1$ (usually stored as `RAND_MAX` in `C` implementations). We can turn the results into a real number by dividing by m

$$x_n = \frac{I_n}{m} = \frac{I_n}{\texttt{RAND\_MAX}+1} \qquad \text{where} \qquad 0 \leq x_n < 1. \tag{7.2}$$

One problem with this method is that the sequence of numbers will repeat itself with periodicity which is at most m, and for unfortunate choices of a, c & m the sequence length is much less than m (with a significant portion of integers in the range $0 \leq I < m$ being skipped). So generally we want a large m and a choice of a and c that ensures that the period is equal to m. You should be wary of the `rand()` function supplied with a compiler, especially `C` and `C++` compilers. The ANSI C standard specifies that `rand()` return type `int`, which can be as small as two bytes on some systems (i.e. `RAND_MAX`=32767). This is is not a very long period for Monte Carlo applications! These typically require many millions of random numbers.

Another problem with linear congruential generators is that there is correlation between successive numbers; if k successive random numbers are used to plot points in k-dimensional space, then the points do not fill up the space, but lie on distinct planes (of dimension $k-1$).

It is best to use random number generators other than the standard `C` one. There are good ones in the GSL libraries. Of note is the "Mersenne Twister" generator with an exceptionally long period of $m = 2^{19937} - 1 \sim 10^{6000}$, and the "RANLUX" generators which provides the most reliable source of uncorrelated numbers. In `Python` the basic `random()` function uses the Mersenne Twister generator, with a period of $2^{19937} - 1$, and is written in C.

7.2 Non-uniform distributions – Transformation Method

Often we need random deviates with a distribution other than uniform. The most common requirement is for a Gaussian distribution which is ubiquitous due to the Central Limit Theorem.

Given a generator that returns a uniform deviate x in the range 0 to 1, we can use the **transformation method** to obtain a new deviate (random variable) y which is distributed according to a *probability density function* (PDF) $P(y)$. Recall the basic properties of a PDF;

- A PDF must be positive, i.e., $P(y) \geq 0$.
- A PDF must be normalised, i.e., $\int_{-\infty}^{\infty} P(y)\, dy = 1$.

We are given x with PDF

$$U(x)\, dx = \begin{cases} dx & 0 \leq x < 1 \\ 0 & \text{otherwise} \end{cases} \tag{7.3}$$

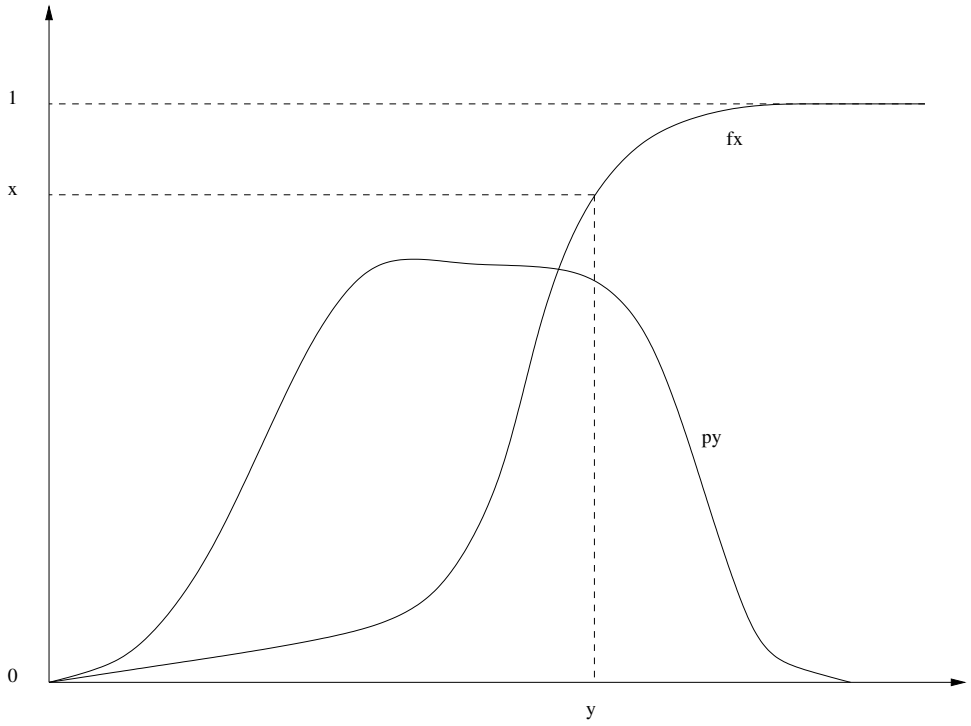

Figure 7.1: The transformation method. The Cumulative Distribution function $F(y) = \int_{-\infty}^{y} P(\tilde{y})d\tilde{y}$ is inverted to find a deviate y for every uniform random deviate x.

which is also normalised; $\int_{-\infty}^{\infty} U(x)\,\mathrm{d}x = 1$. We want our new random variable y to be a function of x, i.e., $y(x)$. The fundamental transformation law of probabilities then relates the PDFs of each variable via

$$|P(y)\,\mathrm{d}y\,| = |P(x)\,\mathrm{d}x\,|, \tag{7.4}$$

where $P(x) = U(x)$ in our case. This satisfies the requirement that both PDFs integrate up to unity. Assuming $dy/dx \geq 0$ (so that the $|\ldots|$ can be discarded) we have

$$\frac{\mathrm{d}x}{\mathrm{d}y} = P(y),$$

since $P(x) = U(x) = 1$. We can then integrate up and define the function

$$\boxed{F(y) = \int_{-\infty}^{y} P(\tilde{y})\,\mathrm{d}\tilde{y}\,,} \tag{7.5}$$

and then using the fundamental transformation law of probabilities it follows that

$$F(y) = \int_{-\infty}^{y} \frac{\mathrm{d}\tilde{x}}{\mathrm{d}\tilde{y}}\,\mathrm{d}\tilde{y} = \int_{0}^{x} d\tilde{x} = x. \tag{7.6}$$

(Note that $\sim$ is used to denote dummy integration variables here.) So if $F(y)$ is an invertible function, i.e.,

$$\boxed{y = F^{-1}(x)} \tag{7.7}$$

can be found, we can map each x given by a uniform deviate generator into a new variable y which will have the desired PDF $P(y)$. Note that $F(y)$ is just the **Cumulative Distribution Function** (CDF) of y. Since the PDF is normalised, the range of $F(y)$ is $0 \leq F(y) < 1$. The transformation method is illustrated in fig. 7.1.

Example – consider the *triangular* distribution $P(y) = 2y$ with $0 < y < 1$. The CDF is $F(y) = y^2 = x$ such that the transformation $y = \sqrt{x}$ gives deviates with the PDF $P(y)$.

7.3 Non-uniform distributions – Rejection Method

If the CDF is not a computable or invertible function then we can use the **rejection method** to obtain deviates with the required distribution.

We first draw a new function $f(y)$ called the ***comparison function*** which lies above $P(y)$ for all y. For simplicity let's define

$$f(y) = C, \tag{7.8}$$

where $C > P(y)$ everywhere. The procedure is then as follows:

(a) Pick a random number y_i, uniformly distributed in the range between $y_{\min}$ and $y_{\max}$.
(b) Pick a second random number p_i, uniformly distributed in the range between 0 and C.
(c) If $P(y_i) < p_i$ then reject y_i and go back to step (a). Otherwise accept.

The accepted y_i will have the desired distribution $P(y)$. To prove this, consider the probability that the algorithm above creates an acceptable y_i in the range between y and $y + dy$; this is

$$\text{Prob} = \frac{\mathrm{d}y}{y_{\max} - y_{\min}} \times \frac{P(y)}{C} = \text{const} \times P(y)\,\mathrm{d}y\,. \tag{7.9}$$

Efficiency – The efficiency of the rejection method is obtained by integrating the probability of acceptance which, since $P(y)$ is normalised, gives

$$\text{Eff} = \frac{1}{C}\frac{1}{y_{\max} - y_{\min}}. \tag{7.10}$$

1/Eff is the average number of times the steps (a)–(c) have to be carried out to get a y_i value. A way to interpret (7.10) is that it is the ratio of areas under the desired PDF $P(y)$ (perhaps a peaked curve) and the cover function $f(y)$ (a rectangle that fits over the PDF). By definition the PDF's area is $A_P = 1$. For the cover function $A_f = C \times (y_{\max} - y_{\min})$. Thus

$$\text{Eff} = \frac{A_P}{A_f}.$$

The rejection algorithm above effectively randomly picks a pair of coordinates (y_i, p_i) uniformly over the area A_f. A fraction of these 'throws' fall outside the area A_P under the PDF, in which case the coordinates are rejected and another throw is taken.

The efficiency can be improved by using a comparison function $f(y)$ that conforms more closely to the desired $P(y)$ (but still lies above it) and is suitable for use in the transformation method (i.e. it has invertible definite integral $\int_{y_{min}}^{y} f(\tilde{y})d\tilde{y}$). Step (a) then becomes;

(a) Using the transformation method, pick a random deviate y_i in the range between $y_{\min}$ and $y_{\max}$, distributed according to the PDF obtained by normalising $f(y)$.

Note that since $\int_{y_{min}}^{y_{max}} P(y)dy = 1$ and $f(y) \geq P(y)$ then the area under $f(y)$ is bigger than unity, so $f(y)$ needs to be normalised to be a PDF.

7.4 Monte Carlo Minimisation

An application of the Monte Carlo approach is to the problem of minimisation (optimisation) of functions. This is discussed in detail in section 8 where we also describe so-called *direct search* methods which do not involve random numbers.

7.5 Monte Carlo Integration

Consider calculating an integral of a d-dimensional function $f(\vec{x})$ over some volume V defined by boundaries a_i and b_i

$$I = \int_{a_1}^{b_1} dx_1 \int_{a_2}^{b_2} dx_2 \ldots f(\vec{x}) = \int_V f(\vec{x})\, d\vec{x}. \tag{7.11}$$

The integral I is related to the mean value of the function in the volume. This can be written as

$$\langle f \rangle = \frac{1}{V} \int_V f(\vec{x})\, d\vec{x}. \tag{7.12}$$

Then I can be obtained from the mean of the function as

$$I \equiv V \langle f \rangle. \tag{7.13}$$

This means we can *estimate* I by taking N random samples of the function in the volume

$$\hat{I} = \frac{V}{N} \sum_{i=1}^{N} f(\vec{x}_i). \tag{7.14}$$

We can also derive an estimate of the error in $\hat{I}$ by considering the estimate of the variance of the samples $f(\vec{x}_i)$, i.e.,

$$\sigma_{f_i}^2 = \frac{1}{N-1} \sum_{i=1}^{N} \left(f(\vec{x}_i) - \langle f \rangle \right)^2. \tag{7.15}$$

Given this, the variance of the mean $\langle f \rangle$ is $\sigma_{\langle f \rangle}^2 = \sigma_{f_i}^2 / N$ and given (7.13), we can write the variance in the estimate of I as

$$\sigma_{\hat{I}}^2 = V^2 \sigma_{\langle f \rangle}^2 = \frac{V^2}{N} \sigma_{f_i}^2. \tag{7.16}$$

Therefore we can state the estimate of I (including its error) as

$$\boxed{\hat{I} = \frac{V}{N} \sum_{i=1}^{N} f(\vec{x}_i) \pm \frac{V}{\sqrt{N}} \sigma_{f_i}.} \tag{7.17}$$

Sampling random points within the integration volume in this way with an equal probability within the volume is called "uniform sampling". A key point is that the error in MC integration scales as $\mathcal{O}(h^{1/2})$.

Example – Consider the integral of the function shown in Fig. 7.2, a uniform circle of radius 1/2;

$$f(x,y) = \begin{cases} 1 & \text{if } (x - \frac{1}{2})^2 + (y - \frac{1}{2})^2 < (\frac{1}{2})^2 \\ 0 & \text{otherwise} \end{cases}. \tag{7.18}$$

In this example, we have the luxury of knowing that

$$I = \int f(x,y)\, dx\, dy = \text{circle area} = \pi r^2 = \frac{1}{4}\pi. \tag{7.19}$$

To calculate I using MC integration, we sample N random, uniformly distributed points in the range $0 \leq x < 1$ and $0 \leq y < 1$. The value of the sampled integrand $f(x_i, y_i)$ will either be 1

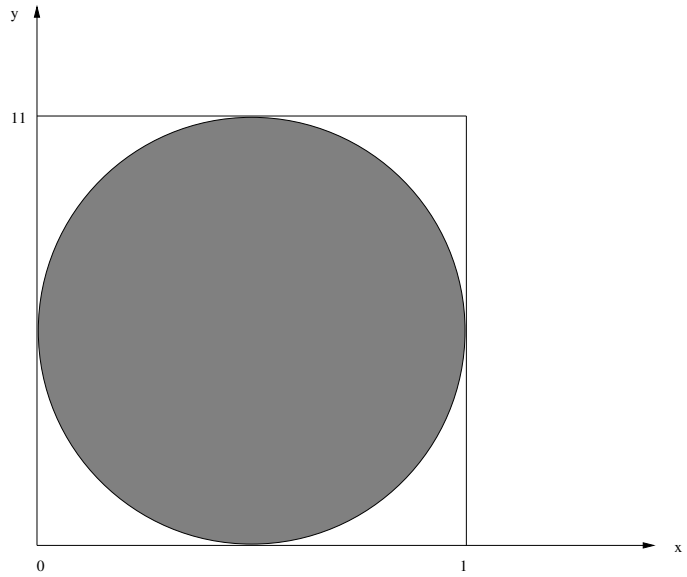

Figure 7.2: Circle of radius 1/2. The function $f(x, y) = 1$ inside the circle and $f(x, y) = 0$ outside

(*success*) or 0 (*fail*) to hit the circle. The ***probability of success***, p, for any given sample is the fraction of the square (area $A = \int_0^1 \int_0^1 dx\, dy = 1$) covered by the circle

$$p \equiv \frac{I}{A}. \tag{7.20}$$

The successes should follow a binomial distribution. Let N_1 be the number of successes. We can define an estimator for p as

$$\hat{p} = \frac{N_1}{N}. \tag{7.21}$$

We know that for a binomial distribution the variance of N_1 is

$$\sigma^2_{N_1} = N\, p\, (1 - p), \tag{7.22}$$

such that

$$\sigma^2_{\hat{p}} = \frac{\sigma^2_{N_1}}{N^2} = \frac{p\,(1-p)}{N}. \tag{7.23}$$

We can then have our estimate for the value of the integral I and its error

$$\boxed{\hat{I} = \hat{p}\, A = \hat{p} = \frac{N_1}{N} \pm \sqrt{\frac{p\,(1-p)}{N}}.} \tag{7.24}$$

Again we see that the error on the integral scales as the square root of the number of samples.

Comparison with the trapezium method—Earlier in the course we covered other methods for integration. The extended trapezium method for calculating the integral of a function is equivalent to a linear expansion (interpolation) of the function in each interval $x_i \to x_i + h$. We know that the truncation error goes as $\mathcal{O}(Nh^3) \sim \mathcal{O}(h^2)$. In general for d-dimensions we will evaluate the function at $N \sim h^{-d}$ points to calculate the integral. (Exactly $N = h^{-d}$ if the integration range in each dimension has unit length.) Therefore

$$h \sim N^{-1/d}, \tag{7.25}$$

such that the error in the trapezium method calculation of the integral is

$$\epsilon \sim \mathcal{O}(h^2) \sim \mathcal{O}(N^{-2/d}). \tag{7.26}$$

Thus for 4 or more dimensions the Monte Carlo method is as accurate or more accurate than the extended trapezium method for the same number of samples.

Error scalings $\longrightarrow$

d	Trapezium	MC
1	N^{-2}	$N^{-1/2}$
2	N^{-1}	$N^{-1/2}$
3	$N^{-2/3}$	$N^{-1/2}$
4	$N^{-1/2}$	$N^{-1/2}$
5	$N^{-2/5}$	$N^{-1/2}$
6	$N^{-1/3}$	$N^{-1/2}$

(7.27)

Importance sampling—For some integrals the integrand is highly weighted to particular regions of $\vec{x}$. It is then more efficient to bias random samples to where the integrand has more 'weight' and less to the larger volume of parameter space where the integrand is vanishingly small. For such an integral the integrand can be split as follows:

$$I = \int_V f(\vec{x})d\vec{x} = \int_V Q(\vec{x})P(\vec{x})d\vec{x} \tag{7.28}$$

where $P(\vec{x})$ behaves as (or is) a PDF (i.e. normalised and ≥ 0) and $Q = f(\vec{x})/P(\vec{x})$. Given that $\int_V P(\vec{x})d\vec{x} = 1$ we can write

$$I = \int_V Q(\vec{x})P(\vec{x})d\vec{x} = \langle Q \rangle \tag{7.29}$$

where

$$\langle Q \rangle \;=\; \frac{1}{N}\sum_{i=1}^{N} Q(\vec{x}_i) \;\pm\; \frac{1}{\sqrt{N}}\sigma_{Q_i} \tag{7.30}$$

and

$$\sigma_{Q_i}^2 = \frac{1}{N-1}\sum_{i=1}^{N}(Q(\vec{x}_i) - \langle Q \rangle)^2 \tag{7.31}$$

From this we can see that Equ. 7.17 is a special case of Equ. 7.30, where $P(\vec{x})$ is uniform and equal to $1/V$ (therefore $Q = Vf(\vec{x})$ and $\langle Q \rangle = V\langle f \rangle$). We are essentially doing the same thing as we did earlier, but with a different strategy for picking the samples. Instead of sampling evenly across the entire domain in which the integral is defined the samples are chosen to be proportional to $P(\vec{x})$. Because P is a PDF, the sum of the values of the samples divided by the number of samples gives the weighted average of Q with P as the weighting function.

The variation in $Q(\vec{x})$ is relatively flat, compared to if we had sampled/averaged $f = QP$ directly. This reduces σ_{Q_i} compared to σ_{f_i} and therefore *makes the error smaller* (for a given number of samples N).

To estimate I we now need to generate our random samples according the probability distribution $P(\vec{x})$. Sometimes this is simple - for some $P(\vec{x})$ distributions you can use the transformation method to convert a uniform random deviate to obtain $P(\vec{x})$, or you could use the rejection method. For more complex PDFs there is a better method.

The Metropolis Algorithm—The Metropolis algorithm can be used to pick samples that concentrate where $P(\vec{x})$ is largest. Essentially, it guides the 'trajectory' of $\vec{x}$ (taken to pick the samples) to seek the maximum of the $P(\vec{x})$ and wander about there. If the density of the samples is proportional to the size of $P(\vec{x})$, then the average of the values of $Q(\vec{x})$ at these sample points corresponds to the weighted integral of Q according to the PDF P.

Statistical Physics is a one example of an area where integrals can often be factored into a quantity to measure and a PDF. One wants to calculate the average of a quantity Q over all states, with a particular state being specified by $\vec{x}$. For example, for a classical system (or a hot

quantum one) where the probability is governed by the Boltzmann energy distribution $P(\vec{x}) \propto \exp(-E(\vec{x})/k_B T)$, the average over states is

$$\langle Q \rangle = \frac{1}{Z} \int_{\vec{x}} Q(\vec{x}) \exp\left(-\frac{E(\vec{x})}{k_B T}\right) d\vec{x}, \tag{7.32}$$

where $Z = \int_{\vec{x}} \exp(-E(\vec{x})/k_B T)\, d\vec{x}$ is the partition function.

We can define

$$P(\vec{x}) = \frac{\exp\left(-\frac{E(\vec{x})}{k_B T}\right)}{Z} \tag{7.33}$$

and so the Metropolis algorithm for calculating $\langle Q \rangle$ is:

- Randomly pick a starting point $\vec{x}$.
- Repeatedly use the Metropolis algorithm to select a sample point.
  - Make a small trial step/change in the parameters to get $\vec{x}'$.
  - Calculate $P(\vec{x}')$ and $P(\vec{x})$. Accept or reject step using

$$p_{\text{acc}} = \begin{cases} 1 & \text{if} \quad P(\vec{x}') \geq P(\vec{x}) \\ P(\vec{x}')/P(\vec{x}) & \text{if} \quad P(\vec{x}') < P(\vec{x}) \end{cases} \tag{7.34}$$

    If accepted $\vec{x} = \vec{x}'$, else $\vec{x} = \vec{x}$.
  - Evaluate $Q(\vec{x})$ and add to running total $\sum Q$ (and $\sum Q^2$ if necessary).
- (Repeat whole process for a series of different starting points.)
- Divide through by the total number of samples to get $\langle Q \rangle$.

In the context of statistical physics the maximum of P corresponds to *equilibrium states* and the wandering corresponds to *thermal fluctuations*. Note that if started far from equilibrium, the samples obtained during the process of finding the equilibrium skew the result somewhat. They should be omitted unless they are a negligibly small proportion of the total samples. Repeating the process for different random starting points is optional but ensures good, even coverage of the integrand. Otherwise, with a single starting point, more samples will need to be taken.

In the above, we have not said anything about how the next point should be chosen. This is given by the proposal density $g(\vec{x}', \vec{x})$, the probability that we choose $\vec{x}'$ as a candidate (proposal) for the next point when we are currently at $\vec{x}$. A typical example is to choose a point near the current point by sampling from a Gaussian probability distribution that is centred on the current point: $g(\vec{x}', \vec{x}) = N(\vec{x}'|\vec{\mu} = \vec{x}, \vec{\sigma})$. The widths of the Gaussian $\vec{\sigma}$ (since we are in multiple dimensions) are problem-specific and need to be adjusted so that the jump to the next point is not too long (compared to the region sizes over which $P(\vec{x})$ remains large or small) and not too short (so that it is difficult to leave any local minima and span the entire space).

The Metropolis algorithm given here is a special case of the "Metropolis-Hastings" algorithm, which allows the proposal density function to be asymmetric—which is to say that the probability to jump from point x_b when you are currently at point x_a does not have to be the same as the probability to jump to point x_a if you are currently at point x_b. So for Metropolis-Hastings, $g(\vec{x}', \vec{x}) = g(\vec{x}, \vec{x}')$ does *not* need to hold. In the Gaussian case above, these probabilities are, of course, identical.

This generalisation to Metropolis-Hastings is allowed if we modify the acceptance criteria (Eq. 7.34 to:

$$p_{\text{acc}} = \begin{cases} 1 & \text{if} A(\vec{x}', \vec{x}) \geq 1 \\ A(\vec{x}', \vec{x}) & \text{if} \quad A(\vec{x}', \vec{x}) < 1 \end{cases} \tag{7.35}$$

where A is given by:

$$A(\vec{x}', \vec{x}) = \frac{P(\vec{x}')}{g(\vec{x}'|\vec{x})} \Big/ \frac{P(\vec{x})}{g(\vec{x}|\vec{x}')} \tag{7.36}$$

An important point to note is that calculating $\langle Q \rangle$ requires $P(\vec{x})$ to be normalisable. In the example above we must be able to calculate the partition function. This is necessary to use the Metropolis and Metropolis-Hastings algorithms for integration, but is not required when using them to minimise functions, as discussed in Chapter 8.

Chapter 9

Ordinary Differential Equations

Outline of Section

- Introduction to solving ODEs
- Finite difference methods
- The Euler method
- Categories of initial values methods
- Higher-order Taylor methods
- Multi-step methods
- Implicit methods
- Runge-Kutta methods
- Relationship to interpolation
- Relationship to integration

9.1 Introduction

In countless problems in physics we encounter the description of a physical system through a set of ordinary differential equations (ODE). While these could be equations for any variables, in physics we often want to solve for changes as a function of time t, so we will use this as the independent variable for our equations in this and the following sections. (However, the methods we use could be applied to any variable type; the mathematics doesn't care what the variables correspond to physically.) Hence, we will often talk about 'timesteps' and 'time-stepping' in solving systems of differential equations. This simply refers to increments in the value of the independent variable (whether that independent variable actually physically corresponds to time or not). We will write $u(t)$ as the function we are trying to solve for. We will be looking at first-derivative ordinary differential equations, which for one variable can be generally written in the form

$$\frac{du}{dt} = f(t, u) \tag{9.1}$$

There are various convenient compact notations for a derivative; a time derivative is often written as $\dot{u}$ although we will use u' as this is more general for use in systems where the independent variable is not t. Hence we have $u' = f(t, u)$. Higher derivatives are indicated by more primes, e.g. u'' for the second derivative or $u^{n\prime}$ for the n^{th} derivative.

We can have more than one variable; e.g. for three variables, the general form is

$$\frac{du}{dt} = u' = f_u(t, u, v, w), \qquad \frac{dv}{dt} = v' = f_v(t, u, v, w), \qquad \frac{dw}{dt} = w' = f_w(t, u, v, w) \tag{9.2}$$

and we can write for a general m equation system

$$\frac{d\vec{u}}{dt} = \vec{u}' = \vec{f}(t, \vec{u}) \tag{9.3}$$

where u_i for $i = 0$ to $m - 1$ is an m component vector and each element has a function $f_i(t, \vec{u})$. Note that the derivatives of the dependent variables are in general a function of the independent

variable t *and* all the dependent variables (u, v, w, etc.), i.e. the system can be **coupled**. Only if the derivatives of each dependent variable are solely a function of that variable (and optionally t) is the system **uncoupled**.

To solve these equations, we are effectively doing integrations. The explicit connection between numerical integration and ODE solutions will be discussed in section (9.11). However, it should be clear that, as for integrations, we will need to know the constants of integration to solve the ODE. There must be one constant for every first-derivative equation, typically specifying the value of u at some time t_0. Clearly, the solution can be found analytically for the simplest cases but if the function describing the derivative is not separable or if the system of many variables is coupled, the equations cannot be integrated analytically and we have to take a numerical approach. We can solve systems numerically if the equations have a continuous solution and we know the information required for the constants of integration.

9.1.1 Higher-derivative equations

Many equations in physics involve higher derivatives, notably Newton's classical laws of motion which we can write, with u as the particle position, as $u'' = F(t, u, u')/m$ for a general force F. As shown, this can include a velocity dependence, e.g. for a charged particle in a magnetic field. There are methods specifically derived for this type of equation. However, it is always possible to use the velocity $v = u'$ as another dependent variable such that we have two coupled first-derivative equations

$$\frac{du}{dt} = u' = v = f_u(t, u, v), \qquad \frac{dv}{dt} = v' = \frac{F(t, u, v)}{m} = f_v(t, u, v) \tag{9.4}$$

where the u' derivative function $f_u(t, u, v) = v$, and so fits into our general approach with a function which happens to have a particularly simple form.[1] A similar approach could be done for a third-derivative equation, using the acceleration $a = v'$ as another dependent variable. In fact, *any* m^{th}-derivative equation can always be expressed as m first-derivative equations. Note there is some arbitrariness in defining the extra variables and a convenient choice can make the solution easier to find.

However, note the converse is not true. Consider the general case of two variables

$$\frac{du}{dt} = u' = f_u(t, u, v), \qquad \frac{dv}{dt} = v' = f_v(t, u, v) \tag{9.5}$$

which we want to express as a single second-derivative equation for u only. We can do

$$\frac{d^2u}{dt^2} = \frac{df_u}{dt} = \frac{\partial f_u}{\partial t} + \frac{\partial f_u}{\partial u}u' + \frac{\partial f_u}{\partial v}v' = \frac{\partial f_u}{\partial t} + \frac{\partial f_u}{\partial u}f_u + \frac{\partial f_u}{\partial v}f_v \tag{9.6}$$

Note the RHS is a function of t, u and v. To remove v, we need to invert $u'(t, u, v)$ to get $v(t, u, u')$ and then substitute this into the above. However, this inversion is not always practical or even possible. (For the previous case which *did* come from a second-derivative equation, $u' = v$ so the inversion is trivial.) Hence, we will study methods for solving m first-derivative equations as these can also be applied to higher-derivative equations as well and hence cover the most general case.

9.1.2 Linear systems of first-derivative ODEs

Linear systems are important in many areas of physics, either in their own right or as approximations to more complicated systems. Consider a linear system with one variable

$$u' = f(t, u) = au \tag{9.7}$$

where a can in general be a function of time. If a is not time-dependent, the analytic solution is easily found to be $u(t) = u_0 e^{(t-t_0)a}$. Hence $a > 0$ gives a rising exponential while $a < 0$ gives a

[1] The classical physics Lagrangian and Hamiltonian approach can be considered as a formal method for taking the Euler-Lagrange second-derivative equations and converting them into twice as many Hamiltonian first-derivative equations.

falling exponential. If it is time-dependent, the solution is $u(t) = u_0 e^{\int_{t_0}^{t} a(t')\,dt'}$, which could rise or fall at different times, depending on the function $a(t)$.

With multiple variables, the generalisation of Eq. (9.7) is

$$\vec{u}' = \mathbf{A}(t) \cdot \vec{u} \tag{9.8}$$

where $\mathbf{A}$ is an $m \times m$ matrix for m variables. If this matrix is not a function of time, the formal solution is $\vec{u} = e^{(t-t_0)\mathbf{A}} \cdot \vec{u}_0$ where the exponential of a matrix is defined through its series expansion

$$e^{(t-t_0)\mathbf{A}} = \mathbf{I} + (t-t_0)\mathbf{A} + \frac{(t-t_0)^2}{2!}\mathbf{A}^2 + \frac{(t-t_0)^3}{3!}\mathbf{A}^3 + \ldots \tag{9.9}$$

In general the matrix $\mathbf{A}$ is not diagonal, meaning the equations for the u_i are coupled. In principle, we can diagonalise $\mathbf{A}$ and transform $\vec{u}$ into uncoupled linear eigenvector combinations of the u_i. This is conceptually similar to a normal modes decomposition. For each positive eigenvalue, there will be a combination which is exponentially rising, and for each negative eigenvalue, one which is exponentially falling. Hence, these solutions are effectively the same as for the single variable case. However, for more than one variable, there is another possible type of solution corresponding to the complex eigenvalues. These can occur even for real matrices and always occur in complex conjugate pairs in that case. The two corresponding eigenvectors of each complex conjugate pair are also complex conjugates of each other and are oscillating variables, where the oscillation frequency is given by the imaginary part of the eigenvalue. If the real part is non-zero, they will also have a rising or falling exponential factor as well as the oscillation. The simplest example is the SHO equation $u'' + \omega^2 u = 0$. Defining a (scaled) velocity v through $u' = \omega v$, then $u'' = \omega v'$ and hence $v' = -\omega u$. These can be written

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \begin{pmatrix} 0 & \omega \\ -\omega & 0 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} \qquad \text{so} \qquad \mathbf{A} = \begin{pmatrix} 0 & \omega \\ -\omega & 0 \end{pmatrix} \tag{9.10}$$

This matrix $\mathbf{A}$ has eigenvalues $\lambda^2 + \omega^2 = 0$ so $\lambda = \pm i\omega$, which are indeed a complex conjugate pair. The eigenvectors correspond to $c = u + iv$ and c^* so, for example, the $-i\omega$ eigenvalue equation is

$$c' = u' + iv' = -i\omega c \qquad \text{so} \qquad c = c_0 e^{-i\omega(t-t_0)} \tag{9.11}$$

is the solution, where the initial conditions are contained in $c_0 = u_0 + iv_0$ In terms of u and v, this means

$$u' + iv' = -i\omega(u + iv) \qquad \text{i.e.} \qquad u' = \omega v \qquad \text{and} \qquad v' = -\omega u \tag{9.12}$$

as required. Note, for both real or complex cases, the solution goes as $e^{\lambda t}$.

Since all linear first-derivative ODEs reduce down to these two types of solutions, i.e. exponential (falling or rising) and oscillating, we will often study these solutions when evaluating ODE numerical methods.

9.2 Finite difference methods

In this course, we will look at **finite difference methods** (FDM). In these methods, we do not attempt to solve to obtain a continuous function $u(t)$, but we solve for values of the dependent variables at discrete points in time $u(t_0)$, $u(t_1)$, etc. If a smooth continuous function is required, then interpolation as described in section (4) can be done between the discrete points afterwards. In all the methods described in detail here, the time steps are taken to be equal and denoted by Δt so the time values will be $t_0 + n\Delta t$ for integer n. However, more sophisticated methods involve varying the step size.

The basic concept for the one variable case is that the constant of integration gives a value of $u = u_0$ at some time t_0 and we then step forward and/or backward in time until we cover the required range of time for which we want the solution. Because we can calculate the derivative at t_0 using $u'_0 = f(t_0, u_0)$, we can make some estimate of the function at some small time step Δt

away, i.e. at $t_1 = t_0 + \Delta t$. All the methods discussed in this chapter are about how to make such estimates. Having got an estimate of e.g. $u_1 = u(t_1)$, we can use this value to find $u_1' = f(t_1, u_1)$ and so project further in time for the next estimate $u_2 = u(t_2)$, and so on. For clarity we will not consider the most general stepping in both directions from an arbitrary starting time. We will assume we only want to step forwards from a time t_0. Hence, the known value of $u_0 = u(t_0)$ is the starting value. The general concept is shown in figure (9.1). This means these problems are solved using what are called **initial value methods**.

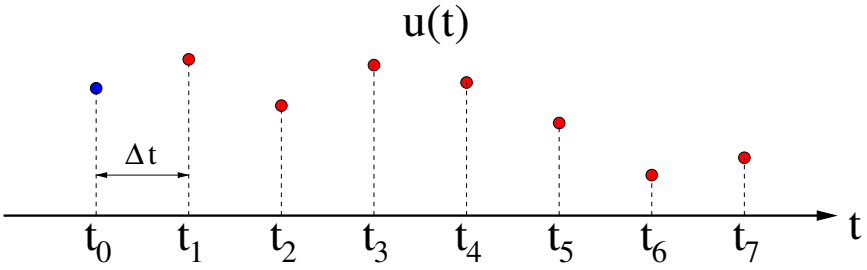

Figure 9.1: *Concept of a finite difference method. Starting from the initial condition $u(t_0)$ at time t_0, (blue circle), the solution is found subsequent times spaced by Δt (red circles).*

The multiple variable case is more complicated. If the values of the variables are all defined for the same time t_0, i.e. $\vec{u}(t_0)$ is known, then the same concept as for the one variable case can be applied as we can calculate the derivatives $\vec{u}' = \vec{f}(t_0, \vec{u}_0)$ and hence can proceed as before; this is also an initial value problem; see figure (9.2) left. However, if any (or all!) are specified at different times, then we do not have enough information to calculate the derivatives at any single time in general, as they can all depend on all components of $\vec{u}$. In this case we cannot use initial value methods to solve within the region between the earliest and latest time at which any of the values are specified, as shown in figure (9.2) right. Instead we have to use different approaches called **boundary value methods**, so-called as there will be at least one variable specified at each of the two ends of the time period, i.e. on the "boundaries" of the time period. Having solved for this time range, initial value methods can be used to extend in time beyond the ends of the range in either or both directions. Again, for clarity, we will just consider solving the boundary value range by itself; these methods are discussed in section (10).

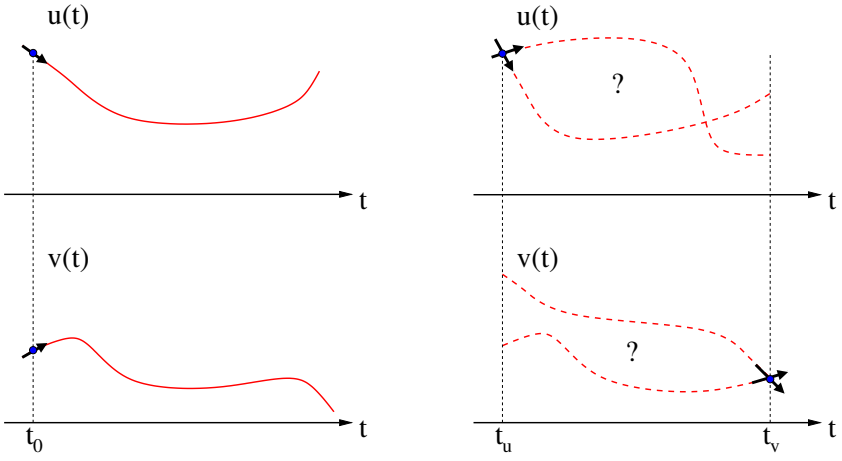

Figure 9.2: *Left: Concept of an initial value problem for two variables. Because the specified values are given at the same time t_0, the derivatives $u' = f_u(t_0, u_0, v_0)$ and $v' = f_v(t_0, u_0, v_0)$ can be calculated at that time. Right: If the two specified values are given at different times t_u and t_v, then neither derivative is known and a boundary value method has to be used.*

All methods for solving these equations have been developed to keep down the number of

evaluations of the functions which give the derivatives. This is because these functions can be non-trivial; they could be the result of a significant numerical calculation themselves. Hence, the methods are optimised to squeeze as much accuracy as possible out of the evaluations that are done.

9.3 The Euler method

The simplest initial value method for numerical integration of an ODE is obtained from the first two terms of a Taylor expansion for the step, which gives

$$\begin{aligned} u(t+\Delta t) &= u(t) + u'(t,u)\Delta t + +u''(t,u)\frac{\Delta t^2}{2!} + +u'''(t,u)\frac{\Delta t^3}{3!} + \dots \\ &\approx u(t) + f(t,u)\Delta t \end{aligned} \tag{9.13}$$

This is called a **first-order** method as it keeps terms up to first order in Δt. Other methods are designed to keep more terms and generally an $\boldsymbol{m^{th}}$**-order** method keeps terms up to Δt^m.

The method means that if we know the value of the variable u at t we can use a finite step Δt to calculate the next value of u at $t + \Delta t$ up to $\mathcal{O}(\Delta t^2)$ accuracy. This is illustrated in figure (9.3).

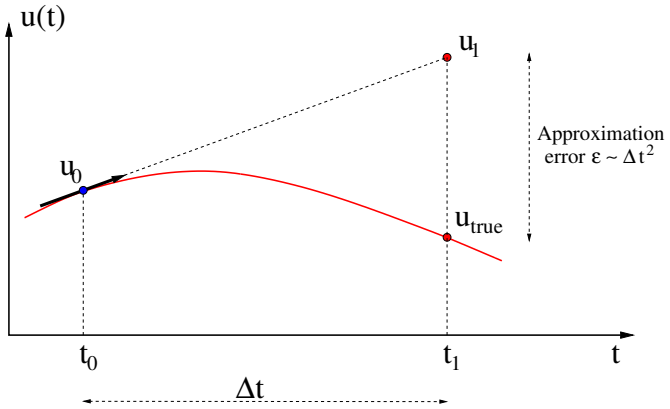

Figure 9.3: *The Euler method applied to a non-linear function shown in red. The first step from t_0 to t_1 is made by projecting along the derivative at t_0, as shown by the black arrow. A truncation (or approximation) error, from dropping higher terms in the Taylor series, arises because the gradient is evaluated at the starting point and used to obtain the next point, while the true function is non-linear.*

This Euler step can be repeated, stepping the solution forward Δt each time; we can write this as

$$\boxed{u_{n+1} = u_n + f(t_n, u_n)\Delta t} \tag{9.14}$$

where we use the subscript n to denote values of the function u at the beginning of the **time step** which starts at time

$$t_n = t_0 + n\,\Delta t. \tag{9.15}$$

It is critical for the function to be continuous and smooth over the time of the step as otherwise the Taylor series and hence Euler method will not give sensible results. If there is a discontinuity, such as a switch being changed in an electronic circuit, then the solution must have a step ending at the time of the discontinuity and the next step is then calculated based on the new circumstances.

The Euler method can also be applied to systems with multiple variables. The method is effectively identical and the equations are

$$\vec{u}_{n+1} = \vec{u}_n + \vec{f}(t_n, \vec{u}_n)\Delta t \tag{9.16}$$

This straightforward generalisation for multiple variables also holds for all methods discussed below. The main thing to note is that in general the functions $\vec{f}$ each depend on *all* the variables $\vec{u}$ so you need to make sure all variables are updated to step n before moving on to calculating any of the $\vec{u}_{n+1}$ values.

9.3.1 The Euler method for linear systems

Consider the linear system with one variable

$$u' = f(t,u) = au \tag{9.17}$$

where a can in general be a function of time. Writing $a(t_n) = a_n$, the Euler method gives

$$u_{n+1} = u_n + (a_n u_n)\Delta t = (1 + a_n \Delta t)u_n \tag{9.18}$$

Hence, the value of u_n is "updated" by a factor of $g_n = 1 + a_n\Delta t$. With multiple variables, the generalisation is

$$\vec{u}' = \mathbf{A}(t) \cdot \vec{u} \tag{9.19}$$

so writing $\mathbf{A}(t_n) = \mathbf{A}_n$, the Euler method gives

$$\vec{u}_{n+1} = \vec{u}_n + (\mathbf{A}_n \cdot \vec{u}_n)\Delta t = (\mathbf{I} + \mathbf{A}_n\Delta t) \cdot \vec{u}_n = \mathbf{G}_n \cdot \vec{u}_n \tag{9.20}$$

where $\mathbf{G}_n$ is called the *update matrix*.

9.4 Categories of initial value methods

So far we have only seen one finite difference method for integrating ODEs; the Euler method

$$u_{n+1} = u_n + f(t_n, u_n)\Delta t. \tag{9.21}$$

The Euler method is the simplest and we will cover a number of more advanced methods for solving ODEs numerically in this chapter. We will find they are all based on the exact identity

$$u_{n+1} = u_n + \overline{u}'_n \Delta t \tag{9.22}$$

where $\overline{u}'_n$ is the average gradient within the time period of the step; see figure (9.4). All methods correspond to estimating $\overline{u}'_n$ by different approximations. For the Euler method, the estimate of the average gradient $\overline{u}'^e_n$ is defined to be $\overline{u}'^e_n = f(t_n, u_n)$, i.e. the average gradient is approximated to be the gradient at the start of the step period.

For (almost) all methods discussed, the average gradient is estimated by a weighted average of one or more function evaluations somewhere within the step(s) used

$$\overline{u}'_n \approx \sum_a w_a\, f(t_n + \delta t_a, u_n + \delta u_a) \qquad \text{where} \qquad \sum_a w_a = 1 \tag{9.23}$$

Hence, the Euler method corresponds to approximating the average gradient by the gradient at the initial time of the step. This means there is one evaluation of the function at t_n, i.e. using $\delta t = 0$, and using the value at that point u_n, so $\delta u = 0$. Since there is only one evaluation, it must therefore have a weight of 1. For explicit comparison with other methods, we can write the Euler method as

$$\boxed{\begin{aligned} f_a &= f(t_n, u_n), \qquad (9.24a)\\ u_{n+1} &= u_n + f_a \Delta t \qquad (9.24b) \end{aligned}}$$

In this chapter we look at some more finite difference methods which are superior to Euler. To improve on the Euler method, we generally need to go to a higher-order truncation error per step to improve the accuracy. To do this requires using more information, specifically from the function $f(t,u)$ or from other values of t and u.

These improved methods can be considered as extensions of the Euler method and the different methods extend in different ways. There are two main categories for methods:

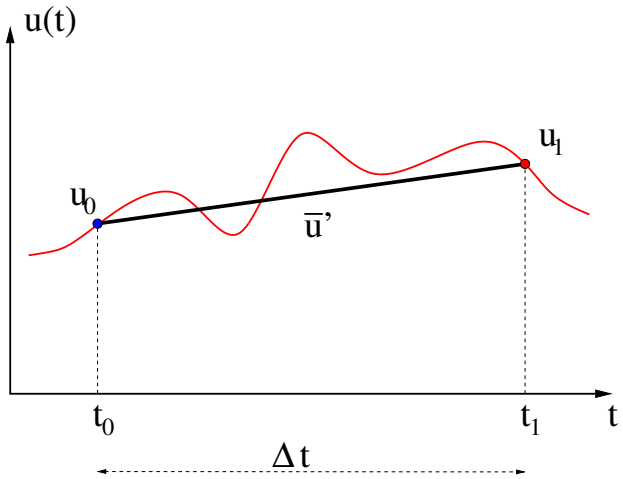

Figure 9.4: *The average gradient $\overline{u}'$ multiplied by the time difference Δt gives the difference between the function values at two times $u(t_0)$ and $u(t_0 + \Delta t = t_1)$.*

- **Single-step** methods only use the solution at the current iteration (u_n) to get the next value u_{n+1}. In contrast, **multi-step** methods use values from other steps as well (e.g. u_{n-1}) in order to get u_{n+1}. Whether single- or multi-step, the function must be smooth over the time period covered by all the steps involved in the method. Euler is a single-step method.
- **Explicit** methods involve evaluating $f(t, u)$ using only known values of u, e.g. u_n. There are also **implicit** methods which use $f(t_{n+1}, u_{n+1})$ as part of the method. Since the resulting value will be used to find u_{n+1}, these require iterating or some other manipulations to find the solution per step. Multi-step implicit methods use $f(t, u)$ both at known and unknown u values. Euler is an explicit method.

Typically, higher-order methods give better accuracy but take more computational time, which can be balanced by adjusting the step size Δt. The appropriate choice depends on the particular problem being solved, but generally fourth-order methods give a reasonable balance of accuracy and speed. Explicit formulae for fourth-order methods are given where appropriate below.

9.5 Higher-order Taylor methods

The Euler method is based on the first two terms of the Taylor series around u_n

$$u_{n+1} = u_n + u'_n \Delta t + u''_n \frac{\Delta t^2}{2!} + u'''_n \frac{\Delta t^3}{3!} + \dots \tag{9.25}$$

where $u'_n = f(t_n, u_n)$ and the terms of Δt^2 and higher are ignored. We want to account for the higher-order terms to improve the step accuracy. The most obvious extension is to directly use more terms in the Taylor expansion. This can be done if the derivative function $f(t, u)$ is known analytically. In principle, this can then be differentiated to give the second total time derivative of u

$$u'' = f' = \frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u} u' = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u} f \tag{9.26}$$

and this can be continued to give $u''' = du''/dt$, etc. These terms are then used in Eq. (9.25) to evaluate u_{n+1}. Hence, the extra information needed in these methods comes from our knowledge of the function $f(t, u)$. These are called **higher-order Taylor (HOT)** methods; HOTm keeps terms up to m^{th}-order. They are explicit single-step methods. Figure (9.5) shows the idea for the second-order Taylor method (HOT2).

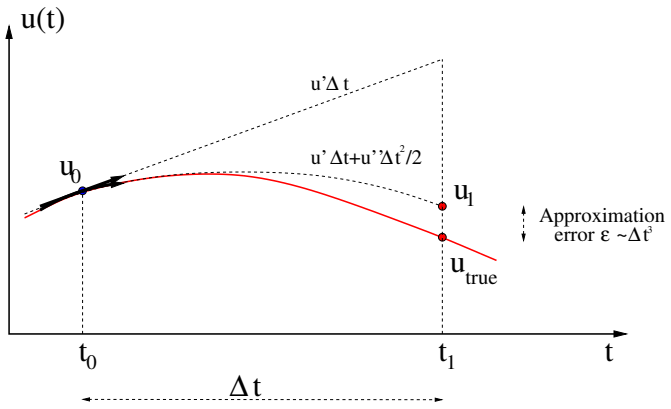

Figure 9.5: *The second-order Taylor method applied to a non-linear function shown in red. The first step from t_0 to t_1 is made by projecting a quadratic curve using the derivative and second derivative at t_0, as shown by the black arrow. An approximation error (from dropping higher terms in the Taylor series) arises because the quadratic is evaluated at the starting point and used to obtain the next point, while the true function is not perfectly quadratic. Compare with figure (9.3).*

The Taylor series can be interpreted to mean the average gradient across the step $\overline{u}'_n$ is given by

$$\overline{u}'_n = u'_n + u''_n \frac{\Delta t}{2!} + u'''_n \frac{\Delta t^2}{3!} + \dots \tag{9.27}$$

which is indeed exact if all terms are included. In this particular method, unlike all the others, we are not using a weighted sum of derivative estimates to find $\overline{u}'_n$.

The higher-order Taylor methods are limited by the complexity of handling the differentiation required (both analytically and in terms of coding) and eventually by the rounding error which will prevent the higher-order Δt^m terms from contributing. However, for relatively simple analytic functions, this is a good choice of method.

In some cases, the function $f(t, u)$ is not necessarily known analytically and even if it is, there can be difficulties in implementing the derivatives accurately. It might be thought that the higher derivatives could be accurately estimated using numerical calculus techniques, e.g. for u'' we need

$$\frac{\partial f}{\partial t} \approx \frac{f(t_n + \epsilon_t, u_n) - f(t_n, u_n)}{\epsilon_t} \qquad \text{and} \qquad \frac{\partial f}{\partial u} \approx \frac{f(t_n, u_n + \epsilon_u) - f(t_n, u_n)}{\epsilon_u} \tag{9.28}$$

for some very small ϵ_t and ϵ_u. While this is true, the number of function evaluations needed per step may be quite high so the method would not be very fast. In a similar way to the secant method compared to the Newton-Raphson method in Section (6.7), calculating with $\epsilon_t \ll \Delta t$ does not bring any advantages as we would only be finding a very precise result for the truncated series (i.e. an approximation) anyway. Hence, it turns out there are clever ways to space out any extra function evaluations which are then faster; these are the Runge-Kutta methods described later in this chapter.

9.5.1 Linear systems

For a linear function $u' = au$, the higher derivatives are trivial to find

$$u''_n = a_n u'_n = a_n^2 u_n, \quad u'''_n = a_n u''_n = a_n^3 u_n, \quad \dots \quad u_n^{m'} = a_n^m u_n \tag{9.29}$$

so the step equation becomes

$$u_{n+1} = \left(1 + a_n \Delta t + a_n^2 \frac{\Delta t^2}{2!} + a_n^3 \frac{\Delta t^3}{3!} + \dots\right) u_n \tag{9.30}$$

up to whatever order is required. A similar equation in terms of the matrix $\mathbf{A}_n$ holds for the multi-variable case.

9.6 Multi-step methods

As stated previously, multi-step methods use other values of t and/or u than just t_n and u_n in order to find u_{n+1}. Clearly, these other values bring in the extra information needed to get higher accuracy.

Explicit multi-step methods use the previous values already calculated in earlier time steps, e.g. u_{n-1} and/or $f(t_{n-1}, u_{n-1})$. Since these values are already known, no extra function evaluations are required and so these methods can be very fast per step.

9.6.1 Multi-step methods using previous u values

How would we do an explicit multi-step method to one order better than Euler, i.e. retaining the Δt^2 term? Writing the Euler step as

$$u_{n+1} = u_n + f(t_n, u_n)\Delta t \qquad \text{gives} \qquad f(t_n, u_n) = \frac{u_{n+1} - u_n}{\Delta t} \tag{9.31}$$

which is seen to be a forward difference first-order approximation for the derivative. As shown in section (5.1), the second-order centre difference approximation would be

$$f(t_n, u_n) = \frac{u_{n+1} - u_{n-1}}{2\Delta t} \tag{9.32}$$

Rearranging gives

$$\boxed{u_{n+1} = u_{n-1} + 2f(t_n, u_n)\Delta t} \tag{9.33}$$

This is called the **leapfrog** method as it jumps over u_n to move from u_{n-1} to u_{n+1}. It can alternatively be derived using a Taylor expansion at u_n to give u_{n-1} and using this to eliminate u''_n. The leapfrog is an explicit multi-step method and is second-order accurate. To put it in the general weighted average form, we can rearrange to give

$$u_{n+1} = u_n + \left[2f(t_n, u_n) - \left(\frac{u_n - u_{n-1}}{\Delta t}\right)\right]\Delta t \tag{9.34}$$

so it can also be interpreted as a weighted average of two approximations to the average gradient. The weights are 2 and -1 and so sum to one, as required.

However, this method is rarely used because of a stability issue, as discussed in the next chapter. Unfortunately, going to higher orders by using u_{n-2}, u_{n-3} or other earlier values does not get round this problem and we will not consider them further.

9.6.2 Second-order multi-step using previous f values

The solution to the instability mentioned above is to use $f(t_{n-1}, u_{n-1})$ rather than u_{n-1} for the extra information needed. To get a second-order step method, doing the Taylor expansion for u' (not u) around u'_n gives

$$u'_{n-1} = u'_n - u''_n\Delta t \qquad \text{so} \qquad u''_n = \frac{u'_n - u'_{n-1}}{\Delta t} \tag{9.35}$$

Using this in the Taylor expansion for u_{n+1} to second-order gives

$$u_{n+1} = u_n + u'_n\Delta t + u''_n\frac{\Delta t^2}{2} = u_n + u'_n\Delta t + \frac{1}{2}(u'_n - u'_{n-1})\Delta_t \tag{9.36}$$

so replacing the u' with f gives

$$\boxed{u_{n+1} = u_n + \frac{1}{2}\left[3f(t_n, u_n) - f(t_{n-1}, u_{n-1})\right]\Delta t} \tag{9.37}$$

This is the **second-order Adams-Bashforth** method, known as AB2. It is an explicit multi-step method. The weights are $3/2$ and $-1/2$ so again sum to 1.

9.6.3 Higher-order multi-step using previous f values

The same method can be enlarged to include more earlier $f(t,u)$ terms so as to evaluate u'''_n and higher derivatives. For example, the **fourth-order Adams-Bashforth** method (AB4) yields

$$\boxed{u_{n+1} = u_n + \frac{1}{24}\left[55f(t_n,u_n) - 59f(t_{n-1},u_{n-1}) + 37f(t_{n-2},u_{n-2}) - 9f(t_{n-3},u_{n-3})\right]\Delta t} \tag{9.38}$$

where again, the weights sum to one. This method can be written as

$$\boxed{\begin{aligned} f_a &= f(t_n,u_n), && \text{(9.39a)}\\ f_b &= f(t_{n-1},u_{n-1}), && \text{(9.39b)}\\ f_c &= f(t_{n-2},u_{n-2}), && \text{(9.39c)}\\ f_d &= f(t_{n-3},u_{n-3}), && \text{(9.39d)}\\ u_{n+1} &= u_n + \frac{1}{24}\left(55f_a - 59f_b + 37f_c - 9f_d\right)\Delta t && \text{(9.39e)} \end{aligned}}$$

9.6.4 Starting off

A significant drawback of multi-step methods is that the previous values are needed; the number of previous points required depends on the order of the method. Hence, we cannot apply a multi-step method immediately from t_0 as we don't have any previous values. Typically, what is done is to use a single-step method for the first iteration(s). For instance to start off the AB2 method, an Euler step can be used to get u_1 from the initial condition u_0. Following this, there are enough points to continue with the AB2 scheme:

$$\begin{aligned} u_1 &= u_0 + f_0\,\Delta t,\\ u_2 &= u_1 + (3f_1 - f_0)\Delta t/2,\\ u_3 &= u_2 + (3f_2 - f_1)\Delta t/2 \end{aligned} \tag{9.40}$$

and so on. It can be the case that a few low accuracy steps at the beginning dominate the overall error of a solution. This is effectively equivalent to an initial condition error. Hence, it is important to be careful with the accuracy of the starting steps. It is better to use a single step method with at least the same order accuracy even if is slower. To improve the accuracy of the starting step, a better single-step method could be used and/or smaller steps can be used (e.g. use k Euler steps with $\Delta t \to \Delta t/k$ to get u_1); the extra computational time should be small if it is only a handful of steps.

Multi-step methods also are more complicated to use with non-smooth functions. They cannot be run continuously over the time of a discontinuity as the discontinuity time will always appear within the period being used by one or more of the steps. These methods have to be halted and then restarted after the discontinuity, using a single step method as described above.

9.7 Implicit methods

Implicit methods for solving an ODE can be thought of as an extension of multi-step methods to include $f(t_{n+1},u_{n+1})$. The simplest is the **implicit Euler method**:

$$\boxed{u_{n+1} = u_n + f(t_{n+1},u_{n+1})\Delta t} \tag{9.41}$$

This is basically the Euler method but using the derivative at the *end* of the time step rather than at the beginning, so is equivalent to the backwards difference derivative approximation for $f(t_{n+1}, u_{n+1})$. Implicit Euler is still a first-order method, just like its explicit cousin.

However, the function requires evaluation using the unknown value u_{n+1} and the problem is how to solve this equation.

9.7.1 Implicit Euler for linear systems

For a linear system, where $f(t, u) = au$, equation (9.41) is easily solved explicitly, as

$$u_{n+1} = u_n + a\Delta t\, u_{n+1} \qquad \text{so} \qquad u_{n+1} = \frac{u_n}{1 - a\Delta t} \tag{9.42}$$

Implicit Euler also works for coupled first-order ODEs. For the linear case

$$\begin{aligned} \vec{u}_{n+1} &= \vec{u}_n + \mathbf{A} \cdot \vec{u}_{n+1}\Delta t \\ \therefore \quad (\mathbf{I} - \mathbf{A}\Delta t) \cdot \vec{u}_{n+1} &= \vec{u}_n \\ \therefore \quad \vec{u}_{n+1} &= (\mathbf{I} - \mathbf{A}\Delta t)^{-1} \cdot \vec{u}_n = \mathbf{G} \cdot \vec{u}_n \end{aligned} \tag{9.43}$$

where $\mathbf{A}$ is the same matrix operator as for explicit Euler. The update matrix $\mathbf{G}$ is now the inverse of the matrix $(\mathbf{I} - \mathbf{A}\Delta t)$. Hence implicit Euler works if $(\mathbf{I} - \mathbf{A}\Delta t)$ is a non-singular matrix so that $\mathbf{G}$ exists and can be found.

Note, this can be slow if $\mathbf{A}$ is time dependent. In this case, the resulting matrix $\mathbf{A}_n$, and hence update matrix $\mathbf{G}_n$, will be different at each step. Hence the matrix inversion has to also be re-calculated per step and not just done once before starting the steps. This would make this technique very slow for a large number of variables.

9.7.2 Implicit Euler for non-linear systems

With non-linear ODEs, things are not so easy. In principle one needs to find the solution of a set of (coupled) non-linear algebraic equations. Equation (9.41), whether for one or more variables, is a non-linear algebraic equation in u_{n+1}, that in principle needs to be solved using something like the bisection, Newton-Raphson or secant method, as discussed in section (6.7). This can be very computationally intensive.

Another approach is to linearise the functions $\vec{f}$ in each ODE about the known 'point' $(t_n, \vec{u}_n)$, by taking the first two terms of a Taylor expansion. However, since the resulting matrix equivalent of $\mathbf{A}$ will depend on $\vec{u}_n$, this means the matrix is different and so the inverse again needs to be done per step.

However, the original equation is only accurate to $\mathcal{O}(\Delta t)$ so an approximate solution to the step equation to the same order is sufficient. This can be done iteratively using two steps. The first step uses the explicit Euler to get the first iteration of u_{n+1}

$$u^{(0)}_{n+1} = u_n + f(t_n, u_n)\Delta t \tag{9.44}$$

and this is then used at t_{n+1} to get a second iteration

$$u^{(1)}_{n+1} = u_n + f(t_{n+1}, u^{(0)}_{n+1})\Delta t \tag{9.45}$$

For comparison with other methods, we can write the iterative procedure as

$$\boxed{\begin{aligned} f_a &= f(t_n, u_n), && \text{(9.46a)} \\ f_b &= f(t_{n+1}, u_n + f_a\Delta t), && \text{(9.46b)} \\ u_{n+1} &= u_n + f_b\Delta t && \text{(9.46c)} \end{aligned}}$$

which requires two evaluations per step, i.e. one more than explicit Euler. Note, all the terms on the RH sides of these equations are known and all u values come from t_n. This actually makes implicit Euler solved by this technique a single-step method.

The pattern of using a explicit method to get an approximate result and an implicit method to then update is widely used for all implicit methods. It is called **predictor-corrector**, as the explicit method **predicts** the value of u_{n+1} and the implicit method **corrects** it to the final value.

9.7.3 Second-order implicit methods

Just like we have seen more advanced explicit methods (than Euler), more advanced implicit methods exist. To get second-order accuracy, we have to include the u''_n term using extra information. As for AB2, we take the Taylor series for u'_n but now to evaluate u_{n+1} (rather than u_{n-1})

$$u'_{n+1} = u'_n + u''_n \Delta t \qquad \text{so} \qquad u''_n = \frac{u'_{n+1} - u'_n}{\Delta t} \tag{9.47}$$

Using this in the Taylor expansion for u_{n+1} to second-order gives

$$u_{n+1} = u_n + u'_n \Delta t + u''_n \frac{\Delta t^2}{2} = u_n + u'_n \Delta t + \frac{1}{2}(u'_{n+1} - u'_n)\Delta_t \tag{9.48}$$

which results in

$$\boxed{u_{n+1} = u_n + \frac{1}{2}[f(t_n, u_n) + f(t_{n+1}, u_{n+1})]\Delta t} \tag{9.49}$$

and so is seen to be using an average of the two derivatives at each end of the time step, with equal weights of 1/2. This is the **second-order Adams-Moulton (AM2)** method. It is an implicit multi-step method. This equation has to be solved for u_{n+1}; it is conceptually the second-order equivalent of the first-order Eq. (9.41).

As for implicit Euler, it is straightforward to solve for the linear case using matrices. For the non-linear case, it can again be solved iteratively using an explicit method as a predictor and then using a corrector, in the same way as for implicit Euler. The explicit predictor method should be chosen so it does not degrade the order of the implicit corrector method. Specifically, $f(t_{n+1}, u_{n+1})$ is multiplied by Δt in Eq. (9.49), so as long as the predictor method is one order less (or better) than the corrector method, it will still give a result to the same order as the corrector method. In this case, since the AM2 implicit method is second-order, this means a first-order explicit method, i.e. Euler, can be used, although any second-order explicit method such as AB2 would also work. If we use explicit Euler as the predictor, the procedure becomes

$$\boxed{\begin{aligned} f_a &= f(t_n, u_n), \\ f_b &= f(t_{n+1}, u_n + f_a \Delta t), \\ u_{n+1} &= u_n + \frac{1}{2}\,(f_a + f_b)\,\Delta t \end{aligned}} \tag{9.50a}$$

Higher-order implicit methods use values calculated in earlier steps to evaluate the higher derivatives. As seen previously, it is better to use the f_{n-1}, etc., values rather than the u_{n-1}, etc., values. For example, the **fourth-order Adams-Moulton** implicit multi-step method (AM4) is

$$\boxed{u_{n+1} = u_n + \frac{1}{24}\left[9f(t_{n+1}, u_{n+1}) + 19f(t_n, u_n) - 5f(t_{n-1}, u_{n-1}) + f(t_{n-2}, u_{n-2})\right]\Delta t} \tag{9.51}$$

where some third-order (or higher) method, such as AB3, should be used for the predictor.

9.8 Runge-Kutta Methods

When considered as a "black box", the Runge-Kutta methods are a family of explicit, single-step methods, i.e. they only require u_n as an input to find u_{n+1}.

To get to higher-order, the extra information needed is obtained by doing function evaluations *within* the overall time step. Hence, internally, the method generalises the multi-step idea of using m estimates of the gradient calculated at various points to find a weighted average of gradients by using points in the interval $t_n \le t \le t_{n+1}$.

In general the **RKm** method equates to an m^{th}-order Taylor expansion, and is an **m^{th}-order method**. A method of order m will require m gradient function evaluations. A number of weighting schemes for the averaging of these gradients can be used for each RKm.

9.8.1 Second-order Runge-Kutta

The AB2 and AM2 methods used the function in the previous or next step, respectively, in a Taylor series for u' to solve for u''_n. More generally we could take a step by some fraction α of Δt. For this

$$u'_{n+\alpha} = u'_n + u''_n \alpha \Delta t \qquad \text{so} \qquad u''_n = \frac{u'_{n+\alpha} - u'_n}{\alpha \Delta t} \tag{9.52}$$

Using this in the Taylor expansion for u_{n+1} to second-order gives

$$u_{n+1} = u_n + u'_n \Delta t + u''_n \frac{\Delta t^2}{2} = u_n + u'_n \Delta t + \frac{1}{2\alpha}(u'_{n+a} - u'_n)\Delta_t \tag{9.53}$$

so

$$\boxed{u_{n+1} = u_n + \frac{1}{2\alpha}[(2\alpha - 1)f(t_n, u_n) + f(t_{n+\alpha}, u_{n+\alpha})]\Delta t} \tag{9.54}$$

This obviously reproduces AB2 for $\alpha = -1$ and AM2 for $\alpha = 1$. However for other values of α, the above equation has two unknowns, u_{n+1} and $u_{n+\alpha}$, and so cannot be solved directly, even for the linear case. Hence, $u_{n+\alpha}$ must be estimated by a predictor-correction method first, and then used to solve for u_{n+1}. The RK2 method chooses a value $0 < \alpha < 1$ so it is within the step; $\alpha = 1/2$ or $3/4$ are common choices. The Euler method is used for the predictor, so the overall method is

$$\boxed{\begin{aligned} f_a &= f(t_n, u_n), && \text{(9.55a)} \\ f_b &= f(t_n + \alpha\,\Delta t, u_n + \alpha\,\Delta t\, f_a), && \text{(9.55b)} \\ u_{n+1} &= u_n + \frac{1}{2\alpha}\,[(2\alpha - 1)\,f_a + f_b]\,\Delta t && \text{(9.55c)} \end{aligned}}$$

The value of $u_{n+\alpha}$ is not retained as part of the solution but is discarded after the step calculation.

Because RK2 does all calculations within the step, this is regarded as an explicit, single-step method.

9.8.2 Fourth-order Runge-Kutta

The RK4 method uses four iterations of gradient estimation to calculate an average and is fourth-order accurate.

$$\boxed{\begin{aligned} f_a &= f(t_n, u_n), && \text{(9.56a)} \\ f_b &= f(t_n + \Delta t/2, u_n + f_a \Delta t/2), && \text{(9.56b)} \\ f_c &= f(t_n + \Delta t/2, u_n + f_b \Delta t/2), && \text{(9.56c)} \\ f_d &= f(t_n + \Delta t, u_n + f_c \Delta t) && \text{(9.56d)} \\ u_{n+1} &= u_n + \frac{1}{6}\,(f_a + 2\,f_b + 2\,f_c + f_d)\,\Delta t && \text{(9.56e)} \end{aligned}}$$

Again the first stage is an Euler estimate, as in RK2. The second and third stages estimate the gradient at $\Delta t/2$ using the best estimate of $u'_{n+1/2}$ from the previous stage in each case. The fourth stage estimates the gradient at Δt using the gradient function f evaluated at the best estimate yet of the true value of u_{n+1}, i.e. $u_n + f_c \Delta t$. The final value is a weighted average of the four estimates. The values of the weights and time shifts are careful chosen to match the Taylor series terms to $\mathcal{O}(\Delta t^4)$.

The RK4 method is a standard 'workhorse' for real numerical ODE solving. Even higher-order RK can be used but these require more (e.g. 11 for RK6) evaluations per step and are therefore slower. Only for $m < 4$ are $\leq m$ evaluations required per step. One practical advantage of RK methods over multi-step methods is that we do not need to store any previous steps; this makes coding it a little easier and avoids the starting issue seen with multi-step methods.

9.9 Variable step size

For real applications, varying the step size is a crucial tool to controlling errors when the function is changing rapidly. This is a complex area and a detailed discussion is beyond this course. With single-step methods, it is pretty easy to vary Δt during the calculation to, e.g., keep the error within a specified bound. Classic examples are the so-called 'RK45' methods, which produce a fifth-order function estimate. The adaptive stepping in this case is done either via step doubling, or by embedding lower-order Runge-Kutta formulae in higher-order ones. See Numerical Recipes for a detailed discussion.

Note that whilst implementing a dynamic step size is pretty easy in single-step algorithms, it is more difficult in multi-step methods, as this requires back-calculating missing intermediate values using a good single-step method.

9.10 Relationship to interpolation

For finite difference methods, we only have the solution at particular times. A smooth function will require interpolation. Interpolation between the discrete values was covered in Section (4). These techniques can be applied to the initial value solution to give a function with the required continuity properties.

However, these interpolation methods are based on only knowing the function values u_n at the discrete times. In contrast, for the solution of an ODE, we (at least in principle) know the derivatives as well. This is either because they are stored as part of the solution or because we can evaluate $f(t_n, u_n)$ for each point after solving, specifically for use in the interpolation. In general the standard interpolation techniques will give a smooth function, but the derivatives at each step edge will not be the same as $f(t_n, u_n)$ and hence are inconsistent with the ODE to some level. We can include the extra derivative information in the interpolation to avoid this issue.

9.10.1 Interpolation within a step

If we just look within a single step, we have one point at either end. For standard interpolation the only possibility is linear interpolation, which joins the two points by a straight line. If used over more than one step, this gives a continuous function but a discontinuous first derivative at the step times (and so all higher derivatives are not defined).

Including the two derivative values at either end, we have four pieces of information which allows us to solve for a cubic interpolation function

$$u(t) = u_n + (t - t_n)\, f_n + (t - t_n)^2 \frac{u''_n}{2!} + (t - t_n)^3 \frac{u'''_n}{3!} \tag{9.57}$$

which means

$$u'(t) = f_n + (t - t_n)\, u''_n + (t - t_n)^2 \frac{u'''_n}{2!} \tag{9.58}$$

These two expressions have to match to u_{n+1} and f_{n+1}, respectively, at $t = t_n + \Delta t$ so we have

$$u_{n+1} = u_n + f_n \Delta t + \frac{u''_n \Delta t^2}{2!} + \frac{u'''_n \Delta t^3}{3!} \tag{9.59}$$

$$f_{n+1} = f_n + u''_n \Delta t + \frac{u'''_n \Delta t^2}{2!} \tag{9.60}$$

The solution is

$$\frac{u''_n}{2!} = \frac{1}{\Delta t^2}\left[3(u_{n+1} - u_n) - (f_{n+1} + 2f_n)\Delta t\right] \tag{9.61}$$

$$\frac{u'''_n}{3!} = \frac{1}{\Delta t^3}\left[-2(u_{n+1} - u_n) + (f_{n+1} + f_n)\Delta t\right] \tag{9.62}$$

The cubic polynomial resulting from using these in Eq. (9.57) will now be consistent with the ODE. If used over more than one step, this gives a continuous function and first derivative at the step times but a discontinuous second derivative.

9.10.2 Overall interpolation

A good technique for standard interpolation over all the points is to use cubic splines which give a smooth curve. At the step edges, it in continuous in the function as well as the first and second derivatives, and only discontinuous in the third derivative. However, as stated above,generally the first derivative will not match the value from the ODE.

We already have the internal step interpolation above, which gives a first derivative which is continuous *and* matched to the ODE; this is often sufficient. However, if second derivative matching is required, then the same idea as the cubic splines can be applied. Matching the second derivatives at every point will require another coefficient in each cell, so this results in a quartic spline. This is a straightforward generalisation of the standard technique.

9.11 Relationship to integration

Section (5.2) described various methods for numerical integration of varying accuracy and complexity. These were of the form

$$I = \int_a^b f(x)\,dx = \sum_n \int_{x_n}^{x_n+h} f(x)\,dx \tag{9.63}$$

where the integral was split into multiple small steps, each of width h. We can change the variable names to be consistent with our use in the ODE section, and write one term of the sum as

$$\int_{t_n}^{t_{n+1}} f(t)\,dt \tag{9.64}$$

If we now define a variable $u(t)$ such that $u' = f(t)$, we have

$$\int_{t_n}^{t_{n+1}} f(t)\,dt = \int_{t_n}^{t_{n+1}} \frac{du}{dt}\,dt = \int_{u(t_n)}^{u(t_{n+1})} du = u_{n+1} - u_n \tag{9.65}$$

which also means

$$u_{n+1} = u_n + \int_{t_n}^{t_{n+1}} f(t)\,dt \tag{9.66}$$

Hence, calculating the integral is seen to be equivalent to solving the ODE $u' = f(t)$. This also implies that the average gradient over the step is

$$\overline{u}'_n = \frac{\int_{t_n}^{t_{n+1}} f(t)\,dt}{\Delta t} = \frac{\int_{t_n}^{t_{n+1}} u'(t)\,dt}{\Delta t} \tag{9.67}$$

which is clearly correct. Hence, for a given $f(t)$, the ODE solutions and integrals should be consistent, within the limitations of the numerical methods used.

Importantly, the integral methods are seen to be a special case for the ODEs, namely that the function $f(t)$ does not depend on u. For this special case, many of the ODE methods simplify and in fact several nominally different methods become identical.

Consider the explicit Euler method. This approximates the average gradient by the initial value. This means

$$\int_{t_n}^{t_{n+1}} f(t)\,dt \approx f_n \Delta t \tag{9.68}$$

which is equivalent to putting $f(t) = f_n$, i.e. the gradient is constant across the step and set to the value at the start of the step. This is the basic integral technique called a left Riemann sum. Similarly, implicit Euler is equivalent to setting $f(t) = f_{n+1}$ which is a right Riemann sum.

Moving to second-order methods, since $f(t)$ only, the second-order implicit method simplifies to

$$u_{n+1} = u_n + \frac{1}{2}\left(f_n + f_{n+1}\right)\Delta t \tag{9.69}$$

This is seen to correspond to the trapezoidal integration method. In addition, applying the leapfrog with half the step size gives

$$u_{n+1} = u_n + f\left(t_n + \frac{\Delta t}{2}\right)\Delta t = u_n + f\left(\frac{t_n + t_{n+1}}{2}\right)\Delta t \tag{9.70}$$

where the second expression follows as the time of the middle of the step is the average of the time at the two ends. This is the midpoint Riemann (open rule) sum integration method.

Finally, considering fourth-order ODE methods, the RK4 simplifies to

$$f_a = f(t_n), \tag{9.71}$$
$$f_{bc} = f(t_n + \Delta t/2), \tag{9.72}$$
$$f_d = f(t_n + \Delta t), \tag{9.73}$$
$$u_{n+1} = u_n + \frac{1}{6}\left(f_a + 4f_{bc} + f_d\right)\Delta t \tag{9.74}$$

Note, the middle two function evaluations are now the same so have been combined above into one. In this form, RK4 is identical to Simpson's rule as discussed in section (5.2.1) but with $h \to \Delta t/2$:

$$\int_{t_n}^{t_{n+1}} f(t)\,\mathrm{d}u = \frac{\Delta t}{2}\left[\frac{1}{3}f_n + \frac{4}{3}f_{n+\frac{1}{2}} + \frac{1}{3}f_{n+1}\right] + O(\Delta t^5) \tag{9.75}$$

In the end this is not so surprising when we think about the fact that RK4 and Simpson's rule are both designed from the start to be accurate up to $\mathcal{O}(\Delta t^4)$.

Chapter 10

Boundary Value ODE Methods

Outline of Section

- Boundary value methods
- Shooting methods
- Matrix methods
- Second-order matrix method
- Eigen-systems

10.1 Introduction

We have so far only looked at initial value problems, meaning that the constants of integration define values for the variables all at the same time t_0. However, for more than one variable, the variables can be defined at different times, which means we do not have any initial time where all variables are known which we can work from. We have to solve the region between the first and last specified variable times in a coherent way; these are called **boundary value problems**. There are in fact several ways of specifying the constants of integration. The above assumes there is a known value of each variable at some time (which can be different for each variable). However, we can instead define two known values for one variable and none for one of the others. For multiple variables there are many combinations in general.

In particular, we have seen that a second-order equation can be represented by two first-order equations for u and the velocity v. Consider the general, linear, second-order equation

$$\alpha \frac{\mathrm{d}^2 u}{\mathrm{d}t^2} + \beta \frac{\mathrm{d}u}{\mathrm{d}t} + \gamma\, u = k\,. \tag{10.1}$$

For an initial value problem, we would require initial conditions specifying the value of the solution at some initial point $t = t_0$, i.e. $u(t_0)$ and its derivative $u'(t_0)$, to integrate the system to a final point. The second value clearly is equivalent to setting $v(t_0)$ when working with two first-order equations. However, you will know that we can instead specify the start and end position, i.e. $u(t_0)$ and $u(t_N)$, where t_N is the end of the boundary time range. This means when we break the equation into two first-order equations, we have specified u twice and not specified v at all.

We should be able to find a solution whenever there are enough independent boundary conditions specified. The ways we do this are called **boundary value methods**. We will discuss two general way to do this; shooting methods and matrix methods.

10.2 Shooting methods

The first method we will look is the **shooting method**. This reuses a lot of the machinery we developed for initial value methods. In fact, the label 'shooting method' is an overall term for the general concept, while internally it uses whichever of the initial value methods we want.

To keep things simple, let's work with two variables $u(t)$ and $v(t)$ and assume we know u at the start time $u_0 = u(t_0)$ and v at the end time $v_N = v(t_N)$. Compared with initial value problems,

the issue is that we don't know $v_0 = v(t_0)$ and so cannot in general calculate $f_u(t_0, u_0, v_0)$ or $f_v(t_0, u_0, v_0)$ to take a first step. We therefore **guess** a value for v_0 which allows us to solve the problem as an initial value problem. We can use any of the initial value methods we like.

Of course, unless we got very lucky, we won't have guessed the correct value for v_0. Hence, when we get to the end of the time period, the $v(t_N)$ value we find will not be equal to the required value v_N. We could put in a lot of v_0 guesses and try to map out the dependence of the resulting $v(t_N)$ values to see which is close to v_N. This is why it is called a 'shooting' method; we keep shooting forward with an initial guess and then adjusting our aim by changing v_0 to get closer to the target.

However, rerunning the initial value method many times would usually be slow. We can be more systematic; we can consider the whole initial value method conceptually as a function which takes a value of v_0 and returns a value of $v(t_N)$, i.e. $v(t_N) = v_{\text{end}}(v_0)$. What we want is for this to be equal to the known value v_N, which we can write as

$$v_{\text{end}}(v_0) - v_N = 0 \tag{10.2}$$

We now see this is an algebraic root-finding problem; these are covered in section (6.7). We would not normally be able to calculate the derivatives of $v_{\text{end}}(v_0)$ analytically so we would use the bisection or secant method. The bisection method requires two values of $v(t_N)$ on either side of v_N which are not always easy to find, so the secant method is normally used. As always with root-finding, the closer the initial guess is to the true value, the quicker and more reliable the convergence. Hence some physics intuition is useful to pick an initial v_0.

Note the shooting method treats the boundary region asymmetrically, i.e. we start at t_0 and perform an initial value method which matches to v_N. We could have started at t_N taking a guess for u_N and doing our time step by $-\Delta t$ each time; this is called a "final value method" but these are essentially identical to the initial value methods. We should find that the two solutions would be the same within rounding errors.

10.2.1 Linear systems

Applying the shooting method to a linear system such as the one in equation (10.1) is easier than the general case. Any two guesses for v_0 will allow us to find the solution without any iteration. In fact, the solution is just what would be found from setting up the general secant method and taking one iteration; it would converge immediately.

We take two guesses v_{0a} and v_{0b} and solve for each using our favourite initial value method to get two end values $v_a(t_N)$ and $v_b(t_N)$. Since it is linear, $v_a(t_N)$ will depend linearly on v_{0a} and the same holds for the other solution. Also, because it is linear, a sum of two solutions is also a solution. We can sum the two solutions using a parameter c so the final values give the known value v_N by doing

$$cv_a(t_N) + (1-c)v_b(t_N) = v_N \tag{10.3}$$

so if the first guess was correct, we would have $c = 1$ and if the second guess was correct, we would have $c = 0$. The general case solution for c of the above equation is easily found to be

$$c = \frac{v_N - v_b(t_N)}{v_a(t_N) - v_b(t_N)} \tag{10.4}$$

We can now sum each step value in the two initial value solutions for v_{0a} and v_{0b} using c in the same way as above to find the boundary value solution. Alternatively, given we know c, we can find the correct v_0 using

$$v_0 = cv_{0a} + (1-c)v_{0b} \tag{10.5}$$

and rerun the initial value method; we should now get $v(t_N) = v_N$ within rounding errors.

10.2.2 Other boundary conditions

The case where u is known at u_0 and u_N, but v is not specified at all, can be handled in an effectively identical way. We again guess values of v_0 and the only difference is that we check on the value of $u(t_N)$ at the end of the initial value method, rather than $v(t_N)$. Hence our algebraic root-finding equation is

$$u_{\text{end}}(v_0) - u_N = 0 \tag{10.6}$$

where $u_{\text{end}}(v_0)$ is now (at least conceptually) a function that returns $u(t_N)$ from the initial value method.

The principle of the shooting method can be adapted to be used to solve more complicated problems; these can be boundary conditions that are defined at more than two different locations (which requires at least three variables) or where instabilities in the problem mean that you cannot shoot *into* a boundary, only *out* from it. For example, the three variable problem (e.g. for u, v and w) requires two guesses at the initial boundary (e.g. for v_0 and w_0) and we will want to match the v and w values where they are known (e.g. to $v(t_N)$ and $w(t_w)$ for an intermediate value of time t_w within the boundary time range). This is then a 2D root-finding problem but we will also need to interpolate in w near t_w as this will not in general sit exactly on a step edge.

10.3 Matrix methods

The shooting method is often the first choice for solving boundary value problems. However, another method is to consider all the points of the solution at the same time, by converting the differential equation into a **finite difference equation** which involves neighbouring points, and solving the resulting matrix equation. These are also called relaxation methods.

The concept of solving ODEs using matrices is applicable to both initial value and boundary value problems. As for the shooting method, the use of matrices is a general system and internally we can use any method per step that we like. For initial value problems, the result will be identical to that obtained using the same method in the previous way; it is just the same equations written in a different form. However, for boundary value problems, matrix methods can solve the whole boundary region in a coherent way. We will show how the matrix methods correspond to the initial value methods and then look at boundary value problems.

10.3.1 Initial value matrix methods

Consider the one variable case. We have seen that all initial value methods come down to approximating the average gradient $\overline{u}'$ in the relation

$$u_{n+1} = u_n + \overline{u}'_n \Delta t \tag{10.7}$$

The first few steps therefore can be written as

$$u_1 = u_0 + \overline{u}'_0 \Delta t, \qquad u_2 - u_1 = \overline{u}'_1 \Delta t, \qquad u_3 - u_2 = \overline{u}'_2 \Delta t, \qquad \ldots \tag{10.8}$$

where the LH sides have the values we want to solve for. Similarly the last few steps are

$$\ldots \qquad u_{N-2} - u_{N-3} = \overline{u}'_{N-3} \Delta t, \qquad u_{N-1} - u_{N-2} = \overline{u}'_{N-2} \Delta t, \qquad u_N - u_{N-1} = \overline{u}'_{N-1} \Delta t \tag{10.9}$$

These equations can be written in an $N \times N$ matrix form as

$$\begin{pmatrix} 1 & 0 & 0 & \dots & 0 & 0 & 0 \\ -1 & 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & -1 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 1 & 0 & 0 \\ 0 & 0 & 0 & \dots & -1 & 1 & 0 \\ 0 & 0 & 0 & \dots & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{pmatrix} = \begin{pmatrix} u_0 + \overline{u}'_0 \Delta t \\ \overline{u}'_1 \Delta t \\ \overline{u}'_2 \Delta t \\ \vdots \\ \overline{u}'_{N-3} \Delta t \\ \overline{u}'_{N-2} \Delta t \\ \overline{u}'_{N-1} \Delta t \end{pmatrix}, \tag{10.10}$$

The matrix is seen to be a lower diagonal matrix $\mathbf{L}$ so these equations are of the type $\mathbf{L} \cdot \vec{u} = \vec{b}$. Section (6.3.4) shows that such matrix equations can be solved by forward substitution. However, there is an issue in this case; the RH vector $\vec{b}$ depends on the values of u_i, as these are needed to calculate the $\overline{u}'_i$. This seems to make the set of equations much more difficult to solve.

A closer look at the RH vector shows that each element is the average of the gradient at an **earlier step** than the u_i values which are being solved for in the corresponding row equation (as long as we are not using an implicit method). This means whatever (explicit) method we use, we will have solved for all the u_i needed to calculate the next $\overline{u}'_i$ before it is used. Therefore, the above matrix equation can be solved by forward substitution, if we include updating the RH vector $\vec{b}$ by substituting the u_i values into each element in turn. If you look at the explicit equations for forward substitution, you would find this exactly reproduces the calculations of the initial value method.

This can be done for two or more variables; for the two variable case we have two matrix equations $\mathbf{L} \cdot \vec{u} = \vec{b}$ and $\mathbf{L} \cdot \vec{v} = \vec{c}$, where the lower diagonal matrix is identical in both cases. The RH vectors $\vec{b}$ and $\vec{c}$ contain the average gradients $\overline{u}'_i$ and $\overline{v}'_i$ respectively but in general these will both depend on u_i and v_i for coupled systems. Hence, we cannot solve for $\vec{u}$ first and $\vec{v}$ afterwards. A solution is possible but we must solve the first equation of both matrices first, use the resulting u_1 and v_1 values to calculate $\overline{u}'_1$ and $\overline{v}'_1$ so we can then solve the next row, and so on. Hence, the two forward substitution solutions must be calculated in parallel and again the resulting equations are identical to the equivalent initial value method.

10.3.2 Boundary value matrix methods

It is usually a lot more convenient to apply a standard initial value method to solve an initial value problem so the matrix method described above is not used in practice for these problems. However, matrix methods can be very useful for boundary value problems. Assume again that we know $u(t_0) = u_0$ and $v(t_N) = v_N$. The matrix form for the $\vec{u}$ equations will be identical to above; we want to solve for u_1 to u_N. However the ones for $\vec{v}$ will now be different; in this case we want to solve for the unknowns v_0 to v_{N-1}. If v was uncoupled and could be solved by itself, we would work backwards from v_N using $v_{n-1} = v_n - \overline{v}'_n \Delta t$. The last few steps can be written as

$$-v_{N-1} = -v_N + \overline{v}'_N \Delta t, \quad v_{N-1} - v_{N-2} = \overline{v}'_{N-1} \Delta t, \quad v_{N-2} - v_{N-3} = \overline{v}'_{N-2} \Delta t, \quad \dots \tag{10.11}$$

so the first few steps are

$$v_3 - v_2 = \overline{v}'_3 \Delta t, \qquad v_2 - v_1 = \overline{v}'_2 \Delta t, \qquad v_1 - v_0 = \overline{v}'_1 \Delta t, \qquad \dots \tag{10.12}$$

This corresponds to a matrix equation of the form

$$\begin{pmatrix} -1 & 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & -1 & 1 & \dots & 0 & 0 & 0 \\ 0 & 0 & -1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -1 & 1 & 0 \\ 0 & 0 & 0 & \dots & 0 & -1 & 1 \\ 0 & 0 & 0 & \dots & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ \vdots \\ v_{N-3} \\ v_{N-2} \\ v_{N-1} \end{pmatrix} = \begin{pmatrix} \overline{v}'_1 \Delta t \\ \overline{v}'_2 \Delta t \\ \overline{v}'_3 \Delta t \\ \vdots \\ \overline{v}'_{N-2} \Delta t \\ \overline{v}'_{N-1} \Delta t \\ -v_N + \overline{v}'_N \Delta t \end{pmatrix}, \tag{10.13}$$

This is seen to be an upper diagonal matrix $\mathbf{U}$ so these equations are now of the type $\mathbf{U} \cdot \vec{v} = \vec{c}$. In principle we would like to solve this using backward substitution but we have hit the same problem as before; we do not know the average gradient values. For $\vec{v}$ alone, we would work from the bottom. However, in the coupled boundary value problem we cannot calculate them as we go, as we need to solve $\vec{u}$ from the top but $\vec{v}$ from the bottom.

We therefore have to guess a solution and iterate. We need to guess values for the whole vector $\vec{u}^{(0)}$ and $\vec{v}^{(0)}$. This could be a rough solution from a shooting method, for example. We then pre-calculate all the average gradients $\overline{u}_i'^{(0)}$ and $\overline{v}_i'^{(0)}$ using these values, and then find the vectors $\vec{b}^{(0)}$ and $\vec{c}^{(0)}$. The matrix equations are then solved for $\vec{u}^{(1)}$ and $\vec{v}^{(1)}$, so $\vec{b}^{(1)}$ and $\vec{c}^{(1)}$ can be calculated, and the whole process iterated until it converges. (This can formally be considered a Newton-Raphson iteration with the approximation of dropping terms of $\mathcal{O}(\Delta t)$ in the derivative.) The solution assumes the RH vectors are constant when finding the next iteration of $\vec{u}$ and $\vec{v}$ so we can solve using standard forward and backward substitution. Alternatively, we can find the solution using $\vec{u}^{(i+1)} = \mathbf{L}^{-1} \cdot \vec{b}^{(i)}$ and $\vec{v}^{(i+1)} = \mathbf{U}^{-1} \cdot \vec{c}^{(i)}$. This is easy as these matrices have simple and known inverses; $\mathbf{L}^{-1}$ has all elements on and below the diagonal = 1 and all above the diagonal = 0, while $\mathbf{U}^{-1}$ has all elements on and above the diagonal = -1 and all below the diagonal = 0.

There are obviously special cases which greatly simplify the above. For example, if the average gradients only depend on time and not u or v then the equations can be solved in one go. If the equations are linear, the parts proportional to u and v in the average gradients can be moved to the LH side and incorporated into the matrix. This leaves only constant (or possibly time-dependent) parts in the RH side vector, so again the system can be solved in one step.

There is also one special case which is very important for physics applications, which we will look at in the next section.

10.4 Second-derivative matrix method

Consider the case of a classical particle moving in a simple potential, meaning one which does not depend on the particle velocity so $V(t, u)$. The particle acceleration is given by

$$\frac{\mathrm{d}^2 u}{\mathrm{d}t^2} = -\frac{1}{m}\frac{\partial V}{\partial u} = f(t, u) \tag{10.14}$$

We want to solve this second-order ODE as a boundary value problem where u is known at $u(t_0) = u_0$ and $u(t_N) = u_N$. We know we can separate this into two equations by using the velocity v

$$\frac{\mathrm{d}u}{\mathrm{d}t} = v \qquad \text{and} \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = f(t, u) \tag{10.15}$$

We can then construct matrix equations for $\vec{u}$ and $\vec{v}$ as above. There is a fiddly difference in this case; we know u_0 and u_N so there are only $N - 1$ unknowns for u_i, which are u_1 to u_{N-1}. In contrast, we have no information on v so all of v_0 to v_N are unknown, which is $N + 1$ unknowns. However, it turns out the value of v_N is never used in the equations (as it would be used to calculate u_N) so we can just consider v_0 to v_{N-1}; this is N unknowns.

The equations can be reduced to a very convenient form if we use the Euler method for (at least) the $u' = v$ equation. This means we use $u_{n+1} = u_n + v_n \Delta t$ and the corresponding matrix equation becomes $\mathbf{L} \cdot \vec{u} = \vec{v}\Delta t + \vec{u}_{0N}$, where the vector $\vec{u}_{0N}$ contains the boundary condition values. (Note, as there are different numbers of $\vec{u}$ and $\vec{v}$ unknowns, the matrix $\mathbf{L}$ is not square.) If we write the $\vec{v}$ equation as $\mathbf{U} \cdot \vec{v} = \vec{c}$ (where $\mathbf{U}$ is also not square) then we can do

$$\mathbf{U} \cdot \mathbf{L} \cdot \vec{u} = \mathbf{U} \cdot \vec{v}\Delta t + \mathbf{U} \cdot \vec{u}_{0N} = \vec{c}\Delta t + \mathbf{U} \cdot \vec{u}_{0N} \tag{10.16}$$

The product $\mathbf{A} = \mathbf{U} \cdot \mathbf{L}$ is a square matrix and has the form

$$\mathbf{A} = \begin{pmatrix} -2 & 1 & 0 & \dots & 0 & 0 & 0 \\ 1 & -2 & 1 & \dots & 0 & 0 & 0 \\ 0 & 1 & -2 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -2 & 1 & 0 \\ 0 & 0 & 0 & \dots & 1 & -2 & 1 \\ 0 & 0 & 0 & \dots & 0 & 1 & -2 \end{pmatrix} \tag{10.17}$$

Note, the elements of this matrix directly reflect the coefficients in the centre-difference approximation to a second derivative. For simplicity, if we use the Euler method for the v' equation also, then the average gradient $\overline{v}'_n = f(t_n, u_n)\Delta t = f_n \Delta t$. This means the combination $\vec{b} = \vec{c}\Delta t + \mathbf{U} \cdot \vec{u}_{0N}$ reduces to the simple form

$$\vec{b} = \begin{pmatrix} f_0 \Delta t^2 - u_0 \\ f_1 \Delta t^2 \\ f_2 \Delta t^2 \\ \vdots \\ f_{N-3} \Delta t^2 \\ f_{N-2} \Delta t^2 - u_N \end{pmatrix}, \tag{10.18}$$

We now have the matrix equations in the form $\mathbf{A} \cdot \vec{u} = \vec{b}$. Again, $\vec{b}$ depends on the values of u so we have to iterate. However note that as the potential does not depend on v, neither does $\vec{b}$ so we do not have to also solve for $\vec{v}$. Importantly, the inverted matrix $\mathbf{A}^{-1}$ is analytically known; $\det(\mathbf{A}) \times \mathbf{A}$ can be written in terms of integers and the determinant is also an integer. Given that we may want to solve for thousands of steps, this is avoids having to do the inversion numerically and so introduce computation error.

As before, there are special cases which mean the equations can be solved immediately. The linear case again allows the matrix to be modified by taking the linear terms from the RH to the LH side of the equation.

10.4.1 Derivative Boundary Conditions

On occasion, we know the derivative at one of the boundaries but not the initial condition for the variable. For example, considering the same system as in Eq. (10.14), we could have boundary conditions given by u'_0 and u_N. We can adapt the finite difference method by taking a central difference for the derivative at the boundary

$$u'_0 \approx \frac{u_1 - u_{-1}}{2\Delta t}, \tag{10.19}$$

and extending the system by one interval so that the first equation reads

$$u_{-1} - 2\,u_0 + u_1 = \Delta t^2\, f_0, \tag{10.20}$$

which, given (10.19), is

$$-2\,u_0 + 2u_1 = \Delta t^2 f_0 + 2\,\Delta t\,u_0'\,. \tag{10.21}$$

The system can then be written as N linear equations

$$\begin{pmatrix} -2 & 2 & 0 & 0 & .... & 0 & 0 \\ 1 & -2 & 1 & 0 & .... & 0 & 0 \\ & & & . & & & \\ & & & . & & & \\ & & & . & & & \\ 0 & 0 & 0 & 0 & .... & 1 & -2 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ . \\ . \\ . \\ u_{N-1} \end{pmatrix} = \begin{pmatrix} \Delta t^2\,f_0 + 2\,\Delta t\,u_0' \\ \Delta t^2\,f_1 \\ . \\ . \\ . \\ \Delta t^2\,f_{N-1} - u_N \end{pmatrix}. \tag{10.22}$$

Note, specifying the boundary conditions using just the derivatives at both boundaries, and neither function value, will not generally work as the overall integration constant may not be determined and so means the absolute value of u can be undefined.

10.5 Eigenvalue Problems

For the particular case where the differential equations are homogeneous and linear we can view the problem as an eigensystem. For euample, consider the wave equation

$$\frac{\mathrm{d}^2 u}{\mathrm{d}t^2} + k^2 u = 0, \tag{10.23}$$

with boundary conditions $u(0) = 0$ and $u(1) = 0$. This describes the vibrations on a string of length 1 which is fixed at the endpoints. The analytic general solution to this system is easily found to be

$$u(t) = A\,\sin(k\,t) + B\,\cos(k\,t)\,. \tag{10.24}$$

The boundarv conditions imply that $B = 0$ and that $k = n\,\pi$. The solution describes fundamental modes of vibrations on the string where $n = 1, 2, 3, \ldots$, etc.

This system (and more complicated ones in particular) can be solved using the previous method to give the eigensystem

$$\begin{pmatrix} -2 & 1 & 0 & 0 & .... & 0 & 0 \\ 1 & -2 & 1 & 0 & .... & 0 & 0 \\ & & & . & & & \\ & & & . & & & \\ & & & . & & & \\ 0 & 0 & 0 & 0 & .... & 1 & -2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ . \\ . \\ . \\ u_{N-1} \end{pmatrix} = -k^2\Delta t^2 \begin{pmatrix} u_1 \\ u_2 \\ . \\ . \\ . \\ u_{N-1} \end{pmatrix}, \tag{10.25}$$

i.e. the same matrix $\mathbf{A}$ is found and

$$\mathbf{A} \cdot \vec{u} = \lambda \vec{u} \tag{10.26}$$

where $\lambda = -k^2\Delta t^2$ are the eigenvalues of the system. One then finds the eigenvalues λ_i and eigenvectors $\vec{e}_i$ of $\mathbf{A}$ using a suitable numerical eigen-solver. These should be close numerical approximations to the eigenvalues and eigenfunctions of the original ODE eigen-problem. It is often useful to find just the eigenvector corresponding to the largest (or smallest) eigenvalues, as these define the fundamental modes of the system. The power method (section 6.6.1) can be used to readily find these.

Chapter 11

Properties of ODE methods

Outline of Section
- Efficiency and convergence
- Consistency
- Accuracy
- Stability

11.1 Introduction

We will now look at how to evaluate the properties of any particular method. We will discuss **accuracy**, **efficiency** and **convergence**.

Accuracy is obvious; it is a measure of how close the method solution is to the true solution.

Efficiency is basically a measure of how much computational power the method needs. A method which takes weeks to calculate a solution is usually not feasible. A significant consideration when selecting a method is therefore the number of computations required for each step. Typically, the step computation is dominated by evaluating the derivative function $f(t, u)$, which can be complicated or even itself a numerical calculation. The Euler method requires one functional evaluation for each step and so is efficient in terms of CPU usage. As we have seen, higher-order methods usually require more evaluations at each step but will in general be more accurate. However, as they are more accurate, higher-order methods can work well with bigger step sizes and hence a smaller number of steps. A compromise must be reached between efficiency and accuracy for any particular system being solved.

Convergence means that the numerical solution will converge to the true solution $\tilde{u}_n$ in the limit of zero step size $\Delta t \to 0$ (in the absence of rounding errors). If a finite difference method is both **consistent** with the ODE being solved and the errors in the method are **stable**, then it will converge.

The properties of **consistency**, **accuracy** and **stability** are discussed in more detail below.

11.2 Consistency

This is a check that the finite difference method (e.g. the Euler method) reduces to the correct differential equation (i.e. $u' = f(t, u)$) in the limit of vanishingly small step size $\Delta t \to 0$. Moreover, a consistency analysis of a finite difference equation reveals the actual differential equation that the finite difference method is effectively solving; the **modified differential equation** (MDE).

Any finite difference method only gives us values of u at the step times, i.e. u_n. These values are not the same as the true solution of the equation being solved, due to inaccuracies. However, in the absence of rounding errors, the calculated values u_n lie on the solution of a *different* differential equation, which is the MDE. This means we can consider a continuous function $U(t)$ which agrees exactly with the method results at all the step times t_n, i.e. $u_n = U(t_n)$ and which obeys the MDE.

To find the MDE, we can do a Taylor expansion in terms of $U(t)$ which by definition should give the same value of u_n and u_{n+1} as the method, i.e. as given by the method average gradient

estimate

$$u_{n+1} = u_n + \overline{u}'^e_n \, \Delta t \tag{11.1}$$

For $U(t)$, we need to start the expansion from the same point, i.e. $U(t_n) = u_n$ so we also have

$$u_{n+1} = u_n + U'_n \, \Delta t + \frac{U''_n}{2!} \Delta t^2 + \frac{U'''_n}{3!} \Delta t^3 + \dots \tag{11.2}$$

The method average gradient estimate in terms of U is $\overline{U}'^e$ so therefore we require $U(t)$ to satisfy the (continuous) equation

$$\overline{U}'^e = U' + \frac{U''}{2!} \Delta t + \frac{U'''}{3!} \Delta t^2 + \dots \tag{11.3}$$

This can be re-expressed as

$$\boxed{\frac{dU}{dt} - f(t,U) = \overline{U}'^e - f(t,U) - \frac{U''}{2!} \Delta t - \frac{U'''}{3!} \Delta t^2 - \dots} \tag{11.4}$$

This gives the MDE for any method. This expression is very convenient as the true equation would give zero on the LH side by definition, so all the terms of the RH side arise only in the MDE. For the example of the Euler method, we approximate $\overline{u}'^e = f(t,u)$, so

$$\frac{dU}{dt} - f(t,U) = -\frac{U''}{2!} \Delta t - \frac{U'''}{3!} \Delta t^2 - \dots \tag{11.5}$$

is the specific MDE for the Euler method. It is now obvious that as $\Delta t \to 0$, all the RHS terms do go to zero and hence the Euler method is consistent. All other methods discussed in the previous chapter are also consistent.

A very useful property of the MDE is that it also allows the order of the method to be deduced if it is not known. The order of the method is the same as the lowest power of Δt appearing in the MDE; for Euler this is $(\Delta t)^1$, showing it is indeed a first-order method.

Another way to understand consistency is by rearranging Eq. (11.3) to give

$$U' - \overline{U}'^e = -\frac{U''}{2!} \Delta t - \frac{U'''}{3!} \Delta t^2 - \dots \tag{11.6}$$

Since $u' = f(t,u)$ and in the limit of $\Delta t \to 0$, the RHS goes to zero, this means any consistent method must satisfy

$$\boxed{\overline{u}'^e \to f(t,u)} \tag{11.7}$$

Hence, for any method, the estimated average gradient should become the true gradient when the step shrinks to zero width. This is obviously true of the Euler method as $\overline{u}'^e = f(t,u)$ for this method, even with non-zero Δt. All the methods we will study have this property, due to being weighted sums of gradient estimates, with $\sum_a w_a = 1$ as described in Eq. (9.23). Since all the arguments of the function evaluations are within the step(s), they all go to t_n and u_n in the limit of $\Delta t \to 0$. Hence, the sum becomes

$$\sum_a w_a \, f(t_n + \delta t_a, u_n + \delta u_a) \to \sum_a w_a \, f(t_n, u_n) = f(t_n, u_n) \sum_a w_a = f(t_n, u_n) \tag{11.8}$$

This means the fact that the weights sum to 1 ensures that the methods are consistent.

Consistency analysis can be carried out with finite difference methods for partial differential equations too, as discussed in Section (12).

11.3 Accuracy

To determine the accuracy of a method we want to find the **local** or **step** error (the error incurred in each step) and use it to estimate the **global error** (the error at the end of the calculation). One contribution to the local error is the size of the timestep.

11.3.1 Timestep size

All the ODE methods are based on a Taylor expansion. Hence, the most basic requirement for accuracy is that Δt must be "small" in some sense; the issue is small compared with what?

Any ODE must have some timescale, often denoted by τ, in the physical system to allow the equation to be correct in terms of dimensions. This can enter through parameters in the function $f(t, u)$ or through initial conditions, etc. E.g. for a harmonic oscillator for which the angular frequency ω is a parameter, the timescale of things happening is clearly of the order of the period so $\tau \sim \mathcal{O}(1/\omega)$. The timestep should in general be small compared with the physical timescale. E.g. for the oscillator, this means we should use a Δt which is $\Delta t \ll 1/\omega$.

Scaling our independent variables to make them dimensionless is a clean way to do this. The scale factor would be τ so we would have the scaled time variable being

$$\hat{t} = \frac{t}{\tau} \tag{11.9}$$

and so $\Delta t \ll \tau$ means in the scale time variable $\Delta\hat{t} \ll 1$ is needed. This is another reason why scaling variables is useful.

However, it can be the case that there is more than one timescale in a system. E.g. an oscillator which is being driven by a force being applied with a higher angular frequency of 1000ω. Clearly choosing a $\Delta t \sim 0.01/\omega$, which would be reasonable for the unforced oscillator, will miss all the detail of the forcing. Hence it is important to set Δt to be much less than the smallest timescale of the problem (although this can sometimes be overcome with the appropriate choice of implicit integrator). This is an important point for PDEs, as shown in chapter (12).

11.3.2 Local errors

The local error can be split into three contributions; the **truncation** (or **approximation**) error, the **rounding** (or **computational**) error and the **propagation** (or **amplification**) error.

Using the Euler method as an example we know that the local truncation error is $\mathcal{O}(\Delta t^2)$, as shown by equation (9.13). This means that if we have the exact value of u_n, then $\tilde{u}_{n+1}$, the **true** value of the function at t_{n+1}, is related to the numerical approximation of the function by

$$u_{n+1} \equiv u_n + f(t_n, u_n)\Delta t = \tilde{u}_{n+1} + \mathcal{O}(\Delta t^2), \tag{11.10}$$

Equation (11.10) shows us the approximation error q incurred in performing one step of the Euler method; $q \propto \Delta t^2$. Note the error here is not statistical but is a defined value; in principle we could calculate the missing terms. Higher-order methods by definition have truncation errors with higher powers of Δt; we have seen methods up to fourth-order which retain terms of Δt^4 and so have truncation errors of $q \propto \Delta t^5$.

As discussed in section (2.1.2), real numbers must in general be rounded to the nearest representable value. Even with the exact value of u_n, the calculation involved in determining u_{n+1} means the result will have a rounding error. We will write this as ru_{n+1} where the value of r depends on the method but is typically dominated by the calculations of the derivative function $f(t, u)$. If this is an analytic function, the factor would be $\mathcal{O}(10^{-8})$ or $\mathcal{O}(10^{-16})$ for 32-bit or 64-bit real numbers, respectively, depending on the function complexity. Functions which are the result of numerical calculations can be significantly less accurate.

The propagation error arises because in reality u_n is not known exactly; it has an error from the previous steps. The inexact value of u_n is used in the method to calculate u_{n+1} so the error on u_n propagates through the method calculations and contributes to the error on u_{n+1}. This is quantified by the amplification factor g which is calculated using the usual propagation of errors formula. We generally consider $u'^e_n(t_n, \Delta t, u_n)$ (plus other u values if a multi-step method). For this

$$\epsilon_{n+1} \approx \left(1 + \left.\frac{\partial u'^e}{\partial u}\right|_n \Delta t\right)\epsilon_n = g\epsilon_n \tag{11.11}$$

For Euler, this is

$$\epsilon_{n+1} \approx \left(1 + \left.\frac{\partial f}{\partial u}\right|_n \Delta t\right) \epsilon_n \qquad \text{i.e.} \qquad g = 1 + \left.\frac{\partial f}{\partial u}\right|_n \Delta t \tag{11.12}$$

Other methods have different dependences on u_n and indeed can use other values such as u_{n-1}; all these would have errors and so would all contribute to the propagation error in general.

The values of q, r and g will in general depend on t and u and hence can be different for each step. Hence, we will write q_n and g_n as these must be calculated using the values of u_n which is input to the step. However, we write r_{n+1} as it is the rounding error on the step output, i.e. u_{n+1}. Hence, we write the total error on u_{n+1} due to the combination of the three sources as

$$\epsilon_{n+1} = q_n + r_{n+1}u_{n+1} + g_n\epsilon_n \tag{11.13}$$

11.3.3 Errors after many steps

Given the local errors, we can start with the initial condition u_0 and in principle work out the error on any later step. The initial condition will only have a rounding error $\epsilon_0 = r_0u_0$. The next step will therefore have an error

$$\epsilon_1 = q_0 + r_1u_1 + g_0\epsilon_0 = q_0 + r_1u_1 + g_0r_0u_0 \tag{11.14}$$

and so

$$\epsilon_2 = q_1 + r_2u_2 + g_1\epsilon_1 = q_1 + g_1q_0 + r_2u_2 + g_1r_1u_1 + g_1g_0r_0u_0 \tag{11.15}$$

and so on. All previous errors are multiplied by g_i factors so if $g_i > 1$, the error can get large very quickly. This should be avoided if possible.

In general, calculating the error is complicated, but we can look at a simple example with the Euler method, which illustrates most of the important effects. Consider the linear case of $u' = au$, i.e. $f(t, u) = au$ with an initial condition $u(t = 0) = u_0 = 1$. This has an analytic solution $u = u_0e^{at}$. Because we know the analytic solution, we can calculate the sizes of all the errors (which is not usually possible, of course). The Euler method step is then

$$u_{n+1} = u_n + (a\Delta t)u_n = (1 + a\Delta t)u_n \tag{11.16}$$

Since $\partial f/\partial u = a$, we also have $g_n = 1 + a\Delta t$ and so in this case, g is the same for every step and does not need an index. It also means $u_{n+1} = gu_n$ and hence $u_n = g^nu_0$. (This reflects the fact that $(1 + a\Delta t)^n \approx e^{an\Delta t} \approx e^{at}$, the exact solution.)

The analytic solution means the exact formula for each step is

$$u_{n+1} = e^{a\Delta t}u_n = \left(1 + a\Delta t + a^2\frac{\Delta t^2}{2!} + a^3\frac{\Delta t^3}{3!} + \dots\right) u_n \tag{11.17}$$

We can now see the approximation error, i.e. the difference of the Euler step and the exact step, is

$$q_n = (1 + a\Delta t - e^{a\Delta t})u_n = -\left(\frac{a^2\Delta t^2}{2!} + \frac{a^3\Delta t^3}{3!} + \dots\right) u_n \approx -\frac{a^2\Delta t^2}{2}u_n \tag{11.18}$$

for small Δt, and so $q_n \propto \Delta t^2$ as expected. The above formulae for the errors of the first few steps simplify to

$$\epsilon_1 = \left[-\frac{a^2\Delta t^2}{2} + g(r_1 + r_0)\right] u_0 \qquad \text{and} \qquad \epsilon_2 = \left[2g\left(-\frac{a^2\Delta t^2}{2}\right) + g^2(r_2 + r_1 + r_0)\right] u_0 \tag{11.19}$$

Hence, the general case of the error on u_n is approximately

$$\epsilon_n \approx \left[n\left(-\frac{a^2\Delta t^2}{2}\right) + \sum_{i=0}^{n} r_i\right] g^n u_0 \tag{11.20}$$

The g^n means the error will rise exponentially with the step number n if $a > 0$, as then $g > 1$. This is the condition for a method being called **unstable**, as discussed below. However, this does *not* mean the method is unusable. For this example, since $g^n u_0 = u_n$ then the relative (or fractional) error is

$$\frac{\epsilon_n}{u_n} \approx n\left(-\frac{a^2\Delta t^2}{2}\right) + \sum_{i=0}^{n} r_i \tag{11.21}$$

which only increases linearly with step number. Specifically, the solution is also rising exponentially (as it is increasing by the same factor g) so the relative error is reasonable. Hence, if the relative, not absolute, error is important, then nominally unstable methods will often be acceptable as long as g is close to 1, which is the case here for small Δt. Note conversely that for $a < 0$, for which $g < 1$ and is called the **stable** case, the error would decrease exponentially. This is of course beneficial although the solution also decreases the same way, so the relative error would still be given by the above equation and so hence would increase as before.

Let's consider the two terms in the relative error in more detail. The first term arises from the approximation error and this is approximately just n times the dropped terms from the Taylor series. The sum is simply over the values of the individual dropped terms.

The sum of the rounding errors is more complicated as they act like statistical errors; some r_i will be positive and some negative, and some might actually be zero if the true value happens to be representable exactly. The sum will give an unpredictable value but with an expected spread which will scale as n^p, where $p = 1/2$ for completely uncorrelated rounding errors, and $p = 1$ for completely correlated errors. Hence the sum can be written as $n^p\sigma_r$. For a 32-bit or 64-bit floating point representation, we expect a width of $\sigma_r \sim 3 \times 10^{-8}$ or 6×10^{-17}, respectively. Putting in the above to the relative error formula results in

$$\frac{\epsilon_n}{u_n} \approx n\left(-\frac{a^2\Delta t^2}{2}\right) + n^p\sigma_r \tag{11.22}$$

Clearly, both contributions to the relative error increase with n as would be expected as each step adds more errors. Generally one of the two terms dominates depending on the relative size of $a^2\Delta t^2$ and σ_r. For larger $a\Delta t$, the approximation error will dominate and for small $a\Delta t$, the rounding error will dominate. Figure (11.1) shows the relative errors calculated using this Euler example compared with the functional forms given above.

11.3.4 Global errors

The previous subsection considered the errors as a function of the number of steps. In general, we need to solve ODEs over a specific time period, e.g. $t = 0$ to T. We need to consider how the error depends on the choice of the number of steps N used to cover the total time range or, since $T = N\Delta t$, equivalently the choice of the size of Δt.

We will work in terms of Δt, so the error on the solution at the end of the time period will be given by the previous expressions if we set $n = N = T/\Delta t$. For the relative error, this gives

$$\frac{\epsilon_n}{u_n} \approx \frac{T}{\Delta t}\left(-\frac{a^2\Delta t^2}{2}\right) + \left(\frac{T}{\Delta t}\right)^p \sigma_r \approx aT\left(-\frac{a\Delta t}{2}\right) + T^p\left(\frac{1}{\Delta t^p}\right)\sigma_r \tag{11.23}$$

It is seen that the approximation term increases with Δt while the rounding error term decreases with Δt. This is straightforward to understand; the approximation term clearly gets smaller if Δt is reduced, while the rounding error is approximately the same per step so the more steps that are taken (i.e. the larger N and hence smaller Δt), the bigger the rounding error will be. There is a minimum when the two error terms have approximately equal magnitudes

$$aT\frac{a\Delta t}{2} \sim T^p\left(\frac{1}{\Delta t^p}\right)\sigma_r \qquad \text{so} \qquad \Delta t^{1+p} \sim \frac{2\sigma_r}{a^2}T^{p-1} \tag{11.24}$$

Figure (11.2) shows the relative errors found for both 32-bit and 64-bit representations for this case with $a = 1$ and $T = 5$. The resulting relative error when using 32-bit precision is around 10^{-3}, while a relative error around 10^{-7} can be obtained with 64-bit precision.

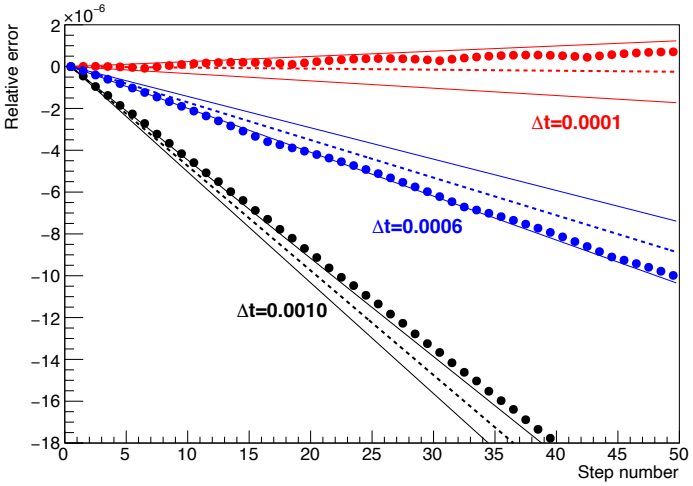

Figure 11.1: *Errors found using the Euler method to solve $f(t,u) = au$ with $a = 1$ as a function of the step number. Results for $\Delta t = 0.0001$, 0.0006 and 0.0010 are shown and the Euler calculation is done using 32-bit precision. For each value of Δt, the circles shown the relative error of the method found by subtracting the exact result, the dashed line shows the expected relative error from the approximation error alone, and the solid lines show the expected spread around the dashed line due to the width of the rounding error distribution, for which a width value of $\sigma_r = 3 \times 10^{-8}$ is used.*

It is clear the $(-a^2\Delta t^2/2)$ term in the above equations comes from the first truncated term of the original Taylor series. Higher-order methods will have different terms; a second-order method will have a truncated term $\mathcal{O}(\Delta t^3)$ and for fourth-order, it would be $\mathcal{O}(\Delta t^5)$. Changing the order of the method changes the optimal choice of Δt and much better accuracy is possible than with Euler, with many fewer steps. Figure (11.3) shows the same Euler values with the 64-bit representation as in figure (11.2) but also with values obtained from several HOT methods. The HOT4 method can reach a global relative error $\sim 10^{-13}$ in this case.

Generally, we have seen the local truncation error at each of the steps simply adds up so we will have

$$\boxed{\textbf{Global truncation error} \propto \text{Local truncation error} \times \text{Number of steps}\,.} \tag{11.25}$$

For a fixed period T, we have seen the number of steps is $\propto 1/\Delta t$. A $\boldsymbol{m^{\text{th}}}$**-order** method keeps terms up to $\boldsymbol{\mathcal{O}(\Delta t^m)}$ in each step, so its **local truncation error** is of $\boldsymbol{\mathcal{O}(\Delta t^{m+1})}$ and hence its **global truncation error** is of $\boldsymbol{\mathcal{O}(\Delta t^m)}$. Therefore, the order of the method also corresponds to the order of the global truncation error.

11.4 Stability

Stability is a important property of a method and describes how an error increases as the finite difference equation is iterated. If the error increases catastrophically with the number of steps, then the solution can become meaningless. However, a increasing error does not always mean the method is unusable if the solution is also rising at around the same rate as the fractional (or relative) error might be well controlled. Conversely, a decreasing error does not mean the method is always usable; it can sometimes lead to unphysical solutions. Hence, stability or instability is a rather coarse categorisation and there are many subtleties.

There are two different types of instability we will look at. One type occurs in a particular set of higher order methods and occurs even for small Δt. The other type arises in many methods

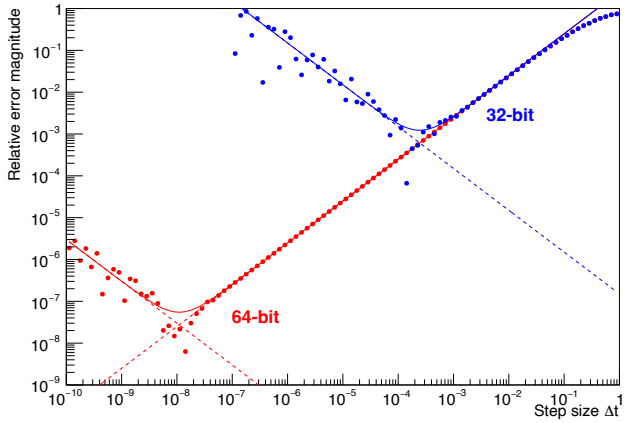

Figure 11.2: *Global errors found using the Euler method to solve $f(t,u) = au$ with $a = 1$ over a time period $T = 5$ as a function of the step size Δt. Results for 32-bit (blue) and 64-bit (red) representations are shown. For each, the circles shown the magnitude of the global relative error of the method found by subtracting the exact result, the dashed lines shows the expected relative error from the truncation and rounding errors (using $p = 1$) separately, and the solid curves show the expectation for the total error.*

when too large a step Δt is used. In both cases, an **unstable** method means the magnitude of the error on u_{n+1} is bigger than the magnitude of the error on u_n. Imagine that the numerical solution at step n has an error ϵ_n

$$u_n = \tilde{u}_n + \epsilon_n, \tag{11.26}$$

where $\tilde{u}_n$ is the true solution of the ODE at $t = t_n$ and u_n is the numerical approximation. In taking one step of a finite difference method the numerical approximate solution and the error

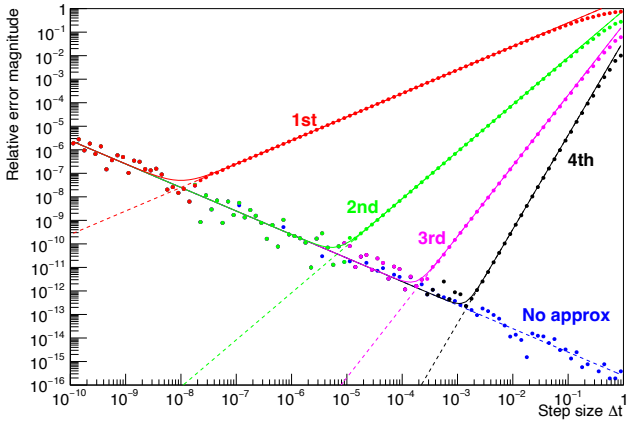

Figure 11.3: *Global errors found using the Euler and several HOT methods to solve $f(t,u) = au$ with $a = 1$ over a time period $T = 5$ as a function of the step size Δt. Results for 64-bit representations are shown. For each, the circles shown the magnitude of the global relative error of the method found by subtracting the exact result, the dashed lines shows the expected relative error from the approximation and rounding errors separately, and the solid curves show the expectation for the total error. Also shown are points for the analytic step solution (effectively to infinite order) which has no approximation errors. This is a 'cheat' and is only possible as we know the analytic solution of the sum of all the terms.*

change and satisfy

$$u_{n+1} = \tilde{u}_{n+1} + \epsilon_{n+1}. \tag{11.27}$$

A method is defined as stable if the error **amplification factor** g_n satisfies

$$\boxed{|g_n| \equiv \left| \frac{\epsilon_{n+1}}{\epsilon_n} \right| \leq 1,} \tag{11.28}$$

Eq. (11.20) shows the most important error for stability is the propagation error as it increases exponentially while the effect of the truncation and random errors increases at worst linearly. Hence we will consider only the propagation error here. In terms of the estimate of the average gradient u'^e_n, the step is

$$u_{n+1} = u_n + \overline{u}'^e_n \Delta t \tag{11.29}$$

In general, $\overline{u}'^e_n(t, \Delta t, u_n, u_{n-1}, \dots)$ where the u_{n-1} and other values covers the multi-step methods. Hence, using propagation of errors

$$\epsilon_{n+1} = \epsilon_n + \sum_{i=0} \frac{\partial \overline{u}'^e_n}{\partial u_{n-i}} \Delta t \ \epsilon_{n-i} \tag{11.30}$$

where the sum is over all the steps involved if a multi-step method. For example, the Euler method has $\overline{u}'^e_n = f(t, u_n)$ so

$$\epsilon_{n+1} = \left(1 + \frac{\partial f_n}{\partial u_n} \Delta t\right) \epsilon_n \qquad \text{i.e.} \qquad g_n = 1 + \frac{\partial f_n}{\partial u_n} \Delta t \tag{11.31}$$

and so it is seen that the solution will be unstable if $\partial f_n / \partial u_n \, \Delta t > 0$ or < -2. In particular, for the linear case where $f(t, u) = au$ then $\partial f_n / \partial u_n = a$ and the relevant equations become

$$u_{n+1} = (1 + a\Delta t) u_n \qquad \text{and} \qquad \epsilon_{n+1} = (1 + a\Delta t)\epsilon_n \tag{11.32}$$

Hence the update factor for u_n is identical to the error amplification factor for ϵ_n. This is why the fractional error scales in the same way as the solution for this system, as seen in Eq. (11.21). Note, stability requires choosing a Δt which satisfies $-2 < a\Delta t < 0$.

The above is for a single variable. The concept of stability is also applied to multi-variable problems. A more general stability analysis can be carried out for N coupled, linear first-order ODEs by looking at the general matrix operator form for the method

$$\vec{u}_{n+1} = \mathbf{G} \cdot \vec{u}_n, \tag{11.33}$$

where $\mathbf{G}$ is the update matrix (e.g. $\mathbf{G} = \mathbf{I} + \mathbf{A}\Delta t$ for the Euler method). This can be written as

$$u_{n+1\,i} = \sum_j G_{ij} u_{n\,j} \qquad \text{so} \qquad \frac{\partial u_{n+1\,i}}{u_{n\,j}} = G_{ij} \tag{11.34}$$

and hence

$$\epsilon_{n+1\,i} = \sum_j G_{ij} \epsilon_{n\,j} \qquad \text{i.e.} \qquad \vec{\epsilon}_{n+1} = \mathbf{G} \cdot \vec{\epsilon}_n. \tag{11.35}$$

It turns out that condition for stability is that the modulus of all the eigenvalues λ_i of the update matrix $\mathbf{G}$ must be less than or equal to unity

$$\boxed{|\lambda_i| \leq 1 \quad \text{for all } i} \tag{11.36}$$

This means the eigenvalue with the largest magnitude must be less than 1. Note, the modulus includes the case of complex conjugate pairs of eigenvalues. This is identical to the convergence criterion found when solving linear algebraic equations using the Jacobi or Gauss-Seidel methods as described in sections (6.5.1) and (6.5.2) and the same techniques for find the largest eigenvalue can be applied here.

To show condition (11.36) it is useful to diagonalise the matrix equation for error propagation so that we are left with m *decoupled* equations which we can deal with individually. Any non-singular $m \times m$ matrix that has m linearly independent eigenvectors can be diagonalised

$$\boxed{\mathbf{G} = \mathbf{R} \cdot \widetilde{\mathbf{G}} \cdot \mathbf{R}^{-1} \quad \text{with} \quad \widetilde{\mathbf{G}} \equiv \text{diag}(\lambda_0, \lambda_1, ...),} \tag{11.37}$$

where λ_i are the eigenvalues of $\mathbf{G}$ and $\mathbf{R}$ is the matrix whose columns are the eigenvectors of $\mathbf{G}$. Substituting equation (11.37) into (11.35) and operating on both sides with a further $\mathbf{R}^{-1}$ we obtain the diagonal (uncoupled) system

$$\vec{\zeta}_{n+1} = \widetilde{\mathbf{G}} \cdot \vec{\zeta}_n, \tag{11.38}$$

where we have defined the linear transformation $\vec{\zeta}_n = \mathbf{R}^{-1} \cdot \vec{\epsilon}_n$ which rotates the coupled vectors onto an uncoupled basis. This reduces to m uncoupled equations for the rotated errors ζ_n^i

$$\zeta_{n+1}^i = \lambda_i \, \zeta_n^i \, . \tag{11.39}$$

Since these are uncoupled we can now impose the constraint on their growth separately for each of them, i.e., for all $i = 0, 1, \ldots, m-1$ we have the requirement;

$$g_n^i = \left| \frac{\zeta_{n+1}^i}{\zeta_n^i} \right| = |\lambda_{i\,n}| \leq 1, \tag{11.40}$$

for a stable method. Since the coupled and uncoupled basis are related by a linear transformation any constraint applied in one basis is equivalent to one applied in the other. That is, the condition in equation (11.40) holds for the original equation (11.35) too.

For a non-linear set of equations, the propagation of errors linearises the error propagation; specifically (for a single-step method, for simplicity)

$$\epsilon_{n+1\,i} = \epsilon_{n\,i} + \sum_j \frac{\partial u'^e_{n\,i}}{\partial u_{n\,j}} \Delta t \; \epsilon_{n\,j} \qquad \text{i.e.} \qquad \vec{\epsilon}_{n+1} = (\mathbf{I} + \mathbf{J}_n \Delta t) \cdot \vec{\epsilon}_n. \tag{11.41}$$

where matrix $\mathbf{J}_n$ is the Jacobian for the derivatives of the estimate of the average gradient with respect to the $\vec{u}_n$. Clearly, now $\mathbf{G}_n = \mathbf{I} + \mathbf{J}_n \Delta t$ and so changes with each step.

11.5 Stability for small step sizes

All methods are based on the basic Taylor expansion

$$u_{n+1} = u_n + u'_n \Delta t + u''_n \frac{\Delta t^2}{2!} + u'''_n \frac{\Delta t^3}{3!} + \ldots \tag{11.42}$$

and for the linear case $f = au$, this gives

$$u_{n+1} = \left(1 + a\Delta t + a^2 \frac{\Delta t^2}{2!} + a^3 \frac{\Delta t^3}{3!} + \ldots \right) u_n = g u_n \tag{11.43}$$

Assuming a is real, this makes it clear that for small $a\Delta t$, where the second term dominates over the rest of the series, the solution is supposedly unstable for $a > 0$ and stable for $a < 0$. This holds for methods of any order. However, as we saw in section (11.3.3), we can get highly accurate fractional errors for both cases, even when the system is nominally unstable. Note, after n steps

$$g^n \approx \left(1 + a\Delta t + a^2 \frac{\Delta t^2}{2!} + a^3 \frac{\Delta t^3}{3!} + \ldots \right)^n \approx e^{na\Delta t} \approx e^{aT} \tag{11.44}$$

for a total period $T = n\Delta t$. Hence changing the step size Δt does not change the global propagation error in this case.

For the case of an oscillating system where we can consider $a = i\omega$, the magnitude of the oscillation amplitude error can be considered to grow as

$$|g| = \left|1 + i\omega\Delta t - \omega^2\frac{\Delta t^2}{2!} - i\omega^3\frac{\Delta t^3}{3!} + \dots\right| \tag{11.45}$$

For a first order method then

$$|g|^2 = 1 + \omega^2\Delta t^2 \qquad \text{so} \qquad |g| = \sqrt{1+\omega^2\Delta t^2} \approx 1 + \frac{1}{2}\omega^2\Delta t^2 \tag{11.46}$$

for small $\omega\Delta t$. Hence, all first order methods are nominally unstable for oscillating solutions but only $\propto \Delta t^2$, not Δt. Specifically

$$|g|^n \approx \left(1 + \frac{1}{2}\omega^2\Delta t^2\right)^n \approx e^{n\omega^2\Delta t^2/2} \approx e^{T\omega^2\Delta t/2} \tag{11.47}$$

It this case, reducing Δt *does* help reduce the size of the errors and so this can help keep the instability under control. For a second-order method

$$|g|^2 = \left(1 - \omega^2\frac{\Delta t^2}{2}\right)^2 + \omega^2\Delta t^2 = 1 + \frac{1}{4}\omega^4\Delta t^4 \qquad \text{so} \qquad |g| = \sqrt{1+\frac{1}{4}\omega^4\Delta t^4} \approx 1 + \frac{1}{8}\omega^4\Delta t^4 \tag{11.48}$$

and hence small $\omega\Delta t$ gives an even bigger suppression of any instability.

All methods must effectively reproduce the above series to give an accurate solution and so all methods will give the above results for error propagation. However, *some* methods can have more than one solution for g, one of which will be the above. With multiple values of g, the value with the biggest magnitude will dominate and the error will effective increase at this worst rate. Hence, a method which has a solution for g which is *bigger* that the one for all methods can be unstable even for $a < 0$. For the methods discussed in the last chapter, the only one which shows this effect is the leapfrog method.

The **leapfrog method** and its higher-order equivalents are rarely used because of this small step size stability issue. For leapfrog, the method is

$$u_{n+1} = u_{n-1} + 2f(t_n, u_n)\Delta t \tag{11.49}$$

For a linear function $f = au$ and assuming $\epsilon_{n-1} = \epsilon_n/g$, the error amplification factor g is given by

$$\epsilon_{n+1} = \frac{\epsilon_n}{g} + 2a\Delta t\ \epsilon_n \qquad \text{so} \qquad g = \frac{\epsilon_{n+1}}{\epsilon_n} = \frac{1}{g} + 2a\Delta t \tag{11.50}$$

Hence g is given by

$$g^2 - 2ga\Delta t - 1 = 0 \qquad \text{so} \qquad g = a\Delta t \pm \sqrt{1 + a^2\Delta t^2} \tag{11.51}$$

This gives two solutions which for small $a\Delta t$ to second-order are

$$g \approx 1 + a\Delta t + a^2\frac{\Delta t^2}{2} \qquad \text{and} \qquad -\left(1 - a\Delta t + a^2\frac{\Delta t^2}{2}\right) \tag{11.52}$$

The first solution is clearly the 'standard' amplification factor as for all methods, here to second order. For $a > 0$ this has the bigger magnitude and so gives the same level of instability as other methods. However, for $a < 0$, the other solution has $|g| > 1$ and so the error increases even for an exponentially *falling* solution. Note for the non-standard solution, g is negative which means if $\epsilon_n > 0$, i.e. the u_n is greater than the true value, then $\epsilon_{n+1} < 0$ and so is below

the true value, and vice versa. Hence, this is an oscillating instability. The same holds for higher-order generalisations of the leapfrog method as well and makes these methods unusable for such systems.

The two solutions for leapfrog clearly arise because the method uses u_n and u_{n-1}. The Adams-Bashforth methods also use earlier steps. In the AB2 case, the linear system gives

$$\epsilon_{n+1} = \epsilon_n + \frac{3}{2}a\Delta t\ \epsilon_n - \frac{1}{2}a\Delta t\ \frac{\epsilon_n}{g} \qquad \text{so} \qquad g = \frac{\epsilon_{n+1}}{\epsilon_n} = 1 + \frac{3a\Delta t}{2} - \frac{a\Delta t}{2g} \tag{11.53}$$

Hence g is given by

$$g^2 - g\left(1 + \frac{3a\Delta t}{2}\right) + \frac{a\Delta t}{2} = 0 \tag{11.54}$$

This gives two solutions which for small $a\Delta t$ to second-order are

$$g \approx 1 + a\Delta t + a^2\frac{\Delta t^2}{2} \qquad \text{and} \qquad \frac{1}{2}a\Delta t - \frac{1}{2}a^2\Delta t^2 \tag{11.55}$$

In this case, for small $a\Delta t$, the non-standard solution is suppressed by a factor of Δt as it enters through $f(t_{n-1}, u_{n-1})\Delta t$. Hence, the standard solution for g always dominates and so AB2 (and higher-order AB methods) are all as stable as other methods.

11.6 Stability for large step sizes

As shown in section (11.3), we should try to make Δt small to get a good approximation to the true solution. However, for very large numbers of u_i variables and/or a need to solve over long time periods, it can take too much computational power to use a small Δt, so a compromise much be found. This is particularly important when solving PDEs, as these are often treated as equivalent to solving large numbers of coupled ODEs, as will be discussed in section (12).

However, unfortunately simply increasing Δt does not only make the solution less accurate, but if taken too far, it can make the solution behave completely incorrectly.

Clearly, computational speeds have increased enormously over the years. Instability from this source used to be a major limitation but with faster computers, this becomes less of an issue.

11.6.1 Euler instability

Large Δt instability can be illustrated using the Euler method, again for a simple linear system for which $u' = f(t, u) = au$ for some constant a. The solution is easily seen to be $u(t) = e^{at}u_0$, i.e. exponential growth or decay, depending on the sign of a. First consider $a > 0$, which is exponential growth. In this case, the solution is nominally unstable as the error also grows exponentially, although as we saw previously, the fractional error is quite well-behaved. In this case, as Δt is increased, the solution gets less accurate but does not change its nature, as shown in figure (11.4) (left). For the case of $a < 0$, at least for $|a|\Delta t \leq 1$, the error exponentially decays so the solution is nominally stable, as shown in figure (11.4) (right). Again, the solution can be very inaccurate but is at least of the right general shape.

However, for $a < 0$ this is only true for $|a|\Delta t \leq 1$. This limit is easy to understand as each step is

$$u_{n+1} = (1 - |a|\Delta t)u_n \tag{11.56}$$

so if $|a|\Delta t > 1$, the solution will flip sign, i.e. u_{n+1} will become negative if u_n is positive, or vice versa; this is called an oscillating error. This is illustrated in figure (11.5) (left) where it is seen that the solution with $|a|\Delta t$ a small amount above 1 does at least decay with time. This is clearly a very bad approximation but the negative values might also cause other problems; e.g. this equation can be used to model the number of radioactive nuclei left in a source as a function of time. Getting a negative number of nuclei is obviously nonsense!

However, if even larger values of Δt are used, specifically $|a|\Delta t > 2$, then $|1 - |a|\Delta t| > 1$ and so the error not only oscillates but exponentially increases, as shown in figure (11.5) (right). Hence, if

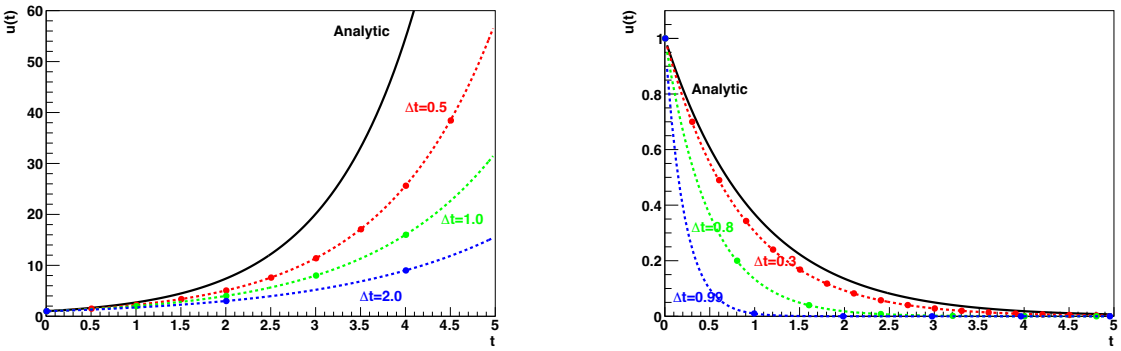

Figure 11.4: *Left: First few steps for the Euler method with $a = 1$ compared with the analytic solution (black), for $\Delta t = 0.5$ (red), 1.0 (green) and 2.0 (blue). The dashed lines show the analytic modified differential equation solutions which the Euler step values obey. Right: The same as the left plot but for $a = -1$, with $\Delta t = 0.3$ (red), 0.8 (green) and 0.99 (blue).*

having to use large steps, it is *essential* that the critical limit on Δt is understood for any method, and must not be exceeded. Even then, the solution will often be so poor as to be unusable unless Δt is chosen to be well away from the critical limit.

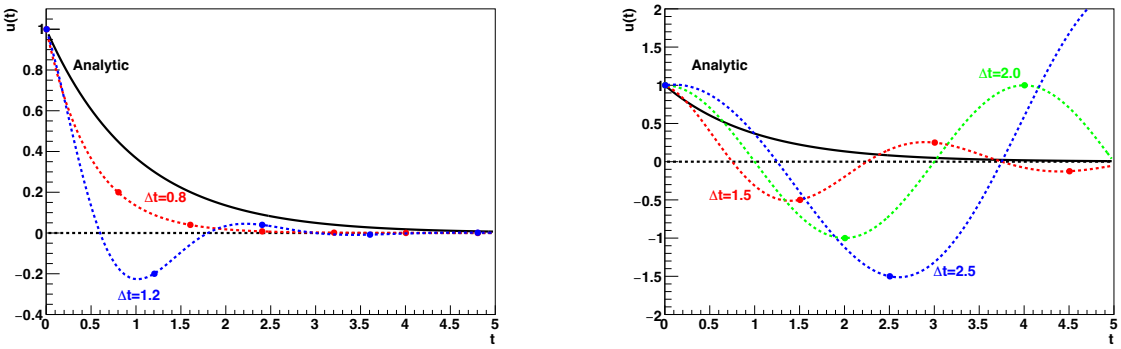

Figure 11.5: *Left: First few steps for the Euler method with $a = -1$ compared with the analytic solution (black), for $\Delta t = 0.8$ (red) and 1.2 (blue). The dashed lines show the analytic modified differential equation solutions which the Euler step values obey. Right: The same as the left plot but for $\Delta t = 1.5$ (red), 2.0 (green) and 2.5 (blue).*

An obvious question is whether instability can be avoided with higher order methods. At least all the explicit second order methods applied to this problem will give a step

$$u_{n+1} = \left(1 + a\Delta t + \frac{1}{2}a^2\Delta t^2\right) u_n \tag{11.57}$$

For the large values of $a\Delta t$ being discussed here, the extra term is no longer a small correction. In particular, it is positive even for $a < 0$ and for that case, the smallest possible value for the quantity in the brackets is 0.5 which occurs when $|a|\Delta t = 1$. Hence, there is no value of Δt which gives oscillating errors in this method. However, this does not mean the method is stable for $a < 0$; a value of $|a|\Delta t > 2$ means the expression in the brackets is > 1 and hence the error will exponentially grow. This is unstable, although it is a non-oscillating instability. Hence, simply going to higher order does not usually help in terms of allowing large increases in Δt.

11.6.2 Implicit Euler stability

Implicit Euler applied to the same linear system gives

$$u_{n+1} = \frac{u_n}{1 - a\Delta t} \tag{11.58}$$

The advantage of implicit Euler over explicit Euler is that it cannot become unstable for a falling exponential solution at large Δt. For a linear function with $a < 0$, i.e. the falling exponential case, the amplification factor will always be less than one. Hence, although again the solution is a bad approximation, it will not become unstable for large Δt. However, for $a > 0$, i.e. a rising exponential, the denominator can become infinite when $a\Delta t = 1$ and will be negative for higher Δt values. Hence, while implicit Euler is good for falling exponentials, it can have very poor stability for rising exponentials. These cases are shown in figure (11.6).

Again higher order implicit methods have similar properties. Many physics problems have solutions which fall exponentially so for these, implicit methods can be favoured over explicit methods for cases where large step sizes are required.

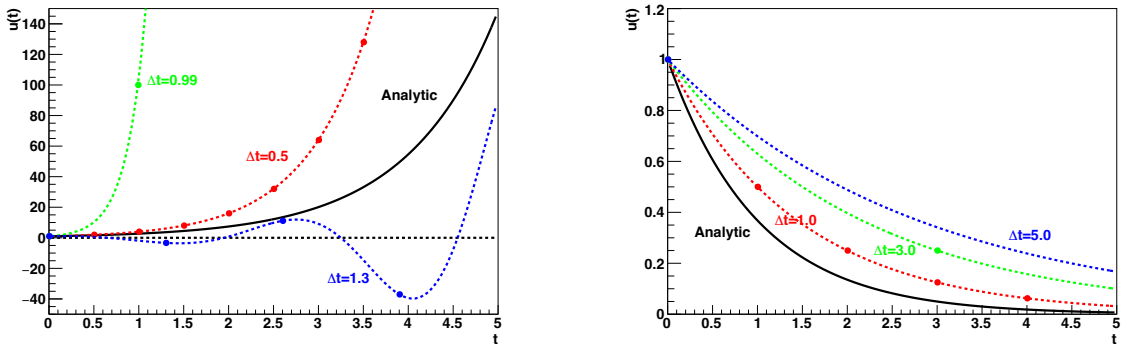

Figure 11.6: *Left: First few steps for the implicit Euler method with $a = 1$ compared with the analytic solution (black), for $\Delta t = 0.5$ (red), 0.99 (green) and 1.3 (blue). The dashed lines show the analytic modified differential equation solutions which the implicit Euler step values obey. Right: The same as the left plot but for $a = -1$, with $\Delta t = 1.0$ (red), 3.0 (green) and 5.0 (blue).*

Chapter 12

Partial Differential Equations

Outline of Section

- Classification of PDEs
- Solving Parabolic PDEs
- Solving Hyperbolic PDEs
- Solving Elliptic PDEs

12.1 Classification of PDEs

Partial differential equations (PDEs) are at the heart of many physical systems, such as Maxwell's equations or the Schrödinger equation. Unlike ODEs, which have a single independent variable, for PDEs we have partial derivatives with respect to more than one variable. This is the most common form of equation encountered when, e.g., we are modelling the spatial (x) and dynamic (t) characteristics of a system, or when we are modelling a static system in more than one dimension, e.g., (x, y). We will consider two-dimensional cases for the examples in this section of the course.

A general PDE can be classified through the coefficients multiplying the various derivatives. Most of the PDEs of interest in physics are second-order linear PDEs. The general form of a second-order linear PDE involving two independent variables x and y with solution $u(x, y)$ takes the form

$$\boxed{A\,\frac{\partial^2 u}{\partial x^2} + B\,\frac{\partial^2 u}{\partial x \partial y} + C\,\frac{\partial^2 u}{\partial y^2} + f\left(x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right) = g(x, y),} \tag{12.1}$$

where $f(\ldots)$ is an arbitrary function involving anything other than second derivatives (but involving u somehow). Note that the coefficients A, B and C of the second derivatives can depend on x, y, u, $\partial u/\partial x$ and/or $\partial u/\partial y$. The function $g(x, y)$ is independent of u and, if present, turns equation (12.1) into an inhomogeneous PDE.

In analogy with conic sections (in geometry) we define the **discriminant**

$$\boxed{Q = B^2 - 4\,A\,C,} \tag{12.2}$$

and mathematically classify PDEs as

$$\boxed{\begin{aligned} Q = 0 &\ : \text{Parabolic},\\ Q > 0 &\ : \text{Hyperbolic},\\ Q < 0 &\ : \text{Elliptic}. \end{aligned}} \tag{12.3}$$

For conic sections, $Q = 0$ yields a parabolic curve, $Q > 0$ a hyperbolic curve and $Q < 0$ an elliptical curve. The presence or absence of the inhomogeneous term $G(x, y)$ does not impact the parabolic/hyperbolic/elliptical classification of the PDE.

The classification can be done for three or more independent variables, e.g. $u(t, x, y, z)$, but is outside the scope of this course. Luckily, for the common PDEs occurring in physics, the

classification based on two variables (either one time and one spatial, or two spatial) happens to be the same when you expand the problem into more spatial dimensions. We will not show this, but merely state it for a few examples.

In physics, those PDEs which are a function of one time and spatial variables often appear in a flux-conservation form:

$$\frac{\partial \mathbf{u}}{\partial t} = -\nabla \cdot \mathbf{F}(\mathbf{u}) \tag{12.4}$$

where $\mathbf{F}$ is the conserved flux.

In terms of a more physical picture, basic examples of the three types are:

(a) **Diffusion Equation (Parabolic)**:

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\left(D\,\frac{\partial u}{\partial x}\right) \qquad \text{so} \qquad D\frac{\partial^2 u}{\partial x^2} + \left[\left(\frac{\partial D}{\partial x}\right)\left(\frac{\partial u}{\partial x}\right) - \frac{\partial u}{\partial t}\right] = 0 \tag{12.5}$$

where $D \equiv D(x,t)$ is the diffusion coefficient and the terms in the square brackets form the function f. This is a flux conservation equation with $F = -D\partial u/\partial x$, i.e. u flows away from regions of high concentration to low concentration. An example of this would be thermal diffusion, with u representing the temperature. Here, $A = D$, $B = C = 0$ hence $Q = 0$. (Note that B here would be the coefficient of $\partial^2 u/\partial t \partial x$ and C pertains to $\partial^2 u/\partial t^2$.) Again, moving to two or three spatial dimensions we have

$$\frac{\partial u}{\partial t} = \nabla \cdot (D \nabla u)$$

which is still a parabolic PDE. The diffusion coefficient can even be a function of u, $D(u)$, such that

$$\frac{\partial u}{\partial t} = D(u)\nabla^2 u + \frac{\partial D(u)}{\partial u}\,|\nabla u|^2$$

and the PDE is non-linear. This is still a parabolic PDE though.

(b) **Wave Equation (Hyperbolic)**:

$$\frac{\partial^2 u}{\partial t^2} = v^2\,\frac{\partial^2 u}{\partial x^2}, \tag{12.6}$$

where v is the (constant) speed of a wave. This is also a flux conservation equation, but now the flux of u is directional: there is two fluxes, one in the positive and the other in the negative x direction, travelling with a speed v. An example of this would be Maxwell's equations for the propagation of electrodynamic waves. Here, $A = v^2$, $B = 0$ and $C = -1$ hence $Q = 4v^2 > 0$. The 3D wave equation

$$\frac{\partial^2 u}{\partial t^2} = v^2\,\nabla^2 u,$$

is still a hyperbolic PDE. The wave speed v can depend on position and even on the wave displacement u and the PDE is still hyperbolic.

(c) **Poisson Equation (Elliptic)**:

$$\nabla^2\,u \equiv \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x,y), \tag{12.7}$$

i.e. how a solution (e.g. potential) reacts to a source (e.g. charge density). If $\rho(x,y) = 0$ then the Poisson equation becomes the **Laplace Equation**. Both the homogeneous (Laplace) and inhomogeneous (Poisson) equations are 'elliptic'. With reference to the general equation (12.1), for Poisson's equation $A = 1$, $B = 0$, $C = 1$ hence $Q = -4 < 0$. The 3D version of the Poisson and Laplace equations, where

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2},$$

are still elliptic PDEs.

Types of physical problems – these are governed by a combination of the PDE and boundary conditions (BCs) and can be classified into the following.

- **Initial value problems** are relevant to **hyperbolic** and **parabolic** PDEs, e.g., as in the diffusion and wave cases where the dynamical solution $u(t, x)$ is sought. In this case we require the initial state of the solution for all x and we solve this problem by *integrating forward in time* using an **initial value method**. We may also impose conditions on the spatial boundaries in this case.
  Comparing to ODEs, the general case changes from $u(t)$ to $u(t, x)$ and
  $$\frac{du}{dt} = f(t, u) \qquad \text{to} \qquad \frac{\partial u}{\partial t} = f\left(t, x, u, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}, \dots\right) \tag{12.8}$$
  where an arbitrary number of higher derivatives may be needed. These equations need an initial condition which fixes $u(0, x)$ to a specified function of x.
- **Boundary value problems** are relevant to **elliptic** PDEs, as in the case of the Poisson equation where we are solving for the static solution $u(x, y)$ requiring constraints on the boundary. The BCs are specified on a **closed surface**. These problems effectively involve *integrating in from the boundaries* and are solved numerically by **indirect matrix inversion** (also called relaxation) methods.
  These problems are the multi-dimensional equivalent of the ODE boundary value problems and are (at least conceptually) solved in a similar way.

Types of boundary conditions –

u defined on boundaries	: Dirichlet conditions,
$\vec{\nabla} u$ defined on boundaries	: Neumann conditions,
both u and $\vec{\nabla} u$ defined on boundaries	: Cauchy conditions.
u or $\vec{\nabla} u$ applied on diff. parts of boundary	: mixed conditions.

$u(x_r) = u(x_l + L) = u(x_l)$	: Periodic BC (e.g. in x).
$u(-x) = u(x)$	: Reflective BC (e.g. about $x_l = 0$).

where, e.g., x_l and x_r denote the left and right edges of the domain of width L. Since the scope of this section is PDEs with two independent variables (i.e. x, y or t, x), for 'boundary surface' we really mean a boundary curve. For $\vec{\nabla} u$ we mean $\partial u / \partial \zeta$ where $\zeta \to x$, y , z as appropriate.

For parabolic and hyperbolic initial-value problems, Cauchy conditions are applied only at $t = t_0$. $t \to \infty$ is known as an **open boundary** with nothing being imposed/specified there. For elliptic problems, this closed curve could be arbitrarily shaped (e.g. outline of a potato). However we will limit ourselves to a rectangular shaped boundary, and to Cartesian coordinates. A reflective BC is equivalent to having, e.g., $\partial u / \partial x = 0$ at a boundary, so is a special case of a Neumann condition. We will not consider subtleties in defining boundaries and applying BCs when there are three (or more) independent variables.

We have already seen Dirichlet conditions in section (11.2) and mixed conditions in section (11.4.1) when solving boundary value problems for ODEs.

12.2 Solving Parabolic PDEs

The example of a parabolic equation we will look at in detail is the diffusion equation in one spatial dimension with D constant, reminding ourselves that it comes from a flux-conservative form:

$$\frac{\partial u}{\partial t} = -\frac{\partial F}{\partial x} = -\frac{\partial}{\partial x}\left(-D\frac{\partial u}{\partial x}\right) = D\,\frac{\partial^2 u}{\partial x^2}. \tag{12.9}$$

This type of equation is the most similar to the ODEs as it has only one time derivative.

We cannot solve for an infinite range in x in a computer so we will assume a range R, such that $0 \geq x \geq L$. We cannot even solve for a continuous function (i.e. an infinite number of values) within this range so we discretise x into $N_x + 1$ equally spaced values, where the spacing is $h = L/N_x$ and $x_i = hi$ for $i = 0$ to N_x. We are then solving in general for all the $u(t_n, x_i)$ values, given the boundary conditions. The usual notation is $u_i^n = u_i(t_n) = u(t_n, x_i)$, i.e. the subscript and superscript on u denote the space and time indices, respectively. The approach is depicted in figure 12.1.

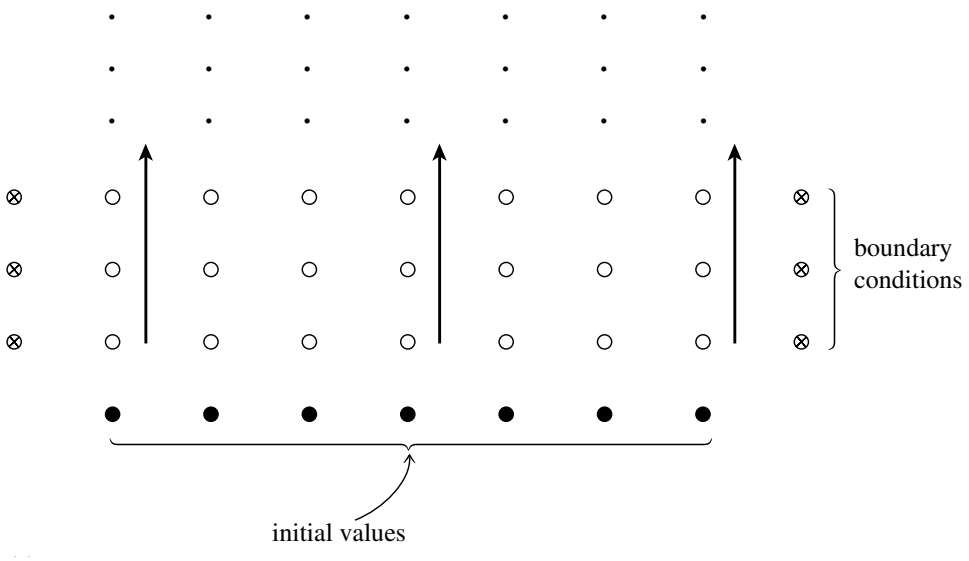

Figure 12.1: Initial value problem in (x, t) using a time and space discretisation scheme. The solution is integrated forward in time from an initial solution at a given "time slice", with boundary conditions given at the values of x at the ends of the range over all times. From Numerical Recipes.

There are two consequences arising from how x is handled which were not present in the ODE cases. The discretisation of space means the spatial derivatives must be estimated using numerical calculus. Also, the finite range means we have to define what happens at the ends of the range, i.e. the boundary conditions. For the latter, the numerical estimates of the spatial derivatives are expressed in terms of neighbouring x_i values and so the equations become

$$\frac{\partial u_i}{\partial t} = f_i\left(t, u_i, u_{i-1}, u_{i-2}, \ldots, u_{i+1}, u_{i+2} \ldots, h\right) \qquad \text{i.e.} \qquad \frac{\partial \vec{u}}{\partial t} = \vec{f}(t, \vec{u}, h) \tag{12.10}$$

which is a set of *coupled* ODE equations, where the use of numerical derivatives is what produces the coupling between the different u_i values. Hence, we can now solve the PDE using the techniques we already know. The initial conditions have to be provided for all x_i at t_0. The only new part remaining is to allow the derivative estimates to work when at the boundary, which depends on the method.

Explicit Euler method

Let us choose the first-order explicit Euler method for doing the time stepping

$$u_i^{n+1} = u_i^n + f_i(t_n, \vec{u}^n)\Delta t \tag{12.11}$$

For the spatial derivative let us choose the simplest approximation, the second order (i.e. $\mathcal{O}(h^2)$ accurate), central difference scheme,

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_i^n = \frac{u_{i-1}^n - 2\,u_i^n + u_{i+1}^n}{h^2}. \tag{12.12}$$

Inserting these approximations into the diffusion equation (12.9) we get

$$u_i^{n+1} = u_i^n + \frac{D\Delta t}{h^2}(u_{i-1}^n - 2u_i^n + u_{i+1}^n), \tag{12.13}$$

which can be written as

$$\boxed{u_i^{n+1} = (1 - 2\,d)\,u_i^n + d\,(u_{i-1}^n + u_{i+1}^n),} \tag{12.14}$$

where

$$d = \frac{D\,\Delta t}{h^2}, \tag{12.15}$$

is the **diffusion number**, a dimensionless parameter. This is a consistent method, usually labelled as $\mathcal{O}(\Delta t) + \mathcal{O}(h^2)$ to reflect the time and spatial accuracy, respectively.

One was to intuitively understand this is to consider how u_i^n is moved around; see figure (12.2). The equivalent equations for u_{i-1}^{n+1} and u_{i+1}^{n+1} would show u_i^n contributes a fraction d of its value to each of u_{i-1}^{n+1} and u_{i+1}^{n+1}, which leaves the remaining fraction $(1 - 2d)$ in u_i^{n+1}. Hence, it smears out its contents in both directions equally, as expected under diffusion, while conserving the total.

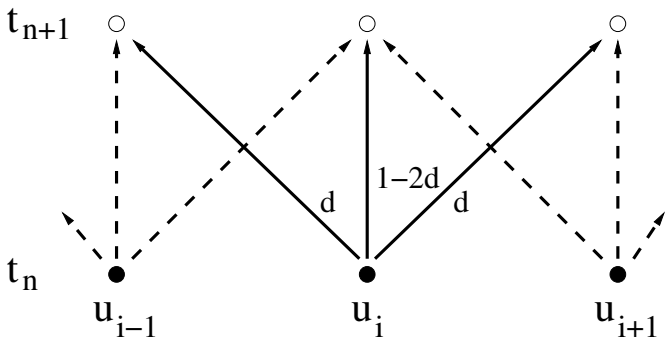

Figure 12.2: **Division of cell contents per step.** The amount of u at each point is divided into three parts. A fraction d goes to each of the left and right points and the remaining $1 - 2d$ is retained.

12.2.1 Boundary conditions

We now need to handle the boundary conditions. Periodic boundary conditions are quite simple to implement. With $u_{N_x} = u_0$, we only need to solve for u_i with $i = 0$ to $N_x - 1$. The second derivative at $i = 0$ is calculated using $u_{-1} = u_{N_x-1}$ and similarly at $i = N_x - 1$, the calculation uses $u_{N_x} = x_0$.

If not periodic, the boundary conditions usually define the u or $\partial u/\partial x$ values at the boundary. For example, in a heat diffusion problem, u would be the temperature and this might be fixed by heat baths to specific values at each end of the range. Alternatively, in a gas diffusion problem, u is the density of gas molecules and the boundary could correspond to a container wall. Hence there must be no gas flowing through the boundary, which means $\partial u/\partial x = 0$.

The two cases are illustrated in figure 12.3. If the boundary conditions define u, this is simple; u_0 and u_{N_x} are first set to the defined values (which can be time dependent) and the time step is only implemented for u_1 up to u_{N_x-1}. If the derivatives are defined, then a typical implementation is to add 'fake' extra u_{-1} and u_{N_x+1} values outside the boundaries. Using the derivative approximation in the same central difference scheme for consistency

$$\left.\frac{\partial u}{\partial x}\right|_i^n = \frac{u_{i+1}^n - u_{i-1}^n}{2h} \tag{12.16}$$

at $i = 0$ and N_x gives

$$u^n_{-1} = u^n_1 - 2h \left.\frac{\partial u}{\partial x}\right|^n_0 \qquad \text{and} \qquad u^n_{N_x+1} = u^n_{N_x-1} + 2h \left.\frac{\partial u}{\partial x}\right|^n_{N_x} \tag{12.17}$$

which fixes the fake values at the end of each step after the real u^n_i are calculated and using the defined derivative, which may again be time dependent. The fake values are then ready to be used to calculate the second derivative in the next step.

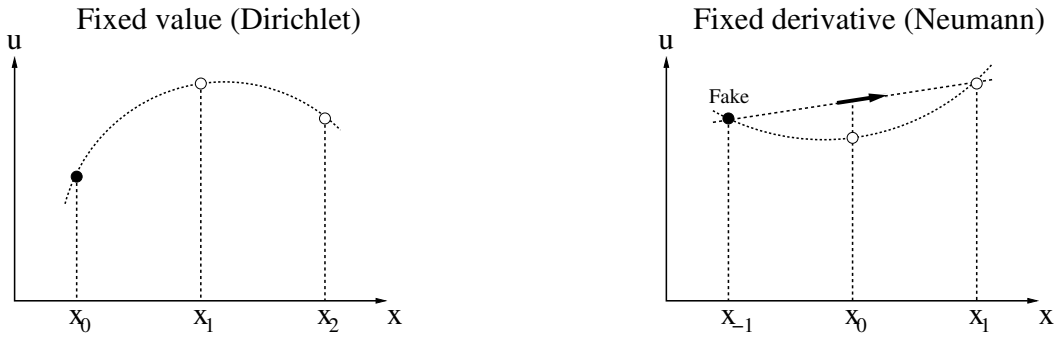

Figure 12.3: **Handling boundary conditions for the diffusion equation.** Left: for a boundary condition specifying the $u(x_0)$ value, the second derivative does not need to be evaluated at x_0, while it can be found at x_1 in the same way as usual. Right: for a boundary condition specifying the $\partial u/\partial x|_{x_0}$ value, this derivative together with the $u(x_1)$ value can be used to form a "fake" value $u(x_{-1})$ using the centre difference first-derivative formula. This can then be used to evaluate the second derivative at x_0 as usual.

An example of the explicit Euler method for diffusion with fixed boundary conditions is shown in figure 12.4.

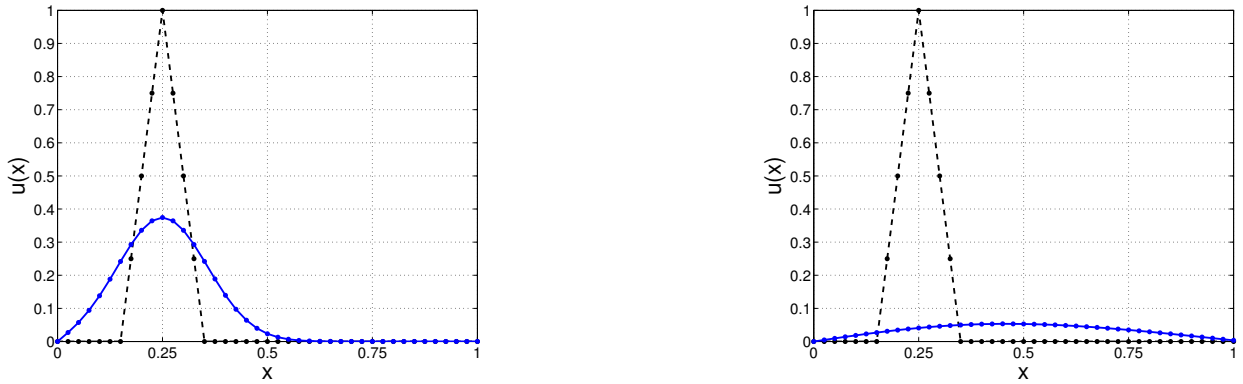

Figure 12.4: **Diffusion via the explicit method.** (Left) $t = 0.005$ and (Right) $t = 0.1$. The initial profile is a triangle (dotted black lines). Fixed (Dirichlet) boundary conditions are used; $u = 0$ on the boundaries. Parameters: $D = 1$, $h = 1/40$, $\Delta t = 0.25\tau_{\text{diff}-\text{grid}}$ (i.e. $d = 0.25$).

12.2.2 Stability

As shown later, either the Von-Neumann method (section 12.2.3) or the matrix method (section 12.2.4) can be used to derive the condition for the method to be stable

$$\boxed{d = \frac{D\,\Delta t}{h^2} < \frac{1}{2} \qquad \text{or equivalently} \qquad \Delta t < \frac{h^2}{2D}.} \tag{12.18}$$

This is an example of a **CFL condition** (named after Courant, Friedrich & Lewy). This bound has a physical implication in terms of the fraction being moved from a given cell as if d does not obey this bound, $1-2d$ would be negative so there would be more of u leaving than was present in the cell. It also tells us that the maximum time step is limited to roughly half the **grid diffusion time**

$$\tau_{\text{diff}-\text{grid}} \equiv \frac{h^2}{D}, \tag{12.19}$$

which characterises the time it takes for information to propagate between sites on the grid. This is an "artificial" new timescale which has been introduced by discretising x and is smaller than the original equation timescale by a factor of $(h/L)^2 = 1/N_x^2$. Figure 12.5 shows an example of the numerical instability that occurs if the CFL condition is violated.

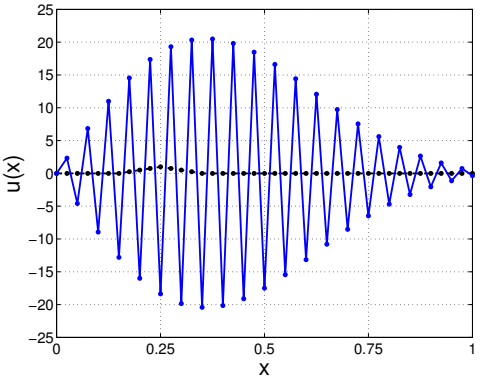

Figure 12.5: **Diffusion via the explicit method.** Numerical instability when the CFL condition is exceeded; shown at $t = 0.1$. Here $d = 0.51$. (The same system, and other parameters, are used as in figure 12.4, but note the change in range of the y axis.)

This is actually quite restrictive. The overall spatial scale of the problem is L so the physical diffusion time scale is $\tau_{_D} \sim R^2/D$. Given the CFL condition above, we have

$$\Delta t \leq \frac{\tau_{_D}}{2}\left(\frac{h}{L}\right)^2 = \frac{\tau_{_D}}{2}\left(\frac{1}{N_x^2}\right).$$

Since $h \ll L$ is required for good spatial resolution, many time steps are needed to observe the dynamics of the system (e.g. diffusive spreading of the pulse).

Thinking in terms of dimensions gives some further insight into the reason for the stability limit on Δt and the problems associated with it in terms of computational time. Assume we are solving the equation over a range R in x; we could then scale to use a dimensionless variable $\hat{x} = x/L$. If we then scale the time variable using $\hat{t} = tD/L^2$, then we get

$$\frac{\partial u}{\partial \hat{t}} = \frac{\partial^2 u}{\partial \hat{x}^2} \tag{12.20}$$

Just on dimensional grounds, we would expect all the interesting behaviour to happen over timescales of around $\hat{t} \sim \mathcal{O}(1)$. If we could do analytic derivatives, then any $\Delta\hat{t} \ll 1$ would give a stable and accurate solution. For example, choosing $\Delta\hat{t} = 0.01$ would mean $\mathcal{O}(1000)$ steps.

However, by dividing the x range into N_x steps, so that $N_x h = L$, we have introduced another (artificial) timescale $h^2/D = L^2/(N_x^2 D)$ into the problem; i.e. the grid diffusion time. In the scaled time, this is $\hat{\tau} = 1/N_x^2$. This means the step time now needs to be small compared to this new timescale as well, so $\Delta\hat{t} \ll 1/N_x^2$. This clearly requires a much smaller value but the overall timescale of the problem is still $\hat{t} \sim \mathcal{O}(1)$. For example, with $N = 100$, then choosing $\Delta\hat{t} = 0.01/N_x^2 = 10^{-6}$ should give an accurate result but this would then take $\mathcal{O}(10^6)$ steps which may be slow for 100 x_i values.

12.2.3 Stability analysis – von Neumann (Fourier) method

Von Neumann stability analysis checks whether the method causes a Fourier mode to grow exponentially in amplitude (i.e. is unstable) or not. It is arguably more physically intuitive than the matrix method (coming up soon), giving us insight about how the FD grid affects the equations. However, it only works for periodic boundary conditions.

The procedure is to consider doing a Fourier decomposition of the solution at a given time

$$u_i^n = \sum_k \widetilde{u}_k^n \, e^{ikx_i}, \tag{12.21}$$

Since the equation is linear

$$u_i^{n+1} \approx g u_i^n = \sum_k g \widetilde{u}_k^n \, e^{ikx_i} \tag{12.22}$$

where g is the amplification factor which determines the error. (This is approximate as u_i^{n+1} depends on other u_j^n but is reasonable if d is small.) The relation between u_i^n and adjacent points on the FD grid is

$$u_{i\pm1}^n = \sum_k \widetilde{u}_k^n \, e^{ik(x_i \pm h)} = \sum_k \widetilde{u}_k^n \, e^{ikx_i} e^{\pm ikh} \tag{12.23}$$

Substituting into the method gives

$$u_i^{n+1} = \sum_k g \widetilde{u}_k^n \, e^{ikx_i} = (1-2d)u_i^n + d(u_{i+1}^n + u_{i-1}^n) = \sum_k \left[(1-2d) + d(e^{ikh} + e^{-ikh}) \right] \widetilde{u}_k^n \, e^{ikx_i},$$

which means for each k

$$g = (1-2d) + 2d\cos(kh) = 1 - 2d\,[1 - \cos(kh)] = 1 - 4d\sin^2(kh/2).$$

The maximum wave-number allowable on the grid is

$$k_{max} = \frac{\pi}{h} \qquad (\text{i.e. } \lambda_{min} = 2h\,), \tag{12.24}$$

so that $\sin^2(kh/2)$ ranges from 0 to 1. Hence, to hold for all k, the condition for stability is

$$|g| \leq 1,$$

i.e., $|1 - 4d| \leq 1$ which, given that d must be positive, requires $d \leq 1/2$ as stated previously.

Referring back to the form of the Fourier mode above, we saw section 3.2 that a discrete (and finite) set of k values are allowed on a spatial grid. For a given, allowed wavelength the numerical scheme determines the effective numerical wave frequency ω_{num} (since $\omega_{\text{num}} = i \, \ln g/\Delta t$) that can propagate. ω_{num} can differ from the true value of kv_x, especially at short wavelengths, leading to incorrect phase speed, a phenomenon known as **numerical dispersion**. If ω_{num} has an imaginary part, then there will be unphysical **numerical damping** (or growth!) of the Fourier mode.

12.2.4 Stability analysis – matrix method

This is the same as the approach used for coupled ODEs in section (10.4). In this context the coupled variables are the u_i at each spatial grid point.

We can define $\tilde{\vec{u}}^n = \vec{u}^n + \vec{\epsilon}^n$ relating the FD approximation ($\tilde{\vec{u}}^n$) to the true solution of the exact PDE ($\vec{u}^n$) via an error ($\vec{\epsilon}^n$). Ignoring terms responsible for gradual, increasing loss of accuracy, as before, we are left with $\vec{\epsilon}^{\,n+1} = \mathbf{G} \cdot \vec{\epsilon}^{\,n}$ upon inserting $\tilde{\vec{u}}^n = \vec{u}^n + \vec{\epsilon}^n$ into the matrix version of the FDE (i.e. section (12.49)).

For stability we require that the spectral radius of the update matrix does not exceed unity, i.e. , $\rho(\mathbf{G}) \leq 1$. Thus all eigenvalues λ_i of the update matrix must satisfy

$$|\lambda_i| \leq 1 \qquad (\text{all } i).$$

Again, the bounds of the eigenvalues can be assessed by Gerschgorin's theorem (see section (6.6.3))

$$|\lambda_i - G_{ii}| \leq \sum_{j \neq i} |G_{ij}|.$$

For the diffusion case, all rows of the matrix are 'similar' so

$$|\lambda - (1 - 2d)| \leq 2d,$$

The upper limit on λ for stability is when the LHS is positive, when $\lambda - 1 + 2d \leq 2d$ so $\lambda \leq 1$. The lower limit is when the LHS is negative, when $-\lambda + 1 - 2d \leq 2d$ so $\lambda \geq 1 - 4d$ which is the CFL condition.

The advantage of the matrix method over Von-Neumann's method is its ability to deal with (i) BCs other than periodic, (ii) cases where the phase speed changes across the grid.

Implicit AM2 method (Crank-Nicolson)

The above used the Euler method for time stepping but in principle any of the ODE methods could be used. As an example, the 'Crank-Nicolson' method is effectively the equivalent of the implicit AM2 second-order method described in section 9.7. This gives

$$\begin{aligned} u_i^{n+1} &= u_i^n + \frac{1}{2}\left[f_i(t_n, \vec{u}^n) + f_i(t_{n+1}, \vec{u}^{n+1})\right]\Delta t \\ &= u_i^n + D\left[\frac{u_{i-1}^n - 2\,u_i^n + u_{i+1}^n}{h^2} + \frac{u_{i-1}^{n+1} - 2\,u_i^{n+1} + u_{i+1}^{n+1}}{h^2}\right]\Delta t \end{aligned} \tag{12.25}$$

Because the diffusion equation is linear, we do not need to use a predictor-corrector for the implicit method, but can solve directly. Rearranging all $n+1$ terms onto the LHS we get the system

$$\boxed{(2 + 2\,d)\,u_i^{n+1} - d\,(u_{i-1}^{n+1} + u_{i+1}^{n+1}) = (2 - 2\,d)\,u_i^n + d\,\left(u_{i-1}^n + u_{i+1}^n\right),} \tag{12.26}$$

or in matrix form

$$\mathbf{M} \cdot \vec{u}^{\,n+1} = \vec{b}^{\,n} + \vec{c}^{\,n}, \tag{12.27}$$

where

$$\mathbf{M} \equiv \begin{pmatrix} (2+2\,d) & -d & 0 & 0 & \cdot & \cdot & \cdot & 0 \\ -d & (2+2\,d) & -d & 0 & \cdot & \cdot & \cdot & 0 \\ 0 & -d & (2+2\,d) & -d & \cdot & \cdot & \cdot & 0 \\ & & & \cdot & & & & \\ & & & \cdot & & & & \\ & & & \cdot & & & & \\ 0 & 0 & \cdot & \cdot & \cdot & 0 & -d & (2+2\,d) \end{pmatrix}, \tag{12.28}$$

and

$$\vec{b}^{\,n} \equiv \begin{pmatrix} (2-2\,d) & d & 0 & 0 & \cdot & \cdot & \cdot & 0 \\ d & (2-2\,d) & d & 0 & \cdot & \cdot & \cdot & 0 \\ 0 & d & (2-2\,d) & d & \cdot & \cdot & \cdot & 0 \\ & & & \cdot & & & & \\ & & & \cdot & & & & \\ & & & \cdot & & & & \\ 0 & 0 & \cdot & \cdot & \cdot & 0 & d & (2-2\,d) \end{pmatrix} \cdot \vec{u}^{\,n}, \tag{12.29}$$

and $\vec{c}^{\,n}$ contains the boundary conditions

$$\vec{c}^{\,n} = \begin{pmatrix} d(u_0^n + u_0^{n+1}) \\ 0 \\ \cdot \\ \cdot \\ \cdot \\ 0 \\ d(u_{N_x}^n + u_{N_x}^{n+1}) \end{pmatrix}. \tag{12.30}$$

where Dirichlet boundary conditions have been assumed in this particular case. Hence the number of unknowns is $m = N_x - 1$ (i.e. u_i for $1 \leq i \leq N_x - 1$).

Crank-Nicolson is a consistent, $\mathcal{O}(\Delta t^2) + \mathcal{O}(h^2)$ scheme. It can be shown that this method is actually stable for all values of d (i.e. any Δt for a given D and h) but of course will still be inaccurate for large d. We can solve equation (12.27) using an iterative matrix solver (Jacobi, Gauss-Seidel or SOR) or through inverting $\mathbf{M}$ if this is computationally tractable. The solution at a given time is then found by the time-step equation (12.27).

12.3 Solving Hyperbolic PDEs

The example of a hyperbolic PDE we will look at is the wave equation, which involves second order derivatives in both time and space. It is

$$\frac{\partial^2 u}{\partial t^2} - v^2 \frac{\partial^2 u}{\partial x^2} = 0, \tag{12.31}$$

in the 1D, linear case where v is the phase speed. To solve this, we need to know the initial conditions, which specify both the functions u and $\partial u/\partial t$ for all x at $t = 0$.

We could treat this as for second-derivative ODEs by defining a new variable $w(t, x)$ (since the symbol v is already being used!) and hence making this two first-derivative in time equations

$$\frac{\partial u}{\partial t} = w \qquad \text{and} \qquad \frac{\partial w}{\partial t} = v^2 \frac{\partial^2 u}{\partial x^2} \tag{12.32}$$

These could be solved together in a similar way to the diffusion equation above. However, this is not the best way in this particular case; some physical insight shows an alternative way to split this equation into two first-derivative equations.

Advection equations

This wave equation can actually be split into two uncoupled first-order PDEs,

$$\frac{\partial f}{\partial t} + v \frac{\partial f}{\partial x} = 0\,, \qquad \frac{\partial g}{\partial t} - v \frac{\partial g}{\partial x} = 0, \tag{12.33}$$

where $v \geq 0$ is the phase speed. They have general solutions $f(x,t) = F(x - vt)$ (an arbitrary function propagating forward, i.e. in the $+x$ direction) and $g(x,t) = G(x + vt)$ (an arbitrary function propagating backwards, i.e. in the $-x$ direction), respectively. These are known as the **advection equations**, or more correctly the 'constant velocity advection' (CVA) equations. The general solution of the linear wave-equation is thus $u(x,t) = F(x - vt) + G(x + vt)$.

However, this form hides the generality and the wave equation's connection to the flux-conservative form. Instead, they can be written as a set of coupled first-order PDEs:

$$\frac{\partial y}{\partial t} = v\frac{\partial w}{\partial x}\,, \qquad\qquad \frac{\partial w}{\partial t} = v\frac{\partial y}{\partial x}\,,$$

where $y = v\partial u/\partial x$ and $w = \partial u/\partial t$. In this case they are now in flux-conservative form with:

$$\mathbf{u} = \begin{pmatrix} f' \\ g' \end{pmatrix} \text{ and } \mathbf{F} = \begin{pmatrix} 0 & -v \\ -v & 0 \end{pmatrix} \cdot \mathbf{u}$$

So the solution of the wave equations reduces to solving the advection equations for two functions. Therefore, most of the effort in numerically solving hyperbolic PDEs concentrates on solving the flux-conservative advection equation, of the general 1D form:

$$\frac{\partial u}{\partial t} = -\frac{\partial}{\partial x}\left(F\right), \tag{12.34}$$

where $F = v_x u$, where now v_x is a velocity, so that its sign determines the direction of propagation. We need to solve for both signs of v_x although the methods are equivalent so we will just look at the forward ($v_x > 0$) case in detail. Simplifying so that

Coupled conservation-law and equation of motion

Physically, wave equations often arise from combining a *conservation law* (e.g. conservation of mass) with an *equation of motion*. For example, in the case of an acoustic wave in gas

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \vec{u}) = 0, \qquad \text{(mass conservation law)} \tag{12.35}$$

$$\frac{\partial \left(\rho \vec{u}\right)}{\partial t} + \vec{u} \cdot \nabla(\rho \vec{u}) = -\nabla P, \qquad \text{(equation of motion)} \tag{12.36}$$

where ρ is mass density, $\vec{u}$ is the fluid velocity and P is the pressure. These are coupled first-order PDEs. For example, using the adiabatic gas law ($PV^\gamma = \text{const.}$) which yields $P\rho^{-\gamma} = P_o\rho_o^{-\gamma}$ (where P_o and ρ_o are the unperturbed values in the absence of a wave) and then ignoring the non-linear term $\vec{u} \cdot \nabla(\rho\vec{u})$ yields the wave equation

$$\frac{\partial^2 \rho}{\partial t^2} = \frac{\gamma P_o}{\rho_o}\nabla^2\rho$$

when the the adiabatic law is linearised. We identify $c = \sqrt{\gamma P_o/\rho_o}$ as the *sound speed*.

In 1D linearised versions of the mass conservation law and equation of motion are

$$\begin{aligned} \frac{\partial \rho}{\partial t} + \rho_o\frac{\partial u}{\partial x} &= 0, \\ \frac{\partial u}{\partial t} + \frac{c^2}{\rho_o}\frac{\partial \rho}{\partial x} &= 0, \end{aligned} \tag{12.37}$$

i.e. a set of coupled equations which can be combined into a single PDE of conservative form,

$$\frac{\partial \vec{U}}{\partial t} = -\frac{\partial(\mathbf{F} \cdot \vec{U})}{\partial x}, \tag{12.38}$$

where **F** is known as a **flux operator**. In our example

$$\vec{U} \equiv \begin{bmatrix} \rho(x,t) \\ u(x,t) \end{bmatrix} \qquad \text{and} \qquad \mathbf{F} \equiv \frac{1}{\rho_o} \begin{bmatrix} 0 & \rho_o^2 \\ c^2 & 0 \end{bmatrix}. \tag{12.39}$$

Equation (12.38) looks like an advection equation for a vector of functions, when we bear in mind that **F** is a matrix of constants so can come outside the spatial derivative. For EM waves in vacuum (plane waves propagating in the x-direction), $\vec{U} = [E_y, cB_z]^T$ and the off-diagonal elements of **F** would become c (speed of light in vacuum).

12.3.1 Upwind method

We take inspiration from the physics of advection and chose a one-sided finite difference scheme (FDS) for $\partial F/\partial x$ in the *direction from which information propagates*. For $v_x > 0$ we use the backwards difference scheme

$$\left.\frac{\partial F}{\partial x}\right|_i^n = \frac{F_i^n - F_{i-1}^n}{h} \qquad \{\,\mathcal{O}(h)\,\}. \tag{12.40}$$

We again choose the explicit Euler method for the time stepping, which gives

$$u_i^{n+1} = u_i^n + f_i(t_n, \vec{x}^{\,n})\Delta t = u_i^n - \frac{\Delta t}{h}\left(F_i^n - F_{i-1}^n\right), \tag{12.41}$$

Now if the velocity is constant, we find:

$$\boxed{u_i^{n+1} = u_i^n - \frac{v_x \Delta t}{h}\left(u_i^n - u_{i-1}^n\right) = (1-a)u_i^n + a u_{i-1}^n}, \tag{12.42}$$

$$u_i^{n+1} = (1-a_i)u_i^n + a_{i-1}u_{i-1}^n$$

where the factor

$$a = |v_x|\Delta t/h, \tag{12.43}$$

is known as the **advection number**, an important dimensionless parameter with respect to the numerical properties of FD methods for hyperbolic PDEs (as we shall see shortly). This is known as the **upwind method**, which when evaluated to first order in space (as done here) is also known as the "donor cell method".

Again, this equation can be interpreted through where u moves to; see figure (12.6). In this case u_i^n moves a fraction a to u_{i+1}^{n+1}, leaving a fraction $1-a$ in u_i^{n+1}. The movement is only in the direction of propagation, which is why the backwards difference derivative approximation is appropriate.

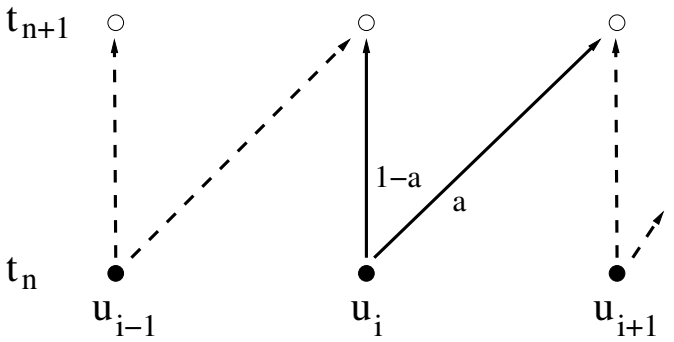

Figure 12.6: **Division of cell contents per step.** The amount of u at each point is divided into two parts. A fraction a goes in the direction of the wave motion and the remaining $1-a$ is retained.

For $v_x < 0$ we choose the forward difference scheme

$$\left.\frac{\partial u}{\partial x}\right|_i^n = \frac{u_{i+1}^n - u_i^n}{h} \qquad \{\,\mathcal{O}(h)\,\}, \tag{12.44}$$

and thus the FDE is

$$u_i^{n+1} = u_i^n + a(u_{i+1}^n - u_i^n) = (1-a)u_i^n + au_{i+1}^n. \tag{12.45}$$

In this case, a fraction a is transferred towards the negative x direction, as is appropriate for this scheme.

Because the physical speed of information propagation is finite for hyperbolic PDEs, explicit schemes like the above are generally best. Implicit schemes are more computationally intensive for the same accuracy. They generally offer no benefits for hyperbolic PDEs.

Figure 12.7 shows the upwind method applied to an initially triangular profile, in the case of positive v_x. The spatial resolution is quite coarse ($n_x = 40$). For an advection number of $a = 0.5$ the profile unphysically spreads and blurs as it moves, due to inaccuracies of the upwind method. For $a = 1.05$ the profile breaks up due to numerical instability. We will look more closely at these two issues (numerical diffusion and instability) below.

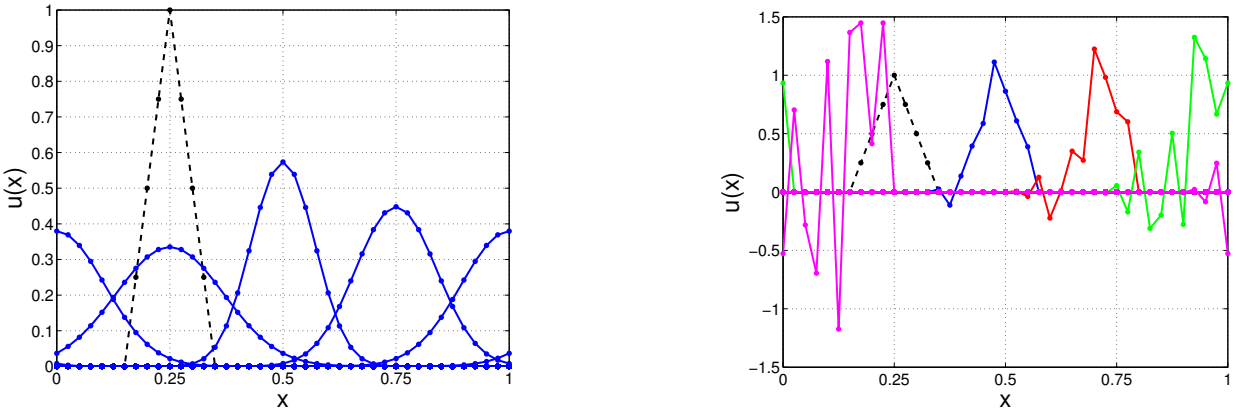

Figure 12.7: **Advection via the upwind method.** (Left) Advection number $a = 0.5$, and (Right) $a = 1.05$. The initial profile is a triangle (dotted black lines). Parameters: $v_x = +1$, $h = 1/40$, (left) $\Delta t = h/2$ and (right) $\Delta t = 1.05h$. The profile moves to the right and eventually returns to its starting location because of the periodic BCs. The four snapshots are at $t = 0.25$, 0.5. 0.75, 1.0 and they are coloured differently in the right plot. For $a = 0.5$, severe numerical diffusion can be seen. For $a = 1.05$ the upwind method is unstable.

Consistency & accuracy – The modified differential equation obtained from (12.42) is

$$\frac{\partial U}{\partial t} + v_x\frac{\partial U}{\partial x} = -\frac{1}{2}\frac{\partial^2 U}{\partial t^2}\Delta t + \mathcal{O}(\Delta t^2) + \frac{1}{2}\frac{\partial^2 U}{\partial x^2}v_x h + \mathcal{O}(h^2). \tag{12.46}$$

The RHS vanishes as $\Delta t \to 0$ and $h \to 0$ which demonstrates that the upwind method is consistent with the advection PDE. The leading error terms show that it is first-order accurate in time and in space; it is a $\mathcal{O}(\Delta t) + \mathcal{O}(h)$ scheme.

Numerical diffusion – The above MDE can be rewritten as

$$\begin{aligned}\frac{\partial U}{\partial t} + v_x\frac{\partial U}{\partial x} &= \frac{|v_x|h}{2}(1-a)\frac{\partial^2 U}{\partial x^2} + \mathcal{O}(\Delta t^2) + \mathcal{O}(h^2) \\ &\approx D_{\text{num}}\frac{\partial^2 U}{\partial x^2},\end{aligned} \tag{12.47}$$

which shows that the upwind method is really giving us the solution of the advection equation with some non-physical diffusion (the leading term on the RHS). This 'numerical diffusion' has a diffusion coefficient D_{num} that grows, the larger h is. Numerical diffusion causes a pulse with sharp features (e.g. top hat function) to artificially blur out as it propagates along. The higher order error terms further artificially modify the dynamics but are weaker than numerical diffusion.

Matrix equation – Practical implementation of (12.42) does not have to be (and should not be!) done via matrices. However formulating the equations applied at each spatial grid point as a matrix equation is useful both conceptually and for, e.g., undertaking a stability analysis. We assume ***periodic spatial boundary conditions***, which means

$$u^n_{N_x} = u^n_0, \tag{12.48}$$

on our choice of spatial grid. Although there are $N_x + 1$ spatial points, periodic BCs reduce the number of unknowns to $m = N_x$. The matrix equation (for the $v_x > 0$ case) is then

$$\begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{N_x} \end{pmatrix}^{n+1} = \begin{pmatrix} (1-a) & 0 & 0 & \dots & 0 & a \\ a & (1-a) & 0 & \dots & 0 & 0 \\ 0 & a & (1-a) & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & a & (1-a) \end{pmatrix} \cdot \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{N_x} \end{pmatrix}^n,$$

$$\vec{u}^{\,n+1} = \mathbf{G} \cdot \vec{u}^{\,n}, \tag{12.49}$$

where $\mathbf{G}$ is the update matrix (analogous to those seen with ODE FD methods in section (10).

12.3.2 Stability analysis

Stability for this case can be handled similarly to the diffusion case. For the Von Neumann method, substituting for a Fourier mode into the upwind method yields

$$g \cancel{y_i^n} = \cancel{y_i^n}(1-a) + \cancel{y_i^n}\, a\, e^{-ikh},$$

and then separating out into real and imaginary parts gives

$$g = (1-a) + a\cos(kh) \;-\; i\, a \sin(hk).$$

After some algebra we get

$$|g|^2 = 1 - 2a(1-a)[1-\cos(kh)],$$

and then using the identity $1 - \cos(2\theta) = 2\sin^2\theta$ yields

$$|g|^2 = 1 - 4a(1-a)\sin^2(kh/2). \tag{12.50}$$

Hence, the condition for stability is

$$|g| \leq 1,$$

i.e., $1 - 4a(1-a) \leq 1$ which yields $a \geq 0$ and

$$\boxed{a \leq 1 \qquad \text{or equivalently} \qquad \Delta t \leq h/|v_x|.} \tag{12.51}$$

which is the CFL condition for the upwind method. The CFL condition is equivalent to

$$|v_x| \leq v_{\text{grid}}, \tag{12.52}$$

i.e. the physical, speed of propagation of information must be less than the *grid speed* $v_{\text{grid}} = h/\Delta t$. Again, the discretisation in x has introduced another artificial timescale $\tau_{\text{grid}} = h/|v_x|$. If the CFL

condition is violated, then the domain of dependence of a given grid point is physically larger than the FD scheme can reach, which leads to problems. Note that in practice the time step needs to be quite a bit smaller than that given by the CFL condition to get good accuracy.

Checking stability with the matrix method for the upwind method (with periodic BCs), again all rows of the matrix are again 'similar' so the eigenvalues must satisfy

$$|\lambda - (1-a)| \leq a,$$

where a is a positive real value. So λ lies within (or on the edge of) a disk of radius a in the complex plane, with this disk centred at $z_o = (1-a)$, i.e., on the real axis. The 'right edge' of this disk is always at $z_{right} = +1$. The 'left edge' lies at $z_{left} = 1 - 2a$. For $a > 1$ we have $z_{left} < -1$ and so $|\lambda| > 1$ (i.e. instability) is possible. (Note that all other points on the circle edge break-out beyond unit distance from the origin when this happens, except the point exactly on the +ve real axis.) Hence $a \leq 1$ is required for $|\lambda| \leq 1$ and therefore stability.

12.4 Solving Elliptic PDEs

To solve an elliptic equation we take a similar approach to the finite difference methods of section 11.4 by discretising the system onto a 2D lattice of points and using finite difference approximations to the partial derivatives. The lattice is also known as a **mesh** or a **grid**. As an example we start with Laplace's equation

$$\nabla^2 u(x,y) = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0, \tag{12.53}$$

We will see that we obtain a matrix equation $\mathbf{A}\cdot\vec{u} = \vec{b}$ (as we did in section 11.4.1 for 1D boundary value problems). This can be solved for the values of u on the 2D grid (packaged into a vector $\vec{u}$) using a relaxation method, i.e. an iterative matrix solver such as Jacobi, Gauss-Seidel or SOR. The vector $\vec{b}$ holds the boundary conditions. We will see that there is a way to implement the Jacobi method without resorting to actual matrix algebra, but rather *directly* working on the 2D array of gridded values for u.

We consider a rectangular domain extending $0 \leq x \leq L_x$ and $0 \leq y \leq L_y$. There are $N_x + 1$ regularly-spaced points in the x-direction including the edges and similarly $N_y + 1$ in the y-direction.[1] To keep things simple the grid spacing will be taken as h in both directions. (The numerical equations below can readily be generalised for different spacings, e.g., $\Delta x \neq \Delta y$. It does not change the approach.) The location of the grid points are defined as

$$x_i = i\,h \qquad (i = 0,\, 1,\, \ldots\,, N_x) \qquad L_x = N_x\,h \tag{12.54}$$

$$y_j = j\,h \qquad (j = 0,\, 1,\, \ldots\,, N_y) \qquad L_y = N_y\,h. \tag{12.55}$$

Thus $i = 0$ corresponds to the left-boundary, $i = N_x$ the right-boundary. Similarly $j = 0$ and $j = N_y$ correspond to the bottom and top boundaries, respectively and we will use $u_{i,j} = u(x_i, y_j)$. There are $m = (N_x - 1) \times (N_y - 1)$ 'interior' points.

For the second order partial derivatives in the Laplacian operator we apply the finite difference approximation seen in section 5.1, i.e., equation (5.13), which yields

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{i,j} = \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2} \qquad \left\{\,\mathcal{O}(h^2)\,\right\}, \tag{12.56}$$

and

$$\left.\frac{\partial^2 u}{\partial y^2}\right|_{i,j} = \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h^2} \qquad \left\{\,\mathcal{O}(h^2)\,\right\}, \tag{12.57}$$

[1] Notational note: "N_x", "N_y", etc. are what we use here for clarity, but you will often see the (horrible) alternative notation "nx" and "ny" in numerical analysis texts. Beware!

giving the finite difference approximation to the Laplacian

$$\boxed{\nabla^2 u_{i,j} = \frac{u_{i-1,j} + u_{i,j-1} + u_{i+1,j} + u_{i,j+1} - 4u_{i,j}}{h^2} \qquad \{\, \mathcal{O}(h^2) \,\}\,.} \tag{12.58}$$

This Laplacian can be represented as a **pictorial operator** acting on each unknown grid point

$$\nabla^2 u_{i,j} = \frac{1}{h^2} \begin{bmatrix} & 1 & \\ 1 & -4 & 1 \\ & 1 & \end{bmatrix} u_{i,j}. \tag{12.59}$$

As depicted in figure 12.8, this visually shows how the four points to the left, right, above and below the centre point (where the Laplacian is being determined) are involved and gives their relative weights (+1). The relative weight of the centre point is −4. This is also known as a **finite difference stencil**.

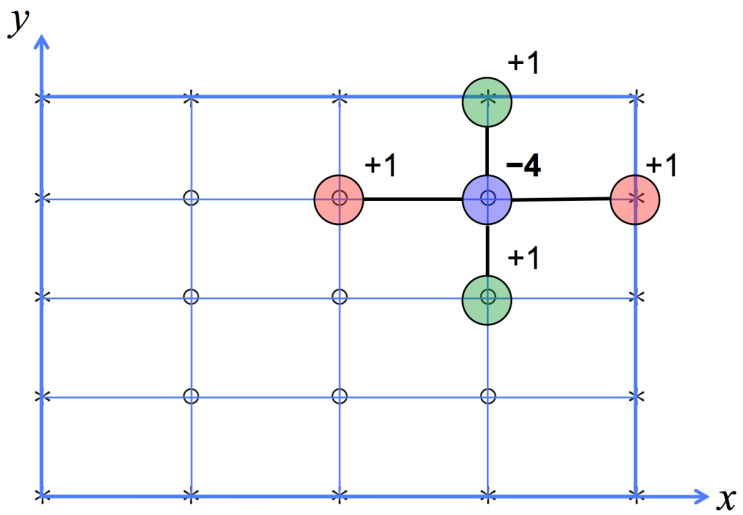

Figure 12.8: **FD stencil for the 2D Laplacian** $\nabla^2 u_{i,j}$. (Note that the $1/h^2$ factor is omitted in the indicated weights.)

12.4.1 Dirichlet boundary conditions

We will first look at this system with Dirichlet boundary conditions, i.e., u specified on the boundary.

To illustrate the method we take $N_x = 4$ and $N_y = 4$ and discretise the system using the following scheme;

$$\begin{array}{ccccc} C_{0,4} & C_{1,4} & C_{2,4} & C_{3,4} & C_{4,4} \\ * & * & * & * & * \\ C_{0,3} & u_{1,3} & u_{2,3} & u_{3,3} & C_{4,3} \\ * & \circ & \circ & \circ & * \\ C_{0,2} & u_{1,2} & u_{2,2} & u_{3,2} & C_{4,2} \\ * & \circ & \circ & \circ & * \\ C_{0,1} & u_{1,1} & u_{2,1} & u_{3,1} & C_{4,1} \\ * & \circ & \circ & \circ & * \\ C_{0,0} & C_{1,0} & C_{2,0} & C_{3,0} & C_{4,0} \\ * & * & * & * & * \end{array} \tag{12.60}$$

The boundary conditions are set as $u_{1,0} = C_{1,0}$, $u_{0,1} = C_{0,1}$, etc., and we need to solve for the nine unknown internal points, i.e. $u_{i,j}$ for both $i, j = 1$ to 3. (Note, for the particular lower-order method used here, the four C corner values are not used but are part of the domain.) This is the same approach as described earlier in one spatial dimension for the initial value problems.

Equation (12.58) applied at each internal grid point forms one of $m = (N_x - 1) \times (N_y - 1)$ simultaneous equations. We can represent the system of simultaneous equations as a matrix equation by stacking all the $u_{i,j}$ into a single "vector of vectors" (of size m) and also forming a "matrix of matrices" (of size $m \times m$, i.e. 9×9 in our example)

$$\left(\begin{array}{ccc|ccc|ccc} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{array}\right) \left(\begin{array}{c} u_{1,1} \\ u_{1,2} \\ u_{1,3} \\ \hline u_{2,1} \\ u_{2,2} \\ u_{2,3} \\ \hline u_{3,1} \\ u_{3,2} \\ u_{3,3} \end{array}\right) = - \left(\begin{array}{c} C_{0,1} + C_{1,0} \\ C_{0,2} \\ C_{0,3} + C_{1,4} \\ \hline C_{2,0} \\ 0 \\ C_{2,4} \\ \hline C_{3,0} + C_{4,1} \\ C_{4,2} \\ C_{4,3} + C_{3,4} \end{array}\right), \tag{12.61}$$

which is of the form

$$\mathbf{A} \cdot \vec{u} \; = \; \vec{b}. \tag{12.62}$$

and where each unknown is one element of the solution vector $\vec{u}$.

We have used a 2D-array to 1D-array indexing scheme to package the unknowns into the vector. Our chosen mapping from the spatial indices i, j into the row index p of the solution vector $\vec{u}$ is

$$p = j + (N_y - 1) \times (i - 1), \tag{12.63}$$

e.g. for our case with $N_x = N_y = 4$, unknown $u_{2,3}$ with $i = 2$ and $j = 3$ is located at row $p = 3 + 3 \times (2 - 1) = 6$. This mapping is not unique: We could equally as well have cycled over the x grid points first, and then the y points. (Nb: for three spatial dimensions, a 3D-array to 1D-array scheme – an extension of (12.63) – needs to be used.) The horizontal lines in (12.61) serve to visually vertically-partition the matrix and vectors into $N_x - 1$ blocks. Each block has $N_y - 1$ rows.

Consistency and Accuracy – The order of accuracy of this FD scheme is $\mathcal{O}(h^2)$. For PDEs, the order can be determined from the modified differential equation (MDE). If Taylor expansions for $u_{i,j\pm1}$ and $u_{i\pm1,j}$ are substituted into (12.58), the FD equivalent of Laplace's equation, then the resulting MDE can be shown to be

$$\nabla^2 U = -\frac{1}{12} \left(\frac{\partial^4 U}{\partial x^4} + \frac{\partial^4 U}{\partial y^4} \right) h^2 + \mathcal{O}(h^4) \tag{12.64}$$

demonstrating **consistency** and **second-order accuracy**. The order is the rate at which the numerical solution converges to the true solution as the step sizes tend to zero.

Solving the FD matrix equation – We can use an iterative matrix solver, as covered in chapter 6, to find the solution to the FD matrix equation. In this case the Jacobi method is suitable, since the spectral radius of the Jacobi update matrix $\mathbf{G}$ (corresponding to $\mathbf{A}$ in equation 12.62) satisfies $\rho(\mathbf{G}) < 1$. [2]

[2] The FD matrix of the 2D (and 3D) Laplacian is an important example of a matrix that is not *strictly* diagonally dominant (since $|A_{ii}|$ equals $\sum_{j \neq i} |A_{ij}|$ for some rows) but does actually work with the Jacobi method. This is because the Jacobi update matrix turns out to have a spectral radius less then unity. Similarly, the FD form of the Laplacian is suitable for Gauss-Seidel and SOR iterative matrix solvers.

Starting with the zeroth-order guess $\vec{u}^{(0)} = \mathbf{D}^{-1} \cdot \vec{b}$ (where the (0) is the iteration number, not the time!) we can iterate with

$$\vec{u}^{(k+1)} = \mathbf{D}^{-1} \cdot \vec{b} - \mathbf{D}^{-1} \cdot (\mathbf{T}_L + \mathbf{T}_U) \cdot \vec{u}^{(k)}. \tag{12.65}$$

In this case $\mathbf{D}^{-1} = -\mathbf{I}/4$ and so we have $\vec{u}^{(0)} = -\vec{b}/4$ and

$$\vec{u}^{(k+1)} = \vec{u}^{(0)} + \frac{1}{4}\left[(\mathbf{T}_L + \mathbf{T}_U) \cdot \vec{u}^{(k)}\right]. \tag{12.66}$$

This can also be represented by the following pictorial operator

$$u_{i,j}^{(k+1)} = u_{i,j}^{(0)} + \frac{1}{4}\begin{bmatrix} & 1 & \\ 1 & 0 & 1 \\ & 1 & \end{bmatrix} u_{i,j}^{(k)}, \tag{12.67}$$

This pictorial operator averages over the nearest four neighbours for each of the internal points. At first sight, it seems that the boundary conditions have been forgotten since $\vec{b}$ is missing in (12.67). However, when updating an unknown on the edge of the internal domain (e.g. $u_{2,1}$) the pictorial operator automatically accesses the necessary boundary value(s) (i.e. $u_{2,0} = C_{2,0}$ in this case).

Thus the algorithm to solve the system using the Jacobi method in 'pictorial operator' form is

- Initialise all grid points including boundary values.
- Operate on all internal points with the pictorial operator, placing all $u_{i,j}^{n+1}$ values in a new array, i.e., leave $u_{i,j}^{n}$ values alone whilst scanning the pictorial operator over each grid point.
- Iterate until the solution has converged.

(See section 6.4.3, page 46 for details on checking for convergence). Note that Gauss-Seidel and SOR can also be implemented via a (different) pictorial operator scanning over the gridded values $u_{i,j}$ but there are extra considerations (not covered here).

Poisson – It is simple to show that the extension to the Poisson case

$$\nabla^2 u = \rho(x, y), \tag{12.68}$$

when using the Jacobi method in pictorial-operator form requires the minor modification

$$u_{i,j}^{(k+1)} = u_{i,j}^{(0)} + \frac{1}{4}\begin{bmatrix} & 1 & \\ 1 & 0 & 1 \\ & 1 & \end{bmatrix} u_{i,j}^{(k)} - \frac{h^2}{4}\rho_{i,j}, \tag{12.69}$$

where the source term is discretised on an identical grid.

Extension to 3D – The methods above can readily be extended to 3D. The unknowns then become $u_{i,j,k}$ where 'k' indexes the third dimension (e.g. z) of the grid. One needs the FD form of the Laplacian in 3D. The (Jacobi) pictorial operator would then be a 3D stencil, accessing the six nearest neighbouring grid points.

Physical interpretation of scheme – The pictorial operator shows how the centre point is locally coupled to four surrounding points. But these local couplings connect up giving a global coupling between all points; everything is interconnected. Hence we end up with a matrix equation to be solved. Physically, information at one point in space instantaneously propagates everywhere else. This is okay, as 'time' does not come into the equations, so there is no need to worry about causality.

12.4.2 Spectral methods – Solving the Poisson Equation using Fourier Transforms

One application of DFTs is their use in solving PDEs. These are known as **spectral methods**. As an example we consider the general Poisson equation

$$\nabla^2 u(\vec{x}) = \rho(\vec{x}), \tag{12.70}$$

where ρ is some source density and u is some potential that we are seeking to solve for. For electrostatic problems this specialises to

$$\nabla^2 \phi(\vec{x}) = -\rho_c(\vec{x})/\epsilon_o,$$

where ϕ and ρ_c are the electrostatic potential and charge-density, respectively, and for (Newtonian) gravitational problems

$$\nabla^2 \Phi(\vec{x}) = 4\pi G \rho(\vec{x})$$

where Φ is gravitational potential, G is Newton's constant and ρ is the mass density of matter.

The iterative method covered in Section 12 works for any boundary conditions; periodic, fixed, etc. However, **for the case of periodic BCs**, using FFTs is much more efficient.

The exact analytical solution can easily be written in terms of FTs as follows. As we have seen the Laplacian is replaced by the factor $-|\vec{k}|^2$ in Fourier space so that exact solution in Fourier space is

$$\tilde{u}(\vec{k}) = -\frac{\tilde{\rho}(\vec{k})}{|\vec{k}|^2}. \tag{12.71}$$

To obtain the exact solution in real (coordinate) space we just need to inverse transform the above

$$u(\vec{x}) = \mathcal{F}^{-1}\left[-\frac{\tilde{\rho}(\vec{k})}{|\vec{k}|^2}\right]. \tag{12.72}$$

In practice for general $\rho(\vec{x})$, tractable, analytic expressions for the Fourier transform integrals will be difficult (even impossible) to find.

Equipped with a discrete Fourier transform, the way to solve (12.70) numerically (in, e.g., 2D) would be to take an FFT of the gridded source density $\rho_{n,m}$ to get $\tilde{\rho}_{p,q}$, divide by $|\vec{k}_{p,q}|^2$ (where $(k_x)_p = p\pi/\Delta x$ and $(k_y)_q = q\pi/\Delta y$) and then take the backwards DFT to get $u_{n,m}$.

12.4.3 Application to discrete system – 1-D

The above procedure does not yield exactly the same solution as solving the discretised PDE, as we did in section 12.4 (i.e. equation (12.69)). The reason is that we have not taken into account the finite difference approximation of the Laplacian but this can be done.

We discretise the system as usual with grid spacing h and use a second order finite difference scheme for the Laplacian

$$\frac{u_{j+1} - 2u_j + u_{j-1}}{h^2} = \rho_j. \tag{12.73}$$

Expanding each term through the backwards DFT we have

$$u_j = \frac{1}{N}\sum_p \tilde{u}_p e^{-i2\pi pj/N}, \tag{12.74}$$

and

$$u_{j\pm1} = \frac{1}{N}\sum_p \tilde{u}_p e^{-i2\pi p(j\pm1)/N} = \frac{1}{N}\sum_p \tilde{u}_p e^{\mp i2\pi p/N} e^{-i2\pi pj/N}. \tag{12.75}$$

Putting these into (12.73) gives

$$\frac{1}{N}\sum_p \tilde{u}_p e^{-i2\pi pj/N}\left[e^{+i2\pi p/N}+e^{-i2\pi p/N}-2\right]=\frac{h^2}{N}\sum_p \tilde{\rho}_p e^{-i2\pi pj/N}. \tag{12.76}$$

Equating the bits inside the sum we have

$$\tilde{u}_p=\frac{h^2\tilde{\rho}_p}{\left[e^{+i2\pi p/N}+e^{-i2\pi p/N}-2\right]}, \tag{12.77}$$

and since

$$e^{i\theta}=\cos\theta+i\sin\theta, \tag{12.78}$$

the denominator simplifies to

$$\left[e^{+i2\pi p/N}+e^{-i2\pi p/N}-2\right]=\left[e^{+i\pi p/N}-e^{-i\pi p/N}\right]^2=(2i)^2\sin^2(\pi p/N), \tag{12.79}$$

giving

$$\tilde{u}_p=-\frac{h^2\tilde{\rho}_p}{4\sin^2(\pi p/N)} \tag{12.80}$$

Notice that the 'monopole' $\tilde{u}_0$ diverges if $\tilde{\rho}_0\neq 0$. (Note that $\tilde{\rho}_0$ is the average source density on the finite, periodically repeated, domain.) Physically, the potential is only defined to within a constant so we can safely set $\tilde{\rho}_0=0$ to make the potential in real space zero when for a system where all the 'mass' is uniformly spread out. We then obtain the solution in coordinate space by taking the inverse DFT of (12.80)

$$u_j=-\frac{h^2}{4N}\sum_p \frac{\tilde{\rho}_p}{\sin^2(\pi p/N)}e^{-i2\pi pkj/N}. \tag{12.81}$$

Note that the sine argument $\pi p/N=hk/2$ so for small hk, $4\sin(\pi p/N)\approx h^2k^2$ which reproduces the previous result for $\tilde{u}_p$. Hence, these agree in the limit of long wavelengths where the discrete derivative approximation works well.

In general the DFTs are replaced by black-box FFT routines from standard libraries and the algorithm can be summarised as

- Generate source lattice ρ_j.
- FFT source lattice $\rho_j\to\tilde{\rho}_p$.
- Solve for $\tilde{u}_p$ (and remove monopole).
- Inverse FFT $\tilde{u}_p\to u_j$.

The method can be easily extended to higher dimensions, e.g., 2D or 3D lattice.

12.4.4 Derivative boundary conditions

Consider the case where we have conditions on the derivatives of u at the boundaries

$$u'_{i,j}=Q_{i,j} \tag{12.82}$$

i.e. Neumann boundary conditions. As we did in section (11.4), we will approximate the derivative as a central difference at the boundary involving a fictitious point. This is the same concept as discussed previously for the one spatial dimension initial value problems. Below, we illustrate derivative boundary conditions in y, i.e., applied at $y=0$ and $y=L_y$. Dirichlet conditions are still

used in x. The setup is as follows for $N_x = N_y = 4$ interior points to be solved for:

$$\begin{array}{ccccc}
C_{0,4} & u'_{1,4} = Q_{1,4} & u'_{2,4} = Q_{2,4} & u'_{3,4} = Q_{3,4} & C_{4,4} \\
* & * & * & * & * \\
C_{0,3} & u_{1,3} & u_{2,3} & u_{3,3} & C_{4,3} \\
* & \circ & \circ & \circ & * \\
C_{0,2} & u_{1,2} & u_{2,2} & u_{3,2} & C_{4,2} \\
* & \circ & \circ & \circ & * \\
C_{0,1} & u_{1,1} & u_{2,1} & u_{3,1} & C_{4,1} \\
* & \circ & \circ & \circ & * \\
C_{0,0} & u'_{1,0} = Q_{1,0} & u'_{2,0} = Q_{2,0} & u'_{3,0} = Q_{3,0} & C_{4,0} \\
* & * & * & * & *
\end{array} \tag{12.83}$$

(Notice that we have defined the corner points to be Dirichlet although they do not affect the solution at this order; generally they could be defined to be either Dirichlet or Neumann.) We then create fictitious points next to the top and bottom boundaries

$$\begin{array}{ccccc}
 & u_{1,5} & u_{2,5} & u_{3,5} & \\
 & * & * & * & \\
C_{0,4} & u_{1,4} & u_{2,4} & u_{3,4} & C_{4,4} \\
* & \circ & \circ & \circ & * \\
C_{0,3} & u_{1,3} & u_{2,3} & u_{3,3} & C_{4,3} \\
* & \circ & \circ & \circ & * \\
C_{0,2} & u_{1,2} & u_{2,2} & u_{3,2} & C_{4,2} \\
* & \circ & \circ & \circ & * \\
C_{0,1} & u_{1,1} & u_{2,1} & u_{3,1} & C_{4,1} \\
* & \circ & \circ & \circ & * \\
C_{0,0} & u_{1,0} & u_{2,0} & u_{3,0} & C_{4,0} \\
* & \circ & \circ & \circ & * \\
 & u_{1,-1} & u_{2,-1} & u_{3,-1} & \\
 & * & * & *
\end{array} . \tag{12.84}$$

This system has $m = (N_x - 1) \times (N_y + 1)$ unknowns. The fictitious points then become the boundaries and can be initialised using the definition of the central difference scheme. For the lower boundary at $y = 0$ this yields

$$\left.\frac{\partial u}{\partial y}\right|_{i,0} = \frac{u_{i,1} - u_{i,-1}}{2h} = Q_{i,0} \qquad \text{giving} \qquad u_{i,-1} = u_{i,1} - 2h\,Q_{i,0}. \tag{12.85}$$

A similar condition needs to be applied at $y = L_y$ to give u_{i,N_y+1} in terms of Q_{i,N_y}. This system of simultaneous equations forms an $m \times m = (N_x - 1)(N_y + 1) \times (N_x - 1)(N_y + 1)$ matrix equation, which can then be solved iteratively as before, taking care to update the fictitious points along with the 'real' ones at each iteration.